Covariance and Correlation
Class 7, 18.05
Jeremy Orloff and Jonathan Bloom

1 Learning Goals

1. Understand the meaning of covariance and correlation.
2. Be able to compute the covariance and correlation of two random variables.

2 Covariance

Covariance is a measure of how much two random variables vary together. For example, height and weight of giraffes have positive covariance because when one is big the other tends also to be big.

Definition: Suppose X and Y are random variables with means μ_X and μ_Y. The covariance of X and Y is defined as

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)].$$

2.1 Properties of covariance

1. $\text{Cov}(aX + b, cY + d) = ac\text{Cov}(X, Y)$ for constants a, b, c, d.
2. $\text{Cov}(X_1 + X_2, Y) = \text{Cov}(X_1, Y) + \text{Cov}(X_2, Y)$.
3. $\text{Cov}(X, X) = \text{Var}(X)$
4. $\text{Cov}(X, Y) = E[XY] - \mu_X\mu_Y$.
5. $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$ for any X and Y.
6. If X and Y are independent then $\text{Cov}(X, Y) = 0$.
   Warning: The converse is false: zero covariance does not always imply independence.

Notes. 1. Property 4 is like the similar property for variance. Indeed, if $X = Y$ it is exactly that property: $\text{Var}(X) = E[X^2] - \mu_X^2$.

By Property 5, the formula in Property 6 reduces to our earlier formula $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$ when X and Y are independent.

We give the proofs below. However, understanding and using these properties is more important than memorizing their proofs.

2.2 Sums and integrals for computing covariance

Since covariance is defined as an expected value we compute it in the usual way as a sum or integral.

Discrete case: If X and Y have joint pmf $p(x_i, y_j)$ then

$$\text{Cov}(X,Y) = \sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)(x_i-\mu_X)(y_j-\mu_Y) = \left(\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)x_iy_j\right) - \mu_X\mu_Y.$$

Continuous case: If X and Y have joint pdf $f(x,y)$ over range $[a,b]\times[c,d]$ then

$$\text{Cov}(X,Y) = \int_c^d\int_a^b (x-\mu_x)(y-\mu_y)f(x,y)\,dx\,dy = \left(\int_c^d\int_a^b xyf(x,y)\,dx\,dy\right) - \mu_x\mu_y.$$

2.3 Examples

Example 1. Flip a fair coin 3 times. Let X be the number of heads in the first 2 flips and let Y be the number of heads on the last 2 flips (so there is overlap on the middle flip). Compute $\text{Cov}(X,Y)$.

Solution: We'll do this twice, first using the joint probability table and the definition of covariance, and then using the properties of covariance.

With 3 tosses there are only 8 outcomes {HHH, HHT,...}, so we can create the joint probability table directly.

$X\backslash Y$	0	1	2	$p(x_i)$
0	1/8	1/8	0	1/4
1	1/8	2/8	1/8	1/2
2	0	1/8	1/8	1/4
$p(y_j)$	1/4	1/2	1/4	1

From the marginals we compute $E[X] = 1 = E[Y]$. Now we use use the definition:

$$\text{Cov}(X,Y) = E[(X-\mu_x)(Y-\mu_Y)] = \sum_{i,j} p(x_i,y_j)(x_i-1)(y_j-1)$$

We write out the sum leaving out all the terms that are 0, i.e. all the terms where $x_i = 1$ or $y_i = 1$ or the probability is 0.

$$\text{Cov}(X,Y) = \frac{1}{8}(0-1)(0-1) + \frac{1}{8}(2-1)(2-1) = \frac{1}{4}.$$

We could also have used property 4 to do the computation: From the full table we compute

$$E[XY] = 1\cdot\frac{2}{8} + 2\frac{1}{8} + 2\frac{1}{8} + 4\frac{1}{8} = \frac{5}{4}.$$

So $\text{Cov}(XY) = E[XY] - \mu_X\mu_Y = \dfrac{5}{4} - 1 = \dfrac{1}{4}$.

Next we redo the computation of $\text{Cov}(X, Y)$ using the properties of covariance. As usual, let X_i be the result of the ith flip, so $X_i \sim \text{Bernoulli}(0.5)$. We have

$$X = X_1 + X_2 \quad \text{and} \quad Y = X_2 + X_3.$$

We know $E[X_i] = 1/2$ and $\text{Var}(X_i) = 1/4$. Therefore using Property 2 of covariance, we have

$$\text{Cov}(X, Y) = \text{Cov}(X_1+X_2, X_2+X_3) = \text{Cov}(X_1, X_2)+\text{Cov}(X_1, X_3)+\text{Cov}(X_2, X_2)+\text{Cov}(X_2, X_3).$$

Since the different tosses are independent we know

$$\text{Cov}(X_1, X_2) = \text{Cov}(X_1, X_3) = \text{Cov}(X_2, X_3) = 0.$$

Looking at the expression for $\text{Cov}(X, Y)$ there is only one non-zero term

$$\text{Cov}(X, Y) = \text{Cov}(X_2, X_2) = \text{Var}(X_2) = \boxed{\frac{1}{4}}.$$

Example 2. (Zero covariance does not imply independence.) Let X be a random variable that takes values $-2, -1, 0, 1, 2$; each with probability $1/5$. Let $Y = X^2$. Show that $\text{Cov}(X, Y) = 0$ but X and Y are not independent.

Solution: We make a joint probability table:

$Y \backslash X$	-2	-1	0	1	2	$p(y_j)$
0	0	0	1/5	0	0	1/5
1	0	1/5	0	1/5	0	2/5
4	1/5	0	0	0	1/5	2/5
$p(x_i)$	1/5	1/5	1/5	1/5	1/5	1

Using the marginals we compute means $E[X] = 0$ and $E[Y] = 2$.

Next we show that X and Y are not independent. To do this all we have to do is find one place where the product rule fails, i.e. where $p(x_i, y_j) \neq p(x_i)p(x_j)$:

$$P(X = -2,\, Y = 0) = 0 \quad \text{but} \quad P(X = -2) \cdot P(Y = 0) = 1/25.$$

Since these are not equal X and Y are not independent. Finally we compute covariance using Property 4:

$$\text{Cov}(X, Y) = \frac{1}{5}(-8 - 1 + 1 + 8) - \mu_X\mu_y = 0.$$

Discussion: This example shows that $\text{Cov}(X, Y) = 0$ does not imply that X and Y are independent. In fact, X and X^2 are as dependent as random variables can be: if you know the value of X then you know the value of X^2 with 100% certainty.

The key point is that $\text{Cov}(X, Y)$ measures the linear relationship between X and Y. In the above example X and X^2 have a quadratic relationship that is completely missed by $\text{Cov}(X, Y)$.

Continuous covariance works the same way, except our computations are done with integrals instead of sums. Here is an example.

Example 3. Continuous covariance. Suppose X and Y are jointly distributed random variables, with range on the unit square $[0, 1] \times [0, 1]$ and joint pdf $f(x, y) = 2x^3 + 2y^3$.

(i) Verify the $f(x, y)$ is a valid probability density.

(ii) Compute μ_X and μ_Y.

(iii) Compute the covariance of $\text{Cov}(X, Y)$

Solution: Part of the point of this example is to show how to set up and compute the integrals using a joint density function. Since the pdf here is a polynomial, these computations are relatively easy.

(i) A valid pdf has two properties: it is nonnegative and the total integral over the entire joint range is 1.

Nonnegativity is clear: $f(x, y) \geq 0$. The integral is not hard to compute

$$\int_0^1 \int_0^1 f(x, y)\, dx\, dy = \int_0^1 \int_0^1 2x^3 + 2y^3\, dx\, dy$$

$$\text{Inner integral: } \int_0^1 2x^3 + 2y^3\, dx = \frac{x^4}{2} + 2xy^3 \Big|_0^1 = \frac{1}{2} + 2y^3.$$

$$\text{Outer integral: } \int_0^1 \frac{1}{2} + 2y^3\, dy = \frac{y}{2} + \frac{y^4}{2} \Big|_0^1 = 1.$$

So, the integral over the entire joint range is 1. Thus, $f(x, y) = x + y$ is a valid probability density.

(ii) We need to compute integrals to find the means. We will write down the integrals, but not show the details of their computation. (Also, by symmetry, we know the two means are the same.)

$$\mu_X = \int_0^1 \int_0^1 x f(x, y)\, dx\, dy = \int_0^1 \int_0^1 2x^4 + 2xy^3)\, dx\, dy = \frac{13}{20}$$

$$\mu_Y = \int_0^1 \int_0^1 y f(x, y)\, dx\, dy = \int_0^1 \int_0^1 2yx^3 + 2y^4\, dx\, dy = \frac{13}{20}$$

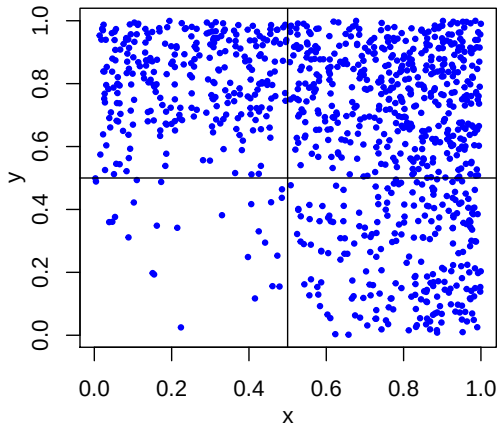
(iii) We know $\text{Cov}(X, Y) = E\left[(X - \mu_x)(Y - \mu_Y)\right]$. This is an integral. Again, we will write down the integral, but not show details of its computation,

$$\text{Cov}(X, Y) = E\left[(X - \mu_x)(Y - \mu_Y)\right] = \int_0^1 \int_0^1 (x - 13/20)(y - 13/20) f(x, y)\, dx\, dy$$

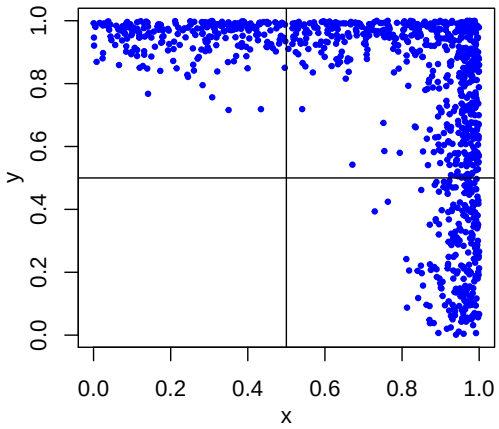
$$= \int_0^1 \int_0^1 (x - 7/12)(y - 7/12)(2x^3 + 2y^3)\, dx\, dy = -\frac{9}{400}$$

(In fact, we wrote down the integral in the most straightforward way, but secretly we did the computation by computing $E[XY] - E[X]E[Y]$.)

Here's a plot of the pseudo-random samples generated from this distribution. Because the R code could do it easily, we also include a plot with a more extreme density function.



Samples from $f(x, y) = 2x^3 + 2y^3$.



Samples from $f(x, y) = 10x^{19} + 10y^{19}$.

3 Correlation

The units of covariance $\mathrm{Cov}(X, Y)$ are 'units of X times units of Y'. This makes it hard to compare covariances: if we change scales then the covariance changes as well. Correlation is a way to remove the scale from the covariance.

Definition: The correlation coefficient between X and Y is defined by

$$\mathrm{Cor}(X, Y) = \rho = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \, \sigma_Y}.$$

3.1 Properties of correlation

1. ρ is the covariance of the standardizations of X and Y.
2. ρ **is dimensionless** (it's a ratio!).
3. $-1 \le \rho \le 1$. Furthermore,
   $\rho = +1$ if and only if $Y = aX + b$ with $a > 0$,
   $\rho = -1$ if and only if $Y = aX + b$ with $a < 0$.

Property 3 shows that ρ measures the linear relationship between variables. If the correlation is positive then when X is large, Y will tend to large as well. If the correlation is negative then when X is large, Y will tend to be small.

Example 2 above shows that correlation can completely miss higher order relationships.

Example 4. We continue Example 1. To compute the correlation we divide the covariance by the standard deviations. In Example 1 we found $\mathrm{Cov}(X, Y) = 1/4$ and $\mathrm{Var}(X) =$

$2\text{Var}(X_j) = 1/2$. So, $\sigma_X = 1/\sqrt{2}$. Likewise $\sigma_Y = 1/\sqrt{2}$. Thus

$$\text{Cor}(X,Y) = \frac{\text{Cov}(X,Y)}{\sigma_X\sigma_Y} = \frac{1/4}{1/2} = \frac{1}{2}.$$

We see a positive correlation, which means that larger X tend to go with larger Y and smaller X with smaller Y. In Example 1 this happens because toss 2 is included in both X and Y, so it contributes to the size of both.

Example 5. Look back at Example 3. See if you can compute the following.

$$\text{Var}(X) = 31/400, \text{ so } \sigma_X = \sqrt{31/400} \approx 0.28$$
$$\text{Var}(Y) = \text{Var}(X), \text{ so } \sigma_Y \approx 0.28$$
$$\text{Cor}(X,Y) = \frac{\text{Cov}(X,Y)}{\sigma_X\sigma_Y} \approx -0.29.$$

3.2 Bivariate normal distributions

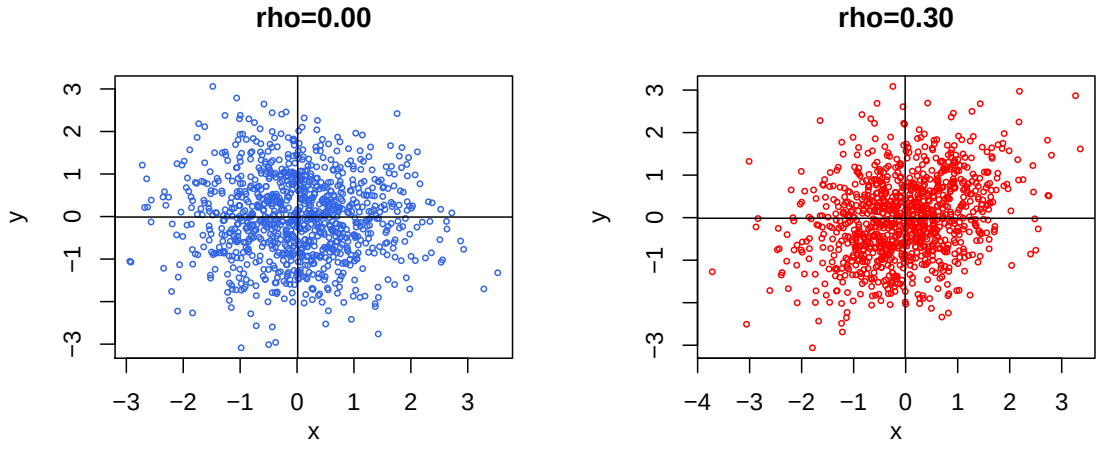
The bivariate normal distribution has density

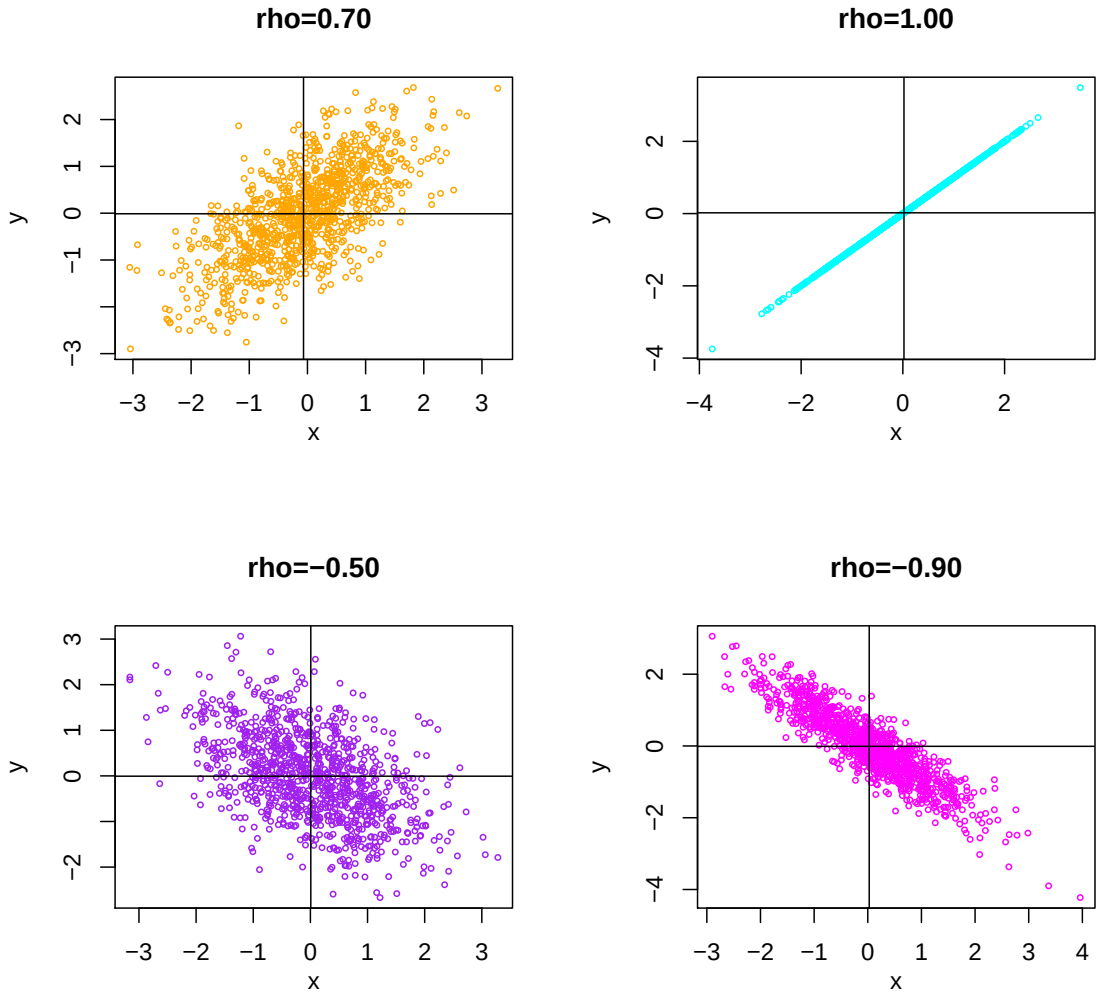
$$f(x,y) = \frac{e^{\frac{-1}{2(1-\rho^2)}\left[\frac{(x-\mu_X)^2}{\sigma_X^2} + \frac{(y-\mu_Y)^2}{\sigma_Y^2} - \frac{2\rho(x-\mu_x)(y-\mu_y)}{\sigma_x\sigma_y}\right]}}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho^2}}$$

For this distribution, the marginal distributions for X and Y are normal and the correlation between X and Y is ρ.

In the figures below we used R to simulate the distribution for various values of ρ. Individually X and Y are standard normal, i.e. $\mu_X = \mu_Y = 0$ and $\sigma_X = \sigma_Y = 1$. The figures show scatter plots of the results.

These plots and the next set show an important feature of correlation. We divide the data into quadrants by drawing a horizontal and a verticle line at the means of the y data and x data respectively. A positive correlation corresponds to the data tending to lie in the 1st and 3rd quadrants. A negative correlation corresponds to data tending to lie in the 2nd and 4th quadrants. You can see the data gathering about a line as ρ becomes closer to ± 1.

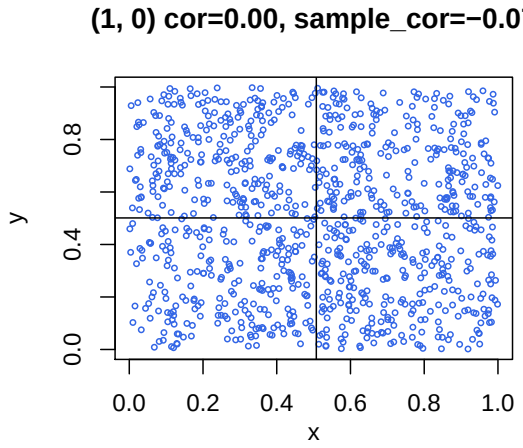


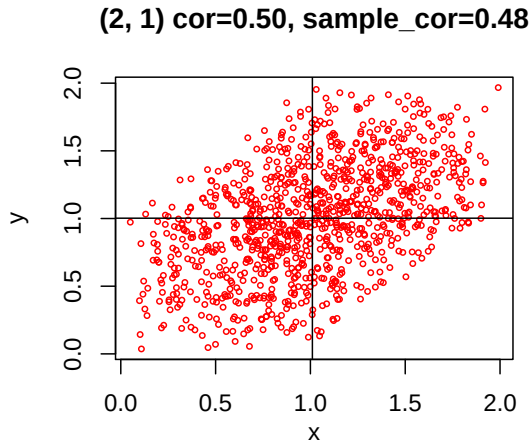


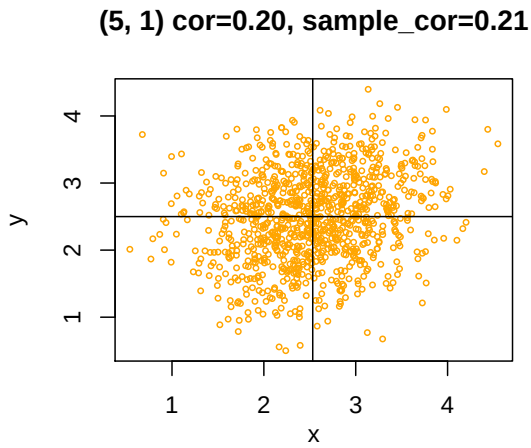
3.3 Overlapping uniform distributions

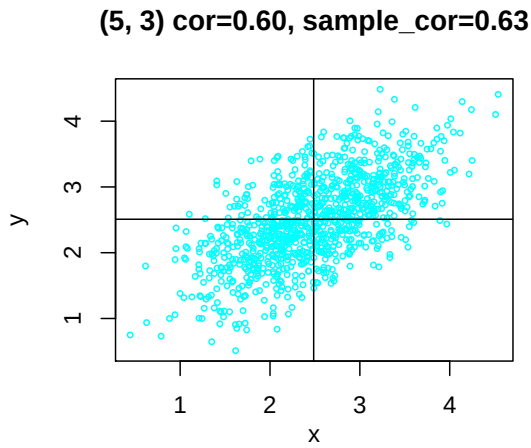
We ran simulations in R of the following scenario. X_1, X_2, ..., X_{20} are i.i.d and follow a $U(0,1)$ distribution. X and Y are both sums of the same number of X_i. We call the number of X_i common to both X and Y the overlap. The notation in the figures below indicates the number of X_i being summed and the number which overlap. For example, 5,3 indicates that X and Y were each the sum of 5 of the X_i and that 3 of the X_i were common to both sums. (The data was generated using `rand(1,1000);`)

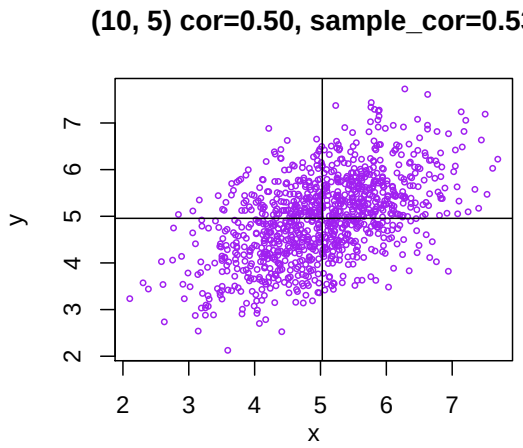
Using the linearity of covariance it is easy to compute the theoretical correlation. For each plot we give both the theoretical correlation and the correlation of the data from the simulated sample.

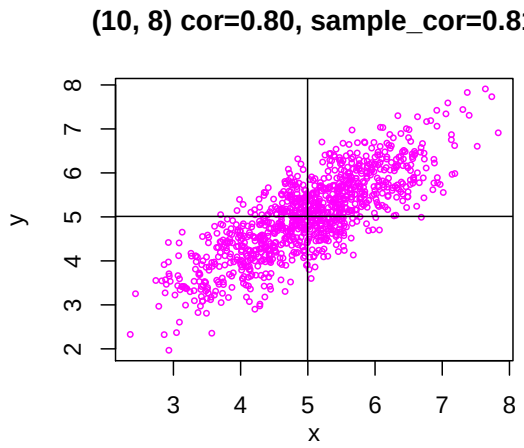

(1, 0) cor=0.00, sample_cor=−0.07
y
x
0.0
0.2
0.4
0.6
0.8
1.0


(2, 1) cor=0.50, sample_cor=0.48
y
x
0.0
0.5
1.0
1.5
2.0


(5, 1) cor=0.20, sample_cor=0.21
y
x
1
2
3
4


(5, 3) cor=0.60, sample_cor=0.63
y
x
1
2
3
4


(10, 5) cor=0.50, sample_cor=0.53
y
x
2
3
4
5
6
7


(10, 8) cor=0.80, sample_cor=0.81
y
x
2
3
4
5
6
7
8

4 Proof of the properties of covariance and correlation

4.1 Proofs of the properties of covariance

1 and 2 follow from similar properties for expected value.

3. This is the definition of variance:

$$\text{Cov}(X, X) = E[(X - \mu_X)(X - \mu_X)] = E[(X - \mu_X)^2] = \text{Var}(X).$$

4. Recall that $E[X - \mu_x] = 0$. So

$$\begin{aligned}\text{Cov}(X, Y) &= E[(X - \mu_X)(Y - \mu_Y)] \\ &= E[XY - \mu_X Y - \mu_Y X + \mu_X \mu_Y] \\ &= E[XY] - \mu_X E[Y] - \mu_Y E[X] + \mu_X \mu_Y \\ &= E[XY] - \mu_X \mu_Y - \mu_X \mu_Y + \mu_X \mu_Y \\ &= E[XY] - \mu_X \mu_Y.\end{aligned}$$

5. Using properties 3 and 2 we get

$$\text{Var}(X+Y) = \text{Cov}(X+Y, X+Y) = \text{Cov}(X, X)+2\text{Cov}(X, Y)+\text{Cov}(Y, Y) = \text{Var}(X)+\text{Var}(Y)+2\text{Cov}(X, Y).$$

6. If X and Y are independent then $f(x, y) = f_X(x) f_Y(y)$. Therefore

$$\begin{aligned}\text{Cov}(X, Y) &= \int\int (x - \mu_X)(y - \mu_Y) f_X(x) f_Y(y)\, dx\, dy \\ &= \int (x - \mu_X) f_X(x)\, dx \int (y - \mu_Y) f_Y(y)\, dy \\ &= E[X - \mu_X] E[Y - \mu_Y] \\ &= 0.\end{aligned}$$

4.2 Proof of Property 3 of correlation

(*This is for the mathematically interested.*)

$$0 \le \text{Var}\left(\frac{X}{\sigma_X} - \frac{Y}{\sigma_Y}\right) = \text{Var}\left(\frac{X}{\sigma_X}\right) + \text{Var}\left(\frac{Y}{\sigma_Y}\right) - 2\text{Cov}\left(\frac{X}{\sigma_X}, \frac{Y}{\sigma_Y}\right) = 2 - 2\rho$$

This implies $\rho \le 1$

Likewise $0 \le \text{Var}\left(\dfrac{X}{\sigma_X} + \dfrac{Y}{\sigma_Y}\right)$, so $-1 \le \rho$.

If $\rho = 1$ then $0 = \text{Var}\left(\dfrac{X}{\sigma_X} - \dfrac{Y}{\sigma_Y}\right) \Rightarrow \dfrac{X}{\sigma_X} - \dfrac{Y}{\sigma_Y} = c.$ ∎

MIT OpenCourseWare
https://ocw.mit.edu

18.05 Introduction to Probability and Statistics
Spring 2022

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms.

FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 2

CORRELATION AND MULTICOLLINEARITY

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Linear Regression
Keywords	Covariance and Correlation, Positively correlated variables, Negative correlated variables, Uncorrelated variables, Collinearity & Multicollinearity, Correlation Matrix, Variance Inflation Factor (VIF)

In the last lesson, we learned the basic steps to use linear regression models to analyze KO's stock excess return. We first explained the types of variables in the model and the tools to check variable correlation. We then built a linear regression model and tested the fit of the model by using various metrics and plots.

In this lesson, we are going to explore the relationship among variables in more detail. Specifically, we would like to investigate what would happen to a regression model if the model's exogenous variables are highly correlated. We will propose some methods to reduce the impact of correlated exogenous variables. We will continue to use KO's stock excess return project as our example.

In [1]:
```
# Libraries
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

plt.rcParams["figure.figsize"] = (12, 9)
```

In [2]:
```
# Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

1. Expanding Independent Variables to Study Correlation

We are going to add more exogenous variables (independent variables) into our *KO* stock excess return model. We selected five stocks to add to the regression model in

order to improve our model performance. The five stocks are:

- Google
- Bank of America
- Walmart
- General Electric
- Starbucks

We're following the last lesson's method to get the quarterly excess returns for the five stocks and then add them to the regression model. Hence, we are expanding our model to include eight variables. Here is the list of the variables:

- Dependent Variable: *KO* quarterly excess return (*KO_Q_EX_R*)
- Independent Variable: *Dow 30* Index quarterly excess return (*Dow_Q_EX_R*)
- Pepsi quarterly excess return (*Pepsi_Q_EX_R*)
- *KO* quarterly revenue return (*REV_Q_R*)
- Google quarterly excess return (*GOOG_Q_R*)
- Bank of America quarterly excess return (*BAC_Q_R*)
- Walmart quarterly excess return (*WMT_Q_R*)
- General Electric quarterly excess return (*GE_Q_R*)
- Starbucks quarterly excess return (*SBUX_Q_R*)

2. Covariance and Correlation

In finance, we are particularly interested in understanding how two variables interact with each other. One popular question is the following: How do two stock returns move together? An answer to this question is provided by the covariance and correlation between two stock returns. **Covariance** is a metric to measure the amount of movement the two variables exhibit. Here is the covariance formula:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

- If the covariance has a positive sign, it means the two variables move in the same direction.
- If the covariance has a negative sign, the two variables move in opposite directions.
- If the covariance is 0, the two variables are **linearly uncorrelated (uncorrelated).**

The higher the absolute value of the covariance of the two variables, the stronger the (positive or negative) relationship the two variables have. The downside of covariance is its value changes when the scales of two variables change. Because of this issue, we use correlation more often in data analysis.

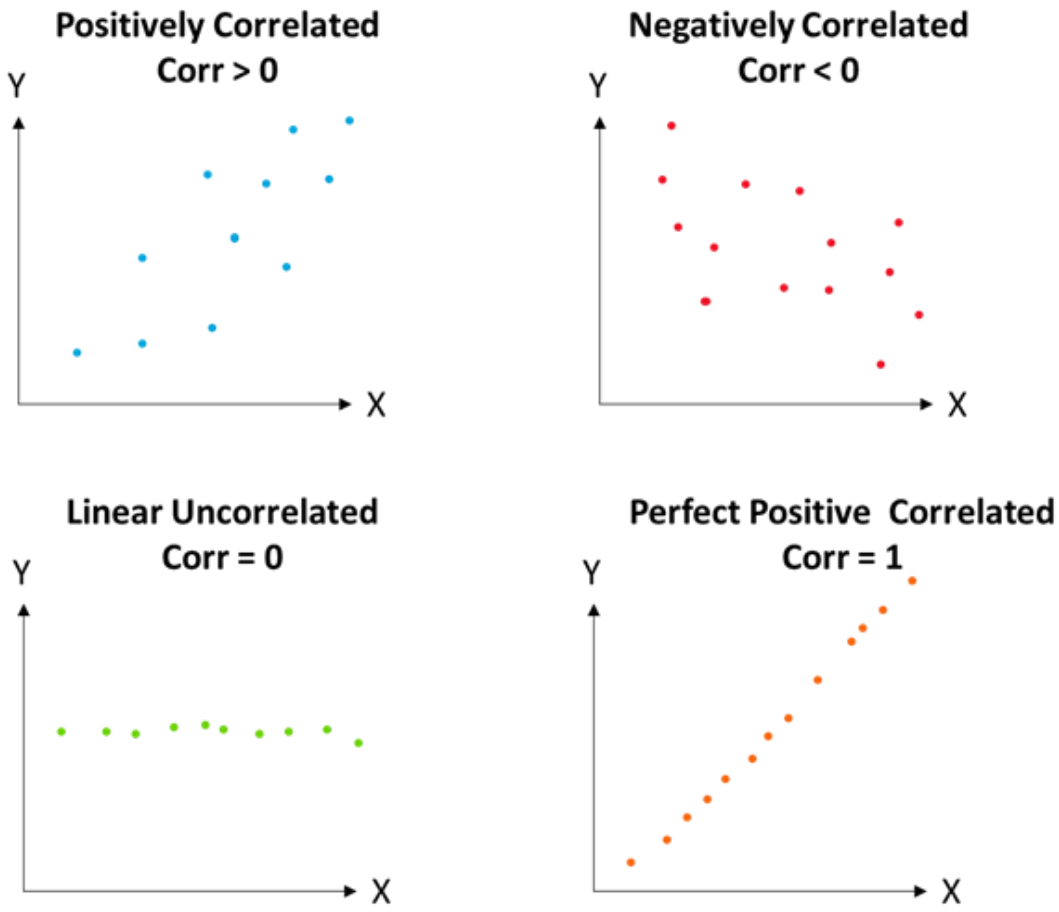
Correlation is also a metric to measure the co-movement of the two variables. However, it eliminates the scale issue mentioned above by dividing covariance with the square root of the multiplication of the two variables' variances. Here is the correlation math formula:

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}$$

- Unlike covariance, the value of correlation is limited between -1 and 1.
- If the correlation of two variables is greater than 0, the two variables are **positively correlated**.
- If the correlation of two variables is less than 0, the two variables are **negatively correlated**.
- If two variables are **perfectly positively correlated**, the correlation will be 1.
- If two variables are **perfectly negative correlated**, the correlation will be -1.
- If the correlation is 0, the two variables are **linearly uncorrelated**.

In Figure 1, we illustrate the relationship of two variables based on their correlation in graphs.

Figure 1: Correlation Demonstration Graphs

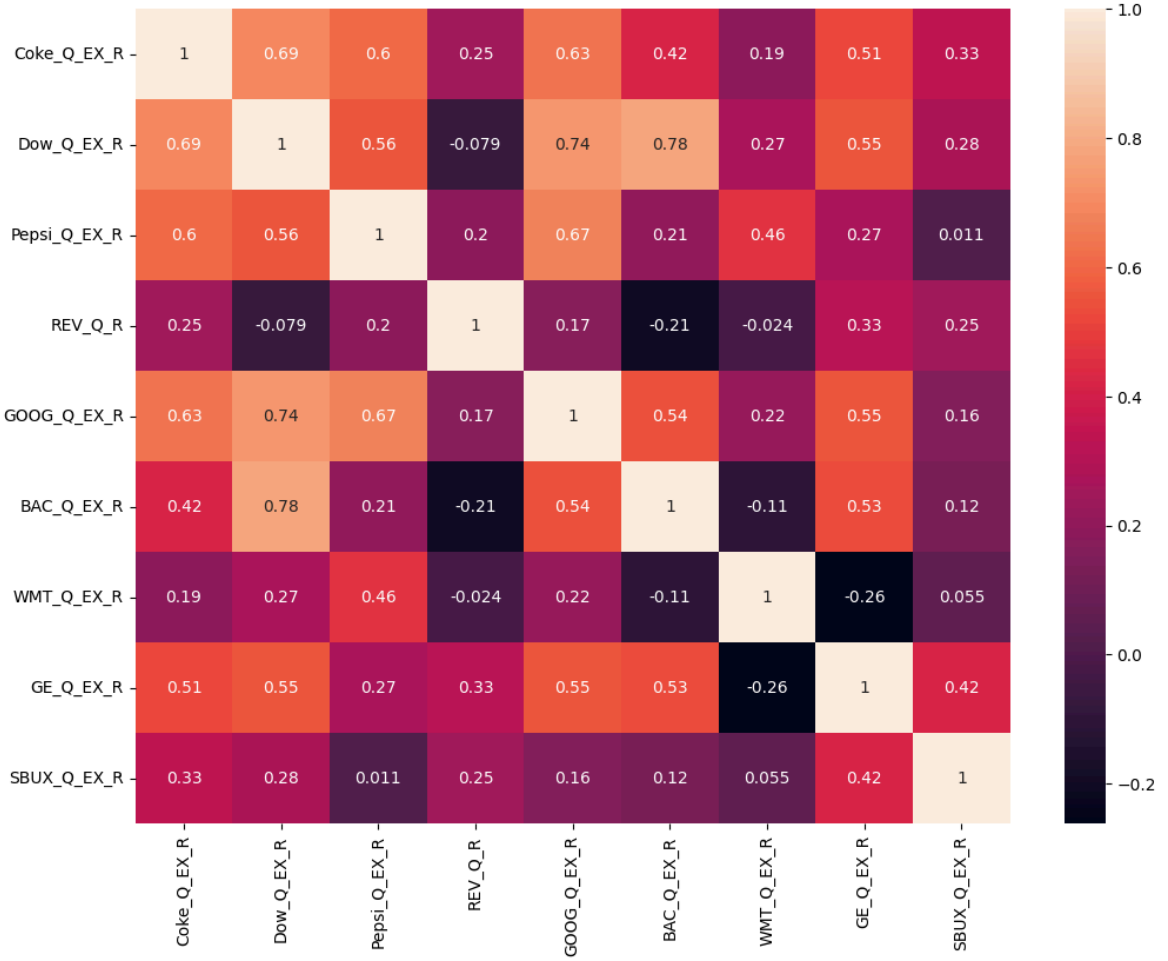


3. Collinear Variables

Most of the financial data we use is collected from various sources and is out of our control in terms of how it is generated. Because we cannot control how financial data is generated, oftentimes, many financial variables move together in systematic ways. Such a problem is called **collinearity** for two variables. We also call the two variables **correlated variables**. If there are more than two variables that are correlated, it is called **multicollinearity**. How can we detect multicollinearity?

A **correlation matrix** is the best way to investigate multicollinearity. A correlation matrix is an ensemble of the correlations of all variables. Let's look at our KO stock excess return model to understand the impact of multicollinearity. Figure 2 is the correlation matrix for the dependent variable and independent variables in our model. The correlation of different variables is shown on the upper right triangle of the correlation matrix. The diagonal of the correlation matrix is the correlation for the variable itself, which is always 1. The lower left triangle of the correlation matrix actually provides the same correlation information as the upper right triangle.

In [3]:
```
# Correlation plot of all the variables
data = data_set[
    [
        "Coke_Q_EX_R",
        "Dow_Q_EX_R",
        "Pepsi_Q_EX_R",
        "REV_Q_R",
        "GOOG_Q_EX_R",
        "BAC_Q_EX_R",
        "WMT_Q_EX_R",
        "GE_Q_EX_R",
        "SBUX_Q_EX_R",
    ]
]
c = data.corr()
sns.heatmap(c, annot=True)
plt.show()
```



Now let's look at the numbers in the correlation matrix.

3.1 Dependent Variable and Independent Variables

As seen in the matrix, let's examine our dependent variable. *KO*'s excess return is fairly positively correlated with three independent variables: *Dow 30* Index excess return, Pepsi excess return, and Google excess return. Therefore, we can see that these three variables are pretty good at explaining *KO*'s excess return variation.

3.2 Low-Correlated Independent Variables

Now let's turn our focus just on independent variables. First, we'll first look at *KO*'s revenue growth, Walmart excess return and Starbucks' excess return. They are the 3 variables that have low correlation with other independent variables. However, they also have low correlation with our dependent variable: *KO* excess return.

3.3 High-Correlated Independent Variables

In the matrix, the *Dow 30* Index excess return has a high correlation with Google's excess return and Bank of America's excess return. Google's excess return also has a high correlation with Pepsi's excess return. With these four fairly correlated independent variables in our model, will they present any issues to our regression result?

4. Impact of Multicollinearity in Regression Model

1. If independent variables are highly correlated, the variances, standard errors and covariances of the coefficients from the regression model may be large. Then, the estimates of the coefficient will be less precise because the confidence intervals for the coefficient estimates will be wide.
2. The high standard errors for the coefficient estimates also mean the estimates are less likely to be statistically significant. The R^2 and adjusted R^2 may be high because the whole model is still good. The collinearity issue makes it hard to separate the individual impact from correlated independent variables in the model.
3. Even though it is hard to isolate each variable's impact in the model, the model can still be a good choice to forecast if the new data from the independent variables have the same collinear issue as the sample data used to build the model.

Let's run our expanded model and see how the multicollinearity issue influences the model result.

Figure 3: KO Excess Return Regression Model with Eight Independent Variables

In [4]:
```
# KO Excess Return Regression Model with Eight Independent Variables
model_1 = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + Pepsi_Q_EX_R + REV_Q_R + GOOG_Q_EX_R + BAC_Q_EX_
    data=data_set,
```

```
).fit()
model_1.summary()
```

Out[4]:

OLS Regression Results

Dep. Variable:	Coke_Q_EX_R	**R-squared:**	0.633
Model:	OLS	**Adj. R-squared:**	0.407
Method:	Least Squares	**F-statistic:**	2.798
Date:	Sat, 22 Nov 2025	**Prob (F-statistic):**	0.0482
Time:	21:44:59	**Log-Likelihood:**	39.396
No. Observations:	22	**AIC:**	-60.79
Df Residuals:	13	**BIC:**	-50.97
Df Model:	8		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.0059	0.017	-0.337	0.742	-0.043	0.032
Dow_Q_EX_R	0.7091	0.471	1.504	0.156	-0.309	1.727
Pepsi_Q_EX_R	0.2686	0.267	1.005	0.333	-0.309	0.846
REV_Q_R	0.1007	0.121	0.834	0.419	-0.160	0.362
GOOG_Q_EX_R	0.0233	0.208	0.112	0.912	-0.425	0.472
BAC_Q_EX_R	-0.0762	0.156	-0.487	0.634	-0.414	0.262
WMT_Q_EX_R	-0.1313	0.213	-0.617	0.548	-0.591	0.328
GE_Q_EX_R	-0.0040	0.106	-0.037	0.971	-0.234	0.226
SBUX_Q_EX_R	0.0824	0.135	0.608	0.554	-0.210	0.375

Omnibus:	1.980	**Durbin-Watson:**	2.255
Prob(Omnibus):	0.372	**Jarque-Bera (JB):**	1.638
Skew:	-0.628	**Prob(JB):**	0.441
Kurtosis:	2.543	**Cond. No.**	45.6

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [5]:
```
# Parameters with high precision
model_1.summary2().tables[1]
```

Out[5]:

	Coef.	Std.Err.	t	P>|t|	[0.025	0.975]
Intercept	-0.005859	0.017409	-0.336544	0.741832	-0.043469	0.031751
Dow_Q_EX_R	0.709085	0.471371	1.504303	0.156406	-0.309250	1.727419
Pepsi_Q_EX_R	0.268625	0.267324	1.004868	0.333299	-0.308893	0.846144
REV_Q_R	0.100740	0.120734	0.834399	0.419126	-0.160090	0.361570
GOOG_Q_EX_R	0.023328	0.207589	0.112377	0.912241	-0.425141	0.471798
BAC_Q_EX_R	-0.076156	0.156360	-0.487054	0.634329	-0.413950	0.261639
WMT_Q_EX_R	-0.131313	0.212743	-0.617238	0.547737	-0.590916	0.328290
GE_Q_EX_R	-0.003975	0.106396	-0.037365	0.970762	-0.233830	0.225879
SBUX_Q_EX_R	0.082359	0.135423	0.608162	0.553557	-0.210205	0.374923

In the above results, we can see no independent variable estimates are significant (*p-values* are all larger than 0.05). Even the *Dow 30* Index excess return is not significant in this model. This variable was significant in the model in Lesson 1. The R^2 is higher in this model (0.633) compared to the R^2 in Lesson 1's model (0.604). This is expected since we have added more independent variables to this model. However, the adjusted R^2 in this model (0.407) is significantly lower than the adjusted R^2 in the last model (0.538).

5. Methods to Address Multicollinearity

Method 1:

This first method to reduce the impact of multicollinearity is to drop independent variables that have high correlation. Usually, the correlation between 0.8 and 0.9 is considered high correlation. You can drop one of the correlated variables from the model and run it again to see if the model improves. The problem with using this method is that it only considers the relationship between two variables one at a time. This method doesn't address multicollinearity with more than two variables. Sometimes, using correlation alone cannot find this issue.

Method 2:

The second method is to treat one of the independent variables in the model as a dependent variable and run a regression with the rest of the independent variables. We can use this method to determine if any of the independent variables can be explained by the other independent variables. Let's use *KO*'s excess return model as an example.

Say we make *Dow 30* Index excess return as a dependent variable and run a regression model with the rest of the independent variables.

Figure 4: Regression Model to Check Multicollinearity Among Independent Variables

In [6]:
```
# Regression model to check multicollinearity among independent variables
model_dow = smf.ols(
    "Dow_Q_EX_R ~ Pepsi_Q_EX_R + REV_Q_R + GOOG_Q_EX_R + BAC_Q_EX_R + WMT_Q_EX_R
    data=data_set,
).fit()
model_dow.summary()
```

Out[6]:

OLS Regression Results

Dep. Variable:	Dow_Q_EX_R	**R-squared:**	0.856
Model:	OLS	**Adj. R-squared:**	0.784
Method:	Least Squares	**F-statistic:**	11.91
Date:	Sat, 22 Nov 2025	**Prob (F-statistic):**	6.13e-05
Time:	21:44:59	**Log-Likelihood:**	51.064
No. Observations:	22	**AIC:**	-86.13
Df Residuals:	14	**BIC:**	-77.40
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.0061	0.010	-0.625	0.542	-0.027	0.015
Pepsi_Q_EX_R	0.1892	0.143	1.324	0.207	-0.117	0.496
REV_Q_R	-0.0605	0.067	-0.910	0.378	-0.203	0.082
GOOG_Q_EX_R	0.0977	0.115	0.851	0.409	-0.148	0.344
BAC_Q_EX_R	0.2319	0.063	3.658	0.003	0.096	0.368
WMT_Q_EX_R	0.1898	0.109	1.734	0.105	-0.045	0.425
GE_Q_EX_R	0.0570	0.058	0.977	0.345	-0.068	0.182
SBUX_Q_EX_R	0.0800	0.074	1.084	0.297	-0.078	0.238

Omnibus:	0.530	**Durbin-Watson:**	2.372
Prob(Omnibus):	0.767	**Jarque-Bera (JB):**	0.588
Skew:	-0.043	**Prob(JB):**	0.745
Kurtosis:	2.204	**Cond. No.**	26.6

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [7]:
```
# Parameters with 6 significant digits
model_1.summary2().tables[1]
```

Out[7]:

	Coef.	Std.Err.	t	P>|t|	[0.025	0.975]
Intercept	-0.005859	0.017409	-0.336544	0.741832	-0.043469	0.031751
Dow_Q_EX_R	0.709085	0.471371	1.504303	0.156406	-0.309250	1.727419
Pepsi_Q_EX_R	0.268625	0.267324	1.004868	0.333299	-0.308893	0.846144
REV_Q_R	0.100740	0.120734	0.834399	0.419126	-0.160090	0.361570
GOOG_Q_EX_R	0.023328	0.207589	0.112377	0.912241	-0.425141	0.471798
BAC_Q_EX_R	-0.076156	0.156360	-0.487054	0.634329	-0.413950	0.261639
WMT_Q_EX_R	-0.131313	0.212743	-0.617238	0.547737	-0.590916	0.328290
GE_Q_EX_R	-0.003975	0.106396	-0.037365	0.970762	-0.233830	0.225879
SBUX_Q_EX_R	0.082359	0.135423	0.608162	0.553557	-0.210205	0.374923

Figure 4 shows the regression result. We can see from Figure 4 that R^2 and adjusted R^2 are high. It means that the information of Dow 30 Index excess return can be mostly explained by the rest of the independent variables. In statistics, the **variance inflation factor (VIF)** is used to determine if an independent variable has a multicollinearity issue. The VIF formula for each independent variable is as follows:

$$\text{VIF} = \frac{1}{1 - R^2}$$

Any software will calculate the VIF for each independent variable. The value of the VIF is from 1 and upward without limit.

- When R^2 is 0, the VIF is 1. It means there is no multicollinearity for this independent variable.
- The higher the VIF, the more multicollinearity is present for this independent variable.
- An independent variable with a VIF between 1 and 5 has no severe multicollinearity.
- An independent variable with a VIF that is higher than 5 indicates severe multicollinearity.

For an independent variable that has a high VIF, we can test the model with and without this variable and then check the model result.

Let's show the list of the VIF values of all independent variables.

In [8]:
```
labels = [
    "Intercept",
    "Dow_Q_EX_R",
    "Pepsi_Q_EX_R",
    "REV_Q_R",
    "GOOG_Q_EX_R",
    "BAC_Q_EX_R",
    "WMT_Q_EX_R",
    "GE_Q_EX_R",
    "SBUX_Q_EX_R",
]
```

```
variables = model_1.model.exog
vif = [variance_inflation_factor(variables, i) for i in range(variables.shape[1]

pd.DataFrame(vif[1:], labels[1:], columns=["VIF"])
```

Out[8]:

	VIF
Dow_Q_EX_R	6.957440
Pepsi_Q_EX_R	2.748897
REV_Q_R	1.612413
GOOG_Q_EX_R	3.241519
BAC_Q_EX_R	4.269485
WMT_Q_EX_R	2.184550
GE_Q_EX_R	3.096814
SBUX_Q_EX_R	1.511017

We can see *Dow 30* Index has a VIF value larger than 5. Since this variable has a severe multicollinearity issue, we need to dig further into this variable and maybe drop it from the model.

Method 3:

The third method we are going to discuss is principal component analysis. Principal component analysis can not only help to mitigate the issue of multicollinearity in a regression but is also an important analytical tool for other financial topics. Because principal component analysis has widespread application in finance, we will spend the next lesson discussing this topic.

6. Conclusion

In this lesson, we introduced covariance and correlation to measure the relationship of two variables. Then, we explained the meaning of collinearity and multicollinearity and how to use a correlation matrix to identify their existence among independent variables in a regression model. The existence of multicollinearity will make coefficient estimates less precise and not statistically significant. We proposed three methods to handle multicollinearity: First, check correlations between each of the independent variables and drop one of the independent variables if the correlation is higher than 0.8. Second, use the VIF metric to test independent variables with a VIF greater than 5. Third, use information from a principal component analysis.

In the next lesson, we will continue our exploration of how to mitigate multicollinearity. Specifically, we will introduce principal component analysis to tackle this issue.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 1

LINEAR REGRESSION ANALYSIS

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Undergraduate-Level Statistics
Keywords	Endogenous Variable (Dependent Variable), Exogenous Variable (Independent Variable), Scatter Plot, Outlier, Leverage Point, Ordinary Least Square Linear Regression (OLS), Parameter (Coefficient), R2, Adjusted R2, Cook's Distance (Cook's D), Influential Point, Influence Plot

In this course, we will use econometrics to analyze how financial data behaves and interacts. Econometrics uses mathematics and statistical inference to explore the relationship among different financial variables. There are several techniques and models in econometrics. In this lesson, we will start by applying linear regression analysis to demonstrate some basic concepts in econometrics. Linear regression analysis is the most frequently used analytical technique in finance and is the foundation of many more advanced econometrics theories. This lesson will also cover the basic steps of implementing linear regression analysis. These basic steps are applicable to future advanced econometrics analyses. Readers are expected to have prior knowledge of undergraduate statistics.

In [1]:
```
# Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf

plt.rcParams["figure.figsize"] = (12, 9)
```

1. Prepare a Project for Linear Regression Analysis

Assume you work for an asset management company as an equity analyst. Your manager asks you to find out what caused the recent increase in Coca Cola's stock (NYSE: KO) price and if there is still potential for the price to keep growing. In this situation, what would you do? Here are three steps to think about:

1.1 Identify Potential Factors Influencing KO's Stock Price

You can do some research to identify potential factors that impact the stock price. It could be the company's sales. It could be that the whole stock market is going up. There might be other factors that explain *KO*'s stock price increase based on your research and understanding of the company. Once you identify the factors, you can start to collect data for analysis.

1.2. Quantify the Relationship between KO's Stock Price and Selected Factors

Once the factors are determined and the data is collected, we need a system in place to tell us how each factor influences the stock price. For example, we would like to know not only if the fluctuation in sales will increase/decrease the stock price but also how much the stock price will move when the sales numbers move. We need to build a model to provide this information. In this lesson, we will use linear regression analysis to build this model. Then, we will assess how accurate the model is and adjust the model until we get a satisfactory one.

1.3 Forecast the future stock price

After we get the regression model, we can create several scenarios to predict the stock price move. For example, if *KO*'s revenue is one of the factors in the model, we can simulate how much the stock price would change given a 0.5% increase, 1% increase, and 2% increase in revenue. This exercise will help us predict *KO*'s stock price increase potential.

2. Endogenous Variable vs. Exogenous Variable

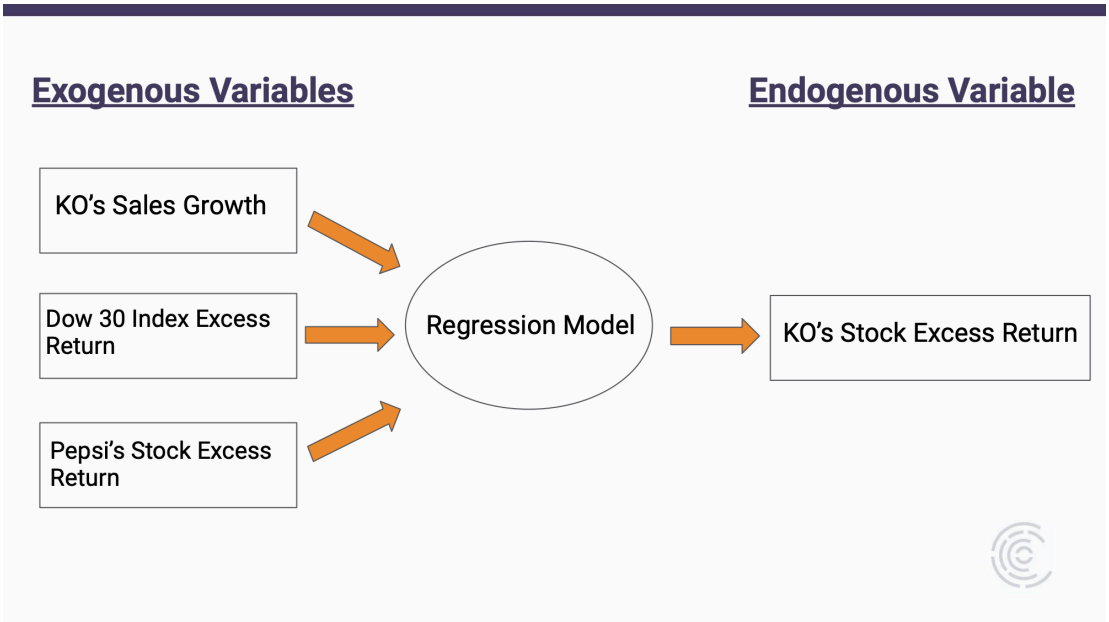
Assume you have identified that the following three factors are most impactful on *KO*'s stock price: *KO*'s revenue, Pepsi's stock price, and the Dow Jones Industrial Average Index. We will use these three factors to create a linear regression model to predict *KO*'s next-day closing stock price. Finance theory tells us a stock excess return is a better variable to analyze than the actual price. The excess return is the actual return minus risk-free interest rate (usually Treasury Bill Rates). Therefore, we will convert all our variables into returns or ratios. In our modeling dataset, we will have the following variables:

> *KO*'s stock excess return (%)
> *KO*'s sales growth (%)
> *DOW 30* Index excess return (%)
> *Pepsi*'s stock excess return (%)

In order to use comparable variables to conduct modeling work, all variables are quarterly ratios (quarterly excess returns and quarterly sales growth).

Once we have our dataset ready, we need to decide which variables are endogenous variables and which are exogenous variables. An **endogenous variable**, also called a **dependent variable**, is a variable whose value is determined by the model. An **exogenous variable**, also called an **independent variable**, is a variable whose value is determined outside of the model or is given to us. In our case, we want to build a regression model to understand and predict how *KO*'s stock excess return will move when the three factors change. *KO*'s stock excess return is an endogenous variable. The other three variables are exogenous variables because their value is not decided by the model.

Figure 1: Relationship between Exogenous Variables and Endogenous Variable



3. Check Relationship between Endogenous Variable and Exogenous Variables

After defining endogenous variables and exogenous variables for our regression model, we will use some graphical tools to investigate the relationships of our selected variables. This step is important because these graphs can give us information as to how to structure our regression model and decide if some selected variables and data points should be excluded. The graphic tool we will use is a **scatter plot**. A scatter plot is a two-dimensional graph with one variable on one axis and another variable on another axis. A scatter plot can show us the relationship of two variables. So what exactly is a scatter plot used for?

First, a scatter plot can check for the linear relationship between the endogenous variable and the exogenous variables. One of the assumptions in running regression analyses is that the endogenous variable has a linear relationship with exogenous variables. By looking at the scatter plot, we can determine if we can draw a straight line where most of the points will be around the line.

Second, a scatter plot can be used to check for independence between the endogenous variable and exogenous variables. Continuing from the previous paragraph, if the straight line drawn is either a horizontal line or a vertical line, it means no matter how much the value of one variable moves, the value of the other variable always stays within a tight range. In this case, the two variables are independent of each other. If an exogenous variable is independent of the endogenous variable, we should drop this exogenous variable from the model.

Lastly, a scatter plot can be used to find extreme points that are well-separated from the other points. There are two types of extreme points. One is called an outlier. An **outlier** is a point that doesn't follow the general trend of the other points. The other one is called a leverage point. A **leverage point** has extreme value for exogenous variables. Both types may or may not impact the estimated result of the regression model. Try to run one model with them and another model without them. Then, compare the two model results to see if these extreme points are crucial to the final model result.

Figure 2: Scatter Plot—KO Stock Excess Return vs. Dow 30 Index Excess Return

In [2]:
```
# Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

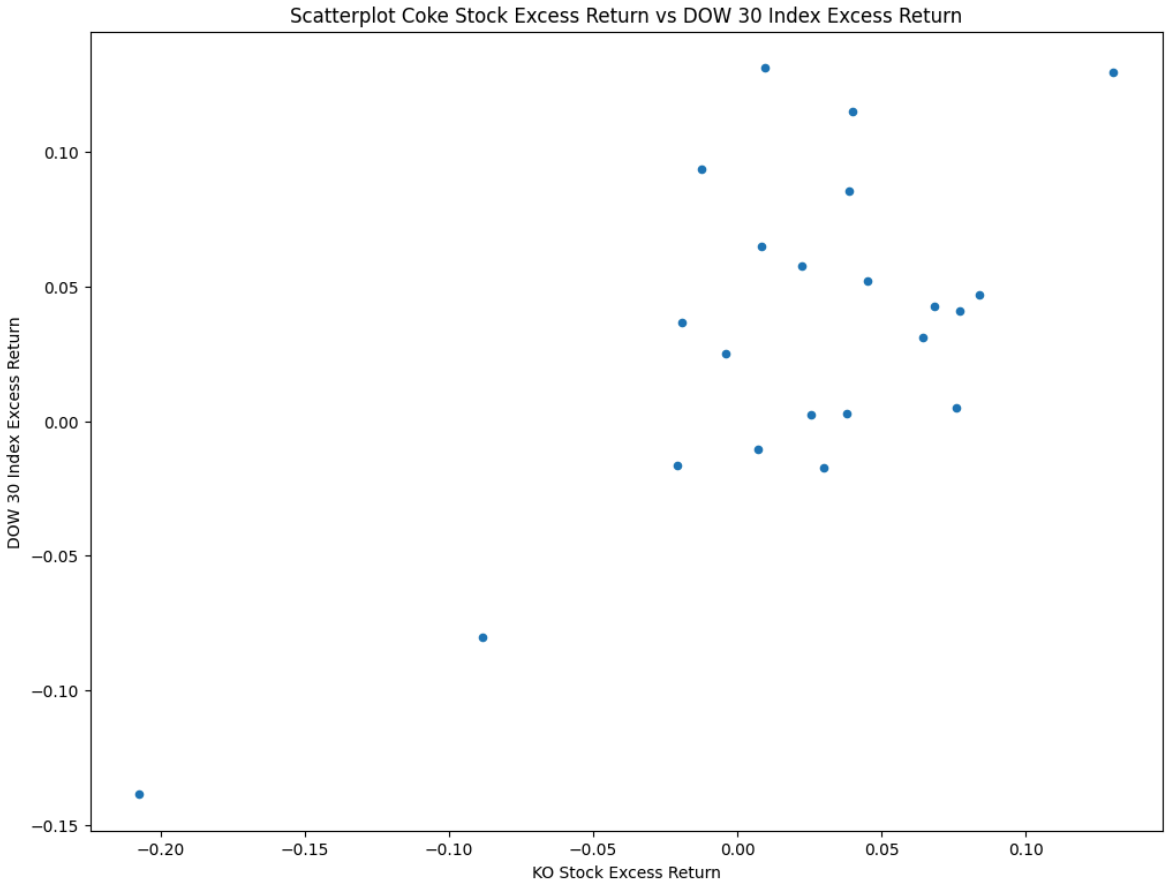
In [3]:
```
data_set.sample(2)
```

Out[3]:

	Year	Quarter	Yr_Qtr	Risk_F_R	Coke_Q_EX_R	Dow_Q_EX_R	Pepsi_Q_EX_R	REV_Q
21	2021	3	2021_3	0.000120	-0.004233	0.025186	0.036565	-0.0085
4	2017	2	2017_2	0.002632	0.068451	0.042763	0.033834	0.0640

2 rows × 24 columns

In [4]:
```
# Create scatter plot
scatt = data_set.plot.scatter(
    x="Coke_Q_EX_R",
    y="Dow_Q_EX_R",
    title="Scatterplot Coke Stock Excess Return vs DOW 30 Index Excess Return",
)
scatt.set(xlabel="KO Stock Excess Return", ylabel="DOW 30 Index Excess Return")
plt.show()
```



The scatter plot in Figure 2 shows that the *KO* stock excess return and *Dow 30* Index excess return tend to move in the same direction within our dataset. A straight line can be drawn from the lower left to the upper right. There are two extreme points in the plot, but they are positioned along the straight line.

We can also use the **Pearson Correlation** metric to check the linear relationship of two variables. The value of the Pearson Correlation is between -1 and 1. If the value of two variables tends to move in the same direction, the correlation is positive. If the two variables tend to move in opposite directions, the number is negative. A correlation of 1 means the two variables have a perfectly positive linear relationship.

Figure 3: Histogram, Correlation, and Scatter Plot Graph Matrix For Independent Variables and Dependent Variable

In [5]:
```
# Function to calculate correlation coefficient between two arrays


def corr(x, y, **kwargs):
    # Calculate the value
    coef = np.corrcoef(x, y)[0][1]
    # Make the label
    label = r"$\rho$ = " + str(round(coef, 2))

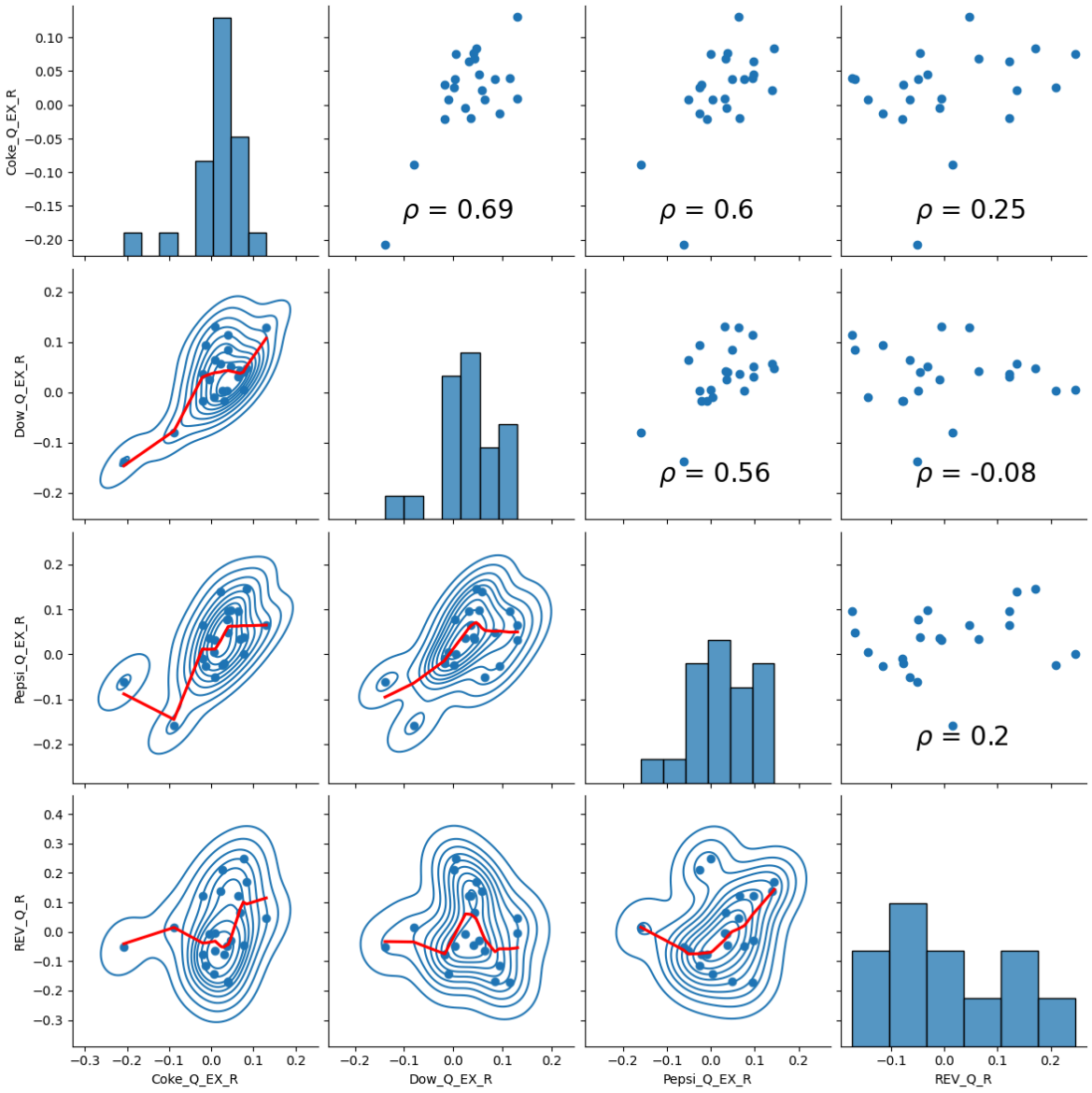
    # Add the label to the plot
    ax = plt.gca()
    ax.annotate(label, xy=(0.3, 0.15), size=20, xycoords=ax.transAxes)
```

In [6]:
```
# Create the default pairplot
grid = sns.pairplot(
```

```
    data_set, vars=["Coke_Q_EX_R", "Dow_Q_EX_R", "Pepsi_Q_EX_R", "REV_Q_R"], hei
)
# Map a scatter plot and Pearson correlation coefficient to the upper triangle
grid = grid.map_upper(plt.scatter)
grid = grid.map_upper(corr)

# Map a histogram to the diagonal
# grid = grid.map_diag(plt.hist)

# Map a density plot and regression line to the lower triangle
grid = grid.map_lower(sns.kdeplot)
grid = grid.map_lower(sns.regplot, lowess=True, line_kws={"color": "red"})
```



In [ ]:

In [ ]:

> Data Period: 2016Q2 – 2021Q3
> Data Source: Yahoo Finance and Coca Cola's website
> *Coke_Q_EX_R*: *KO* Quarterly Stock Excess Return
> *Dow_Q_EX_R*: *DOW 30* Index Quarterly Excess Return

> *Pepsi_Q_EX_R*: Pepsi Quarterly Stock Excess Return
> *REV_Q_R*: *KO* Quarterly Revenue change %

Figure 3 provides a summary of the relationships of variables in *KO*'s stock excess return analysis. The matrix is split into three parts.

- The lower left triangle of the matrix shows the scatter plots for all two-way combinations of the four variables.
- The graphs of the diagonal of the matrix are the histograms of the four variables.
- The upper right triangle of the matrix provides the correlation values of all two-way combinations of the four variables.

The scatter plot from the *KO* return and *Dow* return shows the two variables have a good positive linear relationship, as is the case for the *KO* return and Pepsi return. In both scatter plots, the data points also form an oval shape, more or less. The red line is also tilted upward. These signs suggest these two variables are good choices for modeling *KO*'s return. In the scatter plot for *KO*'s return and *KO*'s revenue growth, we don't see as clear an oval shape, and the red line is not as straight and sloped upward as the other two graphs. Their correlation values on the upper right also confirm the information from the graphs. Dow return and Pepsi return have good positive correlation values with *KO*'s return. However, *KO*'s revenue growth and return have a smaller correlation.

4. Estimate the Regression Model

Now, let's start to build the regression model. The method we are going to use here is ordinary least square (OLS) linear regression. There are two types of OLS linear regression models: **simple OLS** and **multiple OLS**. If there is only one exogenous variable vs. one endogenous variable, then it is called simple OLS linear regression. If there is more than one exogenous variable vs. one endogenous variable, then it is called multiple OLS linear regression. Since we have three exogenous variables for our model, we will build a multiple OLS linear regression.

Here is the linear regression model we are going to estimate:

$$\text{KO excess return} = b_0 + b_1 \text{Dow excess return} + b_2 \text{Pepsi excess return} + b_3 \text{KO reven}$$

b_0, b_1, b_2 and b_3 are called **parameters** or **coefficients**. We will use the OLS method to estimate the values of parameters in the model. Once the values of the parameters are obtained from the OLS calculation, we call these values **estimates**.

Figure 4: Linear Regression Result for KO Excess Return Model

In [7]:
```
# Linear regression result
result = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + Pepsi_Q_EX_R + REV_Q_R", data=data_set
).fit()
result.summary()
```

Out[7]:

OLS Regression Results

Dep. Variable:	Coke_Q_EX_R	**R-squared:**	0.604
Model:	OLS	**Adj. R-squared:**	0.538
Method:	Least Squares	**F-statistic:**	9.150
Date:	Thu, 20 Nov 2025	**Prob (F-statistic):**	0.000677
Time:	17:21:33	**Log-Likelihood:**	38.570
No. Observations:	22	**AIC:**	-69.14
Df Residuals:	18	**BIC:**	-64.78
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.0080	0.011	-0.716	0.483	-0.032	0.015
Dow_Q_EX_R	0.6197	0.195	3.173	0.005	0.209	1.030
Pepsi_Q_EX_R	0.2187	0.179	1.221	0.238	-0.158	0.595
REV_Q_R	0.1435	0.088	1.632	0.120	-0.041	0.328

Omnibus:	1.644	**Durbin-Watson:**	2.307
Prob(Omnibus):	0.439	**Jarque-Bera (JB):**	1.411
Skew:	-0.564	**Prob(JB):**	0.494
Kurtosis:	2.481	**Cond. No.**	24.1

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [8]:
```
# Parameters with 6 significant digits
result.summary2().tables[1]
```

Out[8]:

	Coef.	Std.Err.	t	P>|t|	[0.025	0.975]
Intercept	-0.008017	0.011189	-0.716476	0.482890	-0.031524	0.015490
Dow_Q_EX_R	0.619723	0.195287	3.173393	0.005261	0.209440	1.030006
Pepsi_Q_EX_R	0.218664	0.179107	1.220854	0.237893	-0.157627	0.594955
REV_Q_R	0.143481	0.087912	1.632093	0.120031	-0.041216	0.328178

Figure 4 shows the result of the regression. Let's look at the estimates of the three variables. What does the estimate mean? The estimate for Dow excess return is 0.619723.

It means when Dow excess return increases by 0.01 (1%), *KO* excess return will increase by 0.00619723 (0.619723%).

5. Review the Model

Our next question will focus on how good our estimated model is. There are several ways to assess and test the fitness of the model. We will first look at the performance of each exogenous variable in the model and then assess the overall model fit.

5.1 Performance of Each Exogenous Variable

Under the section of coefficient in the result table, there is a column called *Pr(>|t|)*. This column shows p-values for each coefficient. A p-value is a metric to indicate whether the exogenous variable can explain the change of the endogenous variable. The lower the number of the p-value, the better the explanatory ability this exogenous variable has regarding the endogenous variable. In many research projects, 0.05 is the level that determines whether an exogenous variable is significant. From our result, the only exogenous variable that is significant is *Dow* excess return (p-value: 0.00526 < 0.05). Does that mean we can just drop the other two variables from the model? There are at least two points we can consider before we make that decision.

1. **Does KO's excess return and KO's revenue growth exhibit a linear relationship?**

From the scatter plot for both variables, we know they have a weak linear relationship. Can we transform *KO*'s revenue growth into a different data series and then check the relationship again? We can try to take the square root or the logarithm of *KO*'s revenue growth and draw the scatter plot again.

2. **Are exogenous variables also correlated to each other?**

The correlation between Dow excess returns and Pepsi excess returns is 0.56 based on the scatter plot matrix. The two variables are medium positively correlated. When there are two or more correlated exogenous variables in a regression model, oftentimes, the exogenous variable with a higher correlation to the endogenous variable will absorb all the explanatory ability from other correlated exogenous variables, making other correlated exogenous variables not significant in the model.

The above two points help us rethink our choice of exogenous variables in the model and the way to handle them. We will discuss these two points in more detail and offer possible solutions in the following lessons.

5.2 R^2 and Adjusted R^2

Now, let's look at the general model fit. R^2 , the coefficient of determination, is a commonly used metric to assess the model fit. R^2 tells us how much of the variation in the endogenous variable can be explained by the variation of all exogenous variables in the model. The value of R^2 is between 0 and 1. If R^2 is 1, it means the model perfectly

explains the *KO* excess return behavior. In other words, given a set of numbers from all exogenous variables, the model can 100% correctly produce the number for the endogenous variable. In general, the higher the value of R^2, the better the model fit.

In our model, the R^2 is 0.604. It means that the change in Dow excess return, Pepsi excess return, and *KO* revenue growth together can explain 60% of the change in *KO* excess return.

If your manager thinks 0.604 is not satisfactory, you may be asked to improve the model. Usually, adding a new exogenous variable is an easy starting point. Adding more exogenous variables to a model usually improves the model's R^2. However, it can cause the issue of a model overfitting. What is model overfitting? It means the model can do an excellent job predicting the endogenous variable from the data sample that is used to build the model. However, when you put a different dataset with the endogenous variable and exogenous variables into the model, the model will give a poor forecast of the endogenous variable with the new values for the exogenous variables.

An **adjusted** R^2 can address this issue. What does an adjusted R^2 intend to adjust? Although adding new independent variables to a regression model will never lower R^2, it will add complexity to the model. It can also cause the model overfitting issue as described above. Hence, we have to ask if the extra complexity added to the model is worth it. This is what an adjusted R^2 does. Like R^2, an adjusted R^2 is between 0 and 1. An adjusted R^2 is always lower than R^2. An adjusted R^2 will increase if the degree of improvement from adding one exogenous variable is greater than the penalty imposed by one additional exogenous variable to the model.

In our model, the adjusted R^2 is 0.538. It is hard to tell if this is a good adjusted R^2. In practice, a researcher usually builds several versions of the regression models and then compares their adjusted R^2 to pick a final model. Model building and tweaking is an iterative process; it is rarely the case that the first model is the best model for any project.

6. Outliers and Influence

In the previous section, we reviewed the performance of the exogenous variables in the model and of the model as a whole. In this section, we will look into the impact of each data point from the data sample used to build the regression model.

In the scatter plot section, we discussed extreme values in the dataset. We introduced outliers and leverage points. Not all of them will influence the result of the regression model. The following visual examples demonstrate how an extreme data point can impact the regression result. The red dot is the extreme point. The black line is the regression line without the extreme point. The red line is the regression line with the extreme point.

Figure 5: The Impact of an Outlier on the Regression Model Result

In [9]:
```
# Generate 30 random points
np.random.seed(42)
```

```
n = 30
x = np.random.uniform(0, 5, n)
y = 1 + 3 * x + np.random.normal(0, 1, n)

# Add outlier at (2.5, 17)
x_with_out = np.append(x, 2.5)
y_with_out = np.append(y, 17)
```

In [10]:
```
# Plot best line of fit for 30 original points only (in black) and for 30+outlie
plt.title("Outlier", {"size": 14})
plt.plot(x, y, "o")  # scatterplot
plt.plot(2.5, 17, "ro")  # Outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(ao, bo) = np.polyfit(x_with_out, y_with_out, 1)
plt.plot(
    x_with_out,
    ao * x_with_out + bo,
    "r-",
    label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()
```

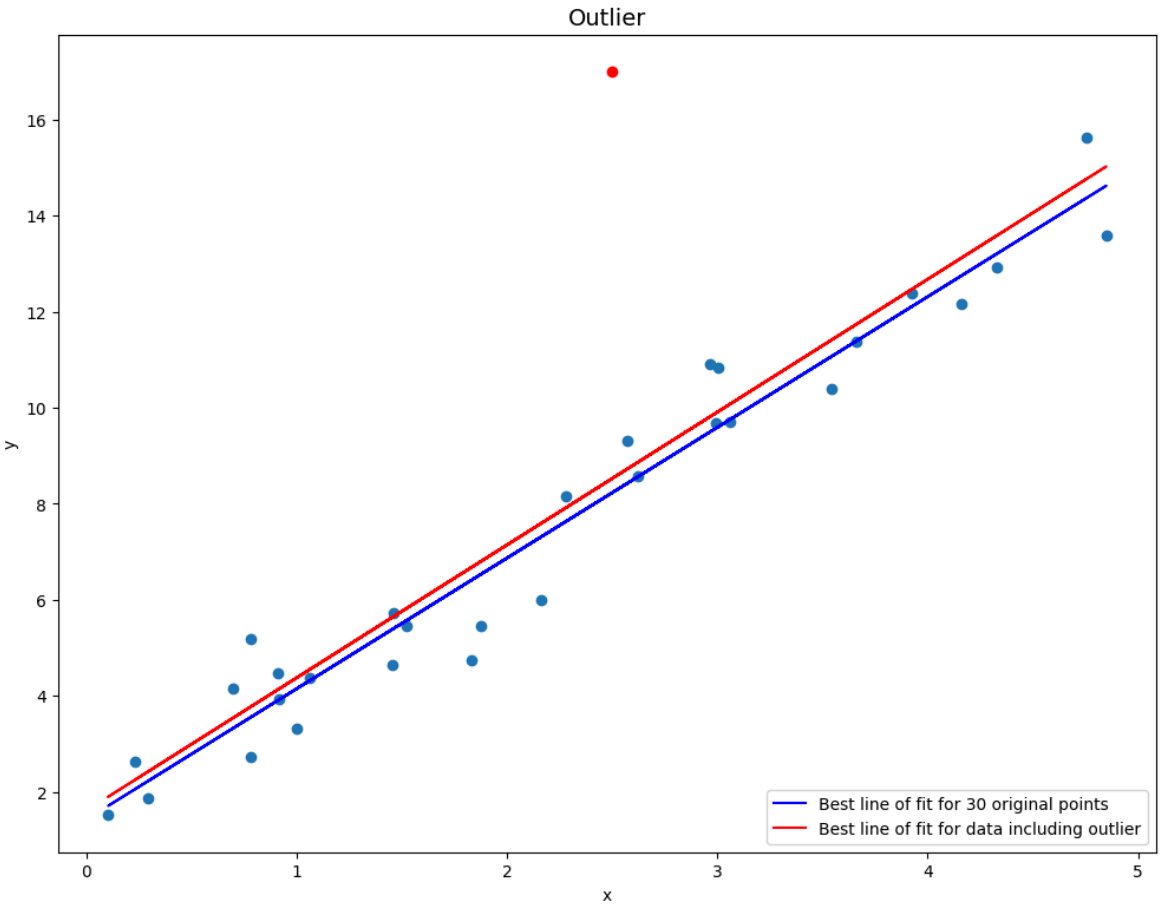


Figure 6: The Impact of a Non-Influential Leverage Point on the Regression Model Result

In [11]:
```
# Add non-influential leverage point at (8, 25)
x_with_lev = np.append(x, 8)
y_with_lev = np.append(y, 25)
```

In [12]:
```
# Plot best line of fit for 30 original points only (in black) and for 30+lev (i
plt.title("Leverage (non influential)", {"size": 14})
plt.plot(x, y, "o")  # scatterplot
plt.plot(8, 25, "ro")  # non-influential outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(al, bl) = np.polyfit(x_with_lev, y_with_lev, 1)
plt.plot(
    x_with_lev,
    al * x_with_lev + bl,
    "r-",
    label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()
```

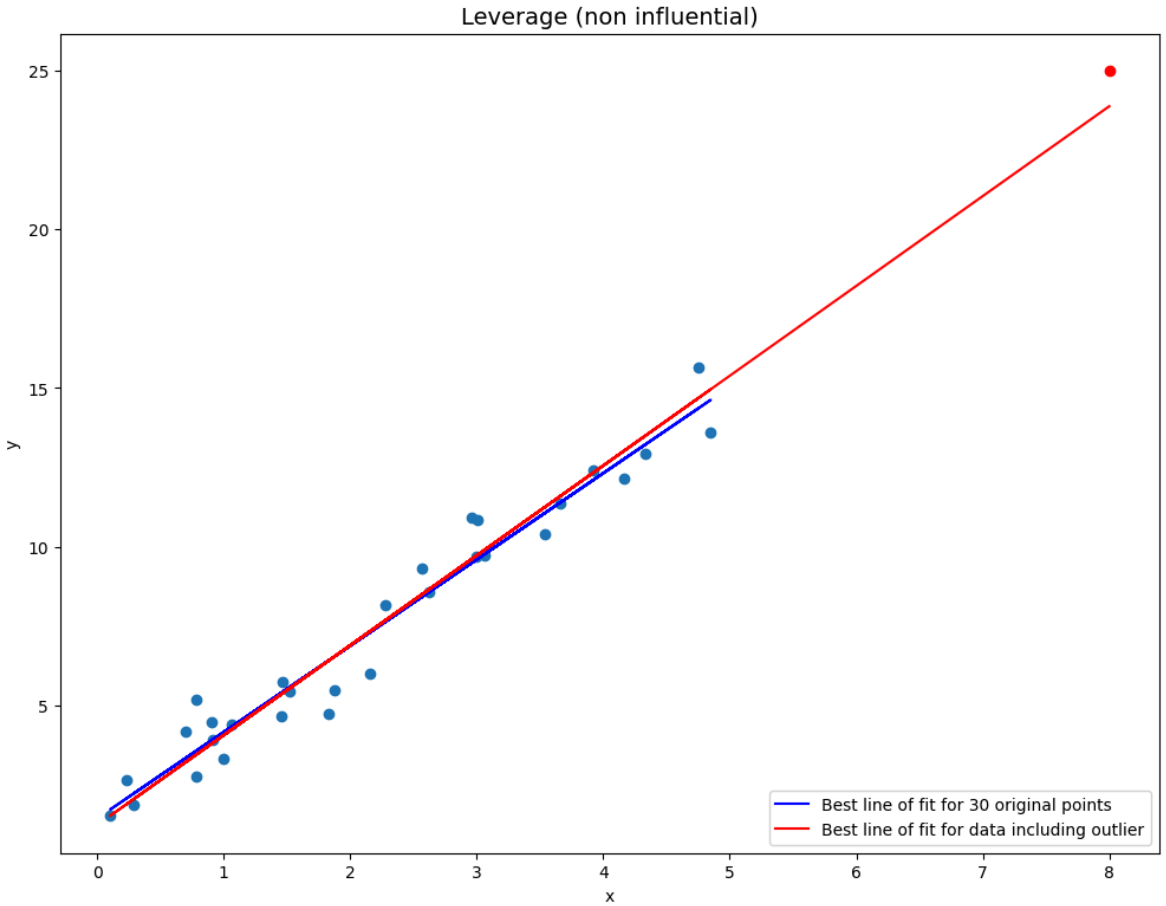


Figure 7: The Impact of an Influential Leverage Point on the Regression Model Result

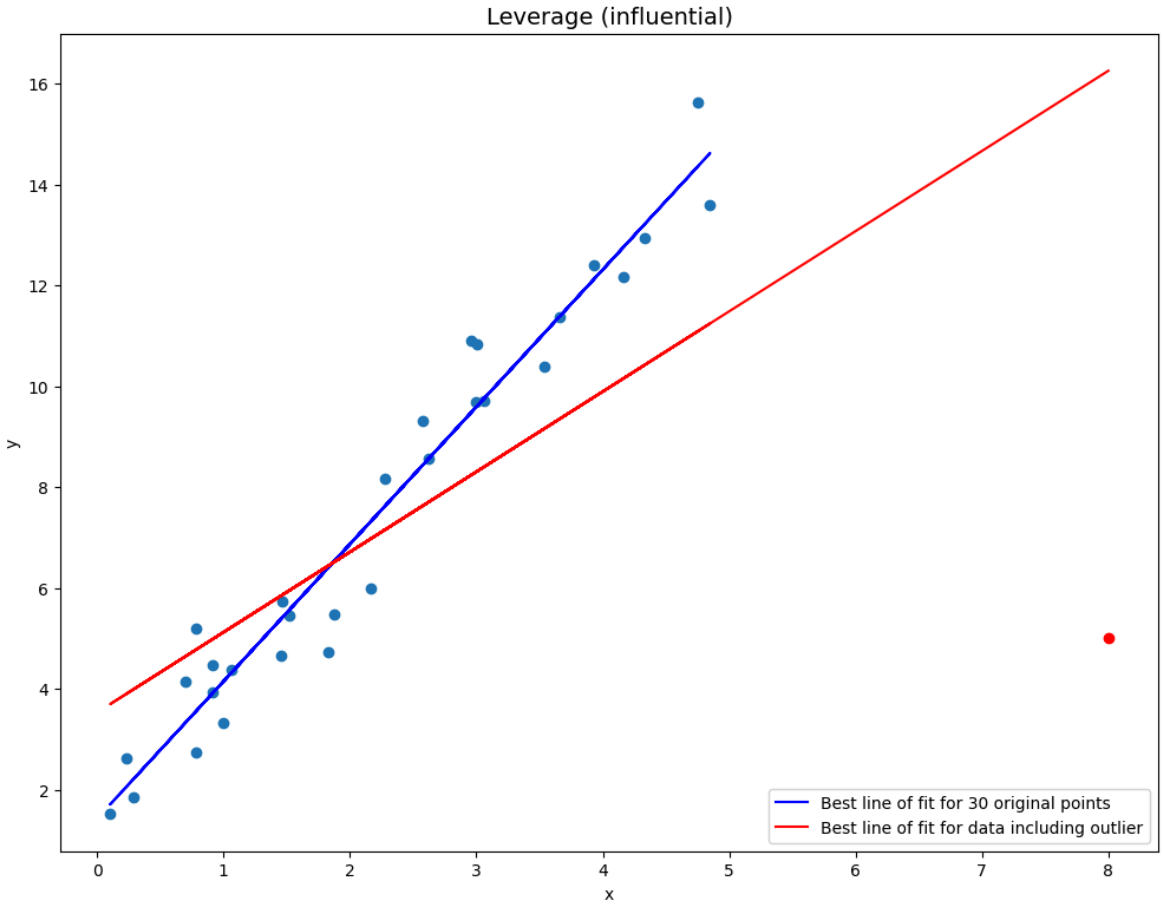
In [13]:
```
# Add non-influential leverage point at (8, 25)
x_with_inf = np.append(x, 8)
y_with_inf = np.append(y, 5)
```

In [14]:
```
# Plot best line of fit for 30 original points only (in black) and for 30+lev (i
plt.title("Leverage (influential)", {"size": 14})
plt.plot(x, y, "o")  # scatterplot
plt.plot(8, 5, "ro")  # influential outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(ai, bi) = np.polyfit(x_with_inf, y_with_inf, 1)
plt.plot(
    x_with_inf,
    ai * x_with_inf + bi,
    "r-",
    label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()
```

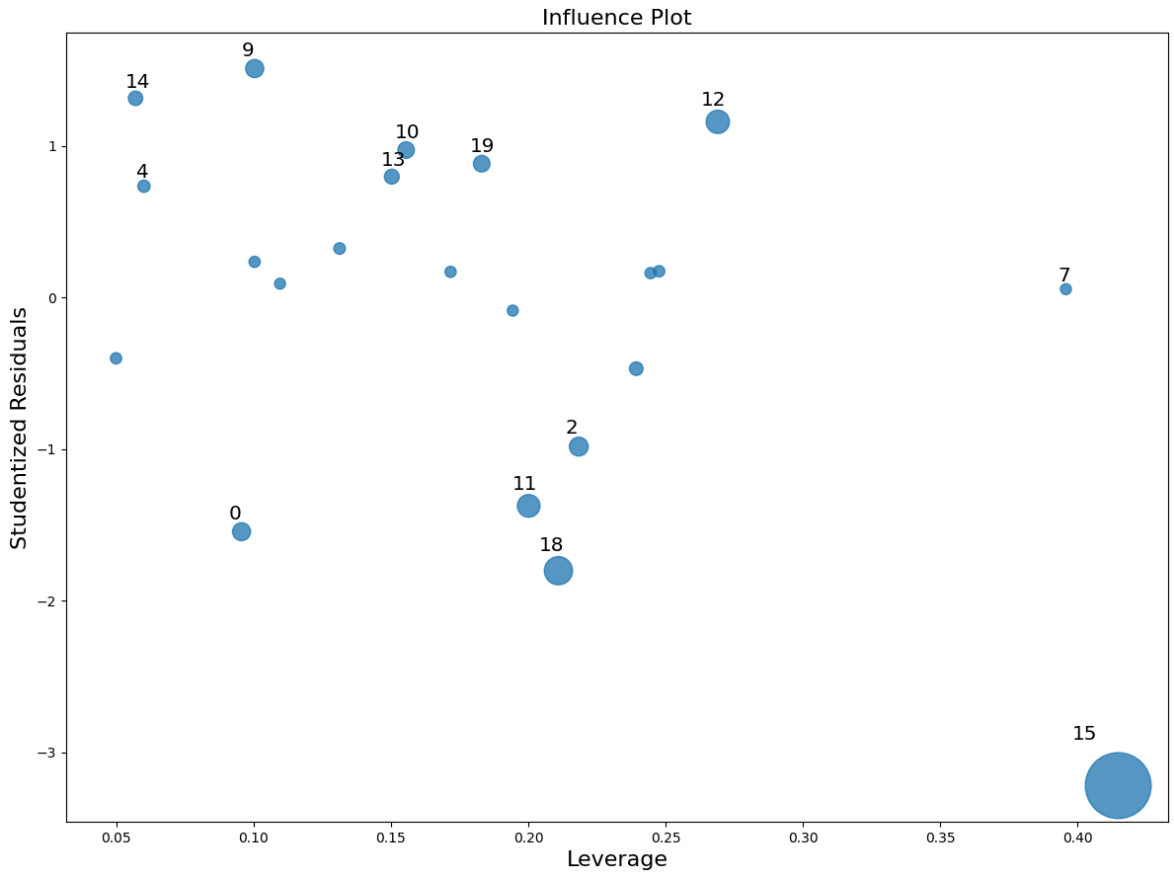


In Figures 5 and 6, we can see the regression lines don't change much with or without the extreme point. However, in Figure 7, the extreme point does significantly alter the regression result. An extreme point that can significantly alter the regression result is called an **influential point**. We need to identify influential points in the dataset and study why they are extreme (a special event or erroneous data input) because they can potentially alter the regression result. Then, we will run one model with the influential point and one without the influential point. Afterwards, we compare the model results and evaluate which results fit our modeling goal the best.

We use **Cook's distance (Cook's D)** to identify influential points in the dataset. *Cook's D* is a metric to calculate the prediction difference from the model with a data point and without a data point. The higher the *Cook's D*, the greater the influence the data point has on estimating the regression model. Oftentimes, a data point with a *Cook's D* larger than 1 is considered an influential point.

Figure 8: Influence Plot for KO's Excess Return Dataset

In [15]:
```
# Influence plot
fig = sm.graphics.influence_plot(result, criterion="cooks", alpha=0.5)
fig.tight_layout(pad=1.0)
```



In [16]:
```
# Finding most influential points
influence = result.get_influence()
inf_sum = influence.summary_frame().sort_values("cooks_d", ascending=False)
inf_sum.head(30)
```

Out[16]:

	dfb_Intercept	dfb_Dow_Q_EX_R	dfb_Pepsi_Q_EX_R	dfb_REV_Q_R	cooks_d	standard
15	-1.889683	2.264630	-0.430822	0.727270	1.208639	-2.(
18	-0.124451	-0.824470	0.459042	-0.147797	0.193197	-1.(
12	0.291357	0.075669	-0.235436	0.625953	0.121111	1.1
11	-0.148011	0.132958	-0.454322	-0.227980	0.112746	-1.3
2	-0.159230	-0.367882	0.329489	0.120792	0.067781	-0.9
9	0.443542	-0.212676	-0.033271	-0.237637	0.059381	1.4
0	-0.255128	0.021979	-0.109979	-0.299454	0.058620	-1.4
19	0.043818	0.341470	-0.123705	0.131239	0.044112	0.8
10	0.279508	-0.201537	0.101374	-0.316001	0.043645	0.9
13	0.179775	-0.221049	0.250137	-0.160919	0.028646	0.8
14	0.238351	-0.011381	0.063579	-0.137167	0.025077	1.2
6	-0.036470	-0.064849	-0.081808	0.179166	0.018177	-0.4
4	0.126545	0.044447	-0.028209	0.087034	0.008822	0.7
20	0.051847	-0.036797	0.069661	0.050285	0.004132	0.3
17	0.046248	0.017238	-0.048866	0.083383	0.002594	0.1
21	-0.080076	0.021967	-0.023327	0.017680	0.002226	-0.4
8	0.019401	-0.023776	0.059551	0.036331	0.002196	0.1
1	0.067437	-0.038266	0.004845	-0.040646	0.001618	0.2
16	0.023219	0.019100	0.007730	-0.055804	0.001555	0.1
7	0.028748	-0.003076	-0.031566	0.008274	0.000511	0.0
5	-0.018065	-0.028030	0.033928	0.001168	0.000485	-0.0
3	0.012976	-0.008273	0.022625	-0.012147	0.000267	0.0

Figure 8 is an **influence plot.** It helps us visualize the *Cook's D* among all data points. On the horizontal axis of the plot is hat value. Hat value measures the difference between the exogenous values of a data point and the mean of exogenous values in the dataset. On the vertical axis of the plot is studentized residuals. These are the standardized residuals from the predicted endogenous value of the data point from the model. The bubbles in the plot indicate the size of the *Cook's D* for each point. The bigger the bubble, the higher the *Cook's D* for a data point.

From the influence plot, we can see that Point 15 and Point 18 have the highest *Cook's D* values. Among the points, Point 15 has a *Cook's D* of 1.21, so we consider it an influential point.

7. Conclusion

In this lesson, we have introduced several important concepts for regression analysis. We started with the idea of an endogenous variable and an exogenous variable. An endogenous variable in a regression model is a variable whose value is determined by the model. An exogenous variable is a variable whose value is given outside of the model. Then, we discussed how a scatter plot can help us check if two variables are linearly correlated, if they are independent, or if they have extreme data points. We used the OLS method to estimate a regression model for our *KO* excess stock price return project. We learned how to use the p-value to check the significance of the coefficient estimate for each exogenous variable. We learned how to use R^2 and an adjusted R^2 to assess the overall performance of regression model results. We introduced the concept of outliers, leverage points, and influence points. We also learned how to use Cook's distance and influence plots to identify an influence point.

During this lesson, we also touched upon issues such as variable transformation and correlated exogenous variables in a model. In the upcoming lessons, we will address these issues in more depth.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

In [ ]:

MICHALE FEE: OK, let's go ahead and get started. So today we're turning to a new topic called that basically focused on principal components analysis, which is a very cool way of analyzing high-dimensional data. Along the way, we're going to learn a little bit more linear algebra. So today, I'm going to talk to you about eigenvectors and eigenvalues which are one of the most fundamental concepts in linear algebra. And it's extremely important and widely applicable to a lot of different things.

So eigenvalues and eigenvectors are important for everything from understanding energy levels and quantum mechanics to understanding the vibrational modes of a musical instrument, to analyzing the dynamics of differential equations of the sort that you find that describe neural circuits in the brain, and also for analyzing data and doing dimensionality reduction. So understanding eigenvectors and eigenvalues are very important for doing things like principal components analysis.

So along the way, we're going to talk a little bit more about variance. We're going to extend the notion of variance that we're all familiar with in one dimension, like the width of a Gaussian or the width of a distribution of data to the case of multivariate Gaussian distributions or multivariate-- which means, it's basically the same thing as high-dimensional data.

We're going to talk about how to compute a covariance matrix from data which describes how the different dimensions of the data are correlated with each other, what the variance in different dimensions is, and how those different dimensions are correlated with each other. And finally, we'll go through actually how to implement principal components analysis, which is useful for a huge number of things.

I'll come back to many of the different applications of principal components analysis at the end. But I just want to mention that it's very commonly used in understanding high-dimensional data and neural circuits. So it's a very important way of describing how the state of the brain evolves as a function of time. So nowadays, you can record from hundreds or even thousands or tens of thousands of neurons simultaneously. And if you just look at all that data, it just looks like a complete mess.

But somehow, underneath of all of that, the circuitry in the brain is going through discrete trajectories in some low-dimensional space within that high-dimensional mess of data. So our brains have something like 100 billion neurons in them-- about the same as the number of stars in our galaxy-- and yet, somehow all of those different neurons communicate with each other in a way that constrains the state of the brain to evolve along the low-dimensional trajectories that are our thoughts and perceptions.

And so it's important to be able to visualize those trajectories in order to understand how that machine is working. OK, and then one more comment about principal components analysis, it's not actually the best way often of doing this kind of dimensionality reduction. But the basic idea of how principal components analysis works is so fundamental to all of the other techniques. It's sort of the base on which all of those other techniques are built conceptually. So that's why we're going to spend a lot of time talking about this.

OK, so let's start with eigenvectors and eigenvalues. So remember, we've been talking about the idea that matrix multiplication performs a transformation. So we can have a vector x that we multiply it by matrix A. It transforms that set of vectors x into some other set of vectors y. And we can go from y back to x by multiplying by A inverse-- if the determinant of that matrix A is not equal to zero.

So we've talked about a number of different kinds of matrix transformations by introducing perturbations on the identity matrix. So if we have diagonal matrices, where one of the elements is slightly larger than 1, the other diagonal element is equal to 1, you get a stretch of this set of input vectors along the x-axis. Now, that process of stretching vectors along a particular direction has built into it the idea that there are special directions in this matrix transformation. So what do I mean by that?

So most of these vectors here, each one of these red dots is one of those x's, one of those initial vectors-- if you look at the transformation from x to y going-- so that's the x that we put into this matrix transformation. When we multiply by y, we see that that vector has been stretched along the x direction. So for most of these vectors, that stretch involves a change in the direction of the vector. Going from x to y

means that the vector has been rotated.

So you can see that the green vector is at a different angle than the red vector. So there's been a rotation, as well as a stretch. So you can see that's true for that vector, that vector, and so on. So you can see, though, that there are other directions that are not rotated. So here's another. I just drew that same picture over again. But now, let's look at this particular vector, this particular red vector. You can see that when that red vector is stretched by this matrix, it's not rotated. It's simply scaled. Same for this vector right here. That vector is not rotated. It's just scaled, in this case, by 1.

But let's take a look at this other transformation. So this transformation produces a stretch in the y direction and a compression in the x direction. So I'm just showing you a subset of those vectors now. You can see that, again, this vector is rotated by that transformation. This vector is rotated by that transformation. But other vectors are not rotated. So again, this vector is compressed. It's simply scaled, but it's not rotated. And this vector is stretched. It's scaled but not rotated. Does that make sense?

OK, so these transformations here are given by a diagonal matrices where the off-diagonal elements are zero. And the diagonal elements are just some constant. So for all diagonal matrices, these special directions, the directions on which vectors are simply scaled but not rotated by that matrix by that transformation, it's the vectors along the axes that are scaled and not rotated-- along the x-axis or the y-axis.

And you can see that by taking this matrix A, this general diagonal matrix, multiplying it by a vector along the x-axis, and you can see that that is just a constant, lambda 1, times that vector. So we take this times this, plus this times this, is equal to lambda 1. This times this plus this times this is equal to zero. So you can see that A times that vector in the x direction is simply a scaled version of the vector in the x direction. And the scaling factor is simply the constant that's on the diagonal.

So we can write this in matrix notation as this lambda, this stretch vector, this diagonal matrix, times a unit vector in the x direction. That's the standard basis

vector, the first standard basis vector. So that's a unit vector in the x direction is equal to lambda 1 times a vector in the x direction. And if we do that same multiplication for a vector in the y direction, we see that we get a constant times that vector in the y direction. So we have another equation.

So this particular matrix, this diagonal matrix, has two vectors that are in special directions in the sense that they aren't rotated. They're just stretched. So diagonal matrices have the property that they map any vector parallel to the standard basis into another vector along the standard basis.

So that now is a general n-dimensional diagonal matrix with these lambdas, which are just scalar constants along the diagonal. And there are n equations that look like this that say that this matrix times a vector in the direction of a standard basis vector is equal to a constant times that vector in the standard basis direction. Any questions about that? Everything else just flows from this very easily. So if you have any questions about that, just ask.

OK, that equation is called the eigenvalue equation. And it describes a property of this matrix lambda. So any vector v that's mapped by a matrix A onto a parallel vector lambda v is called an eigenvector of this matrix. So we're going to generalize now from diagonal matrices that look like this to an arbitrary matrix A. So the statement is that any vector, that when you multiply it by a matrix A that gets transformed into a vector parallel to v, it's called an eigenvector of A.

And the one vector that this is true for that isn't called an eigenvector is the zero vector because you can see that a zero vector here times any matrix is equal to zero. OK, so we exclude the zero vector. We don't call the zero vector an eigenvector. So typically a matrix, an n-dimensional matrix, has n eigenvectors and n eigenvalues. Oh, and I forgot to say that the scale factor lambda is called the eigenvalue associated with that vector v.

So now, let's take a look at a matrix that's a little more complicated than our diagonal matrix. Let's take one of these rotated stretch matrices. So remember, in the last class, we built a matrix like this that produces a stretch of a factor of 2 along a 45-degree axis. And we built that matrix by multiplying it together by basically taking this set of vectors, rotating them, stretching them, and then rotating

them back. So we did that by three separate transformations that we applied successively. And we did that by multiplying phi transpose lambda and then phi.

So let's see what the special directions are for this matrix transformation. So you can see that most of these vectors that we've multiplied by this matrix get rotated. And you can see that even vectors along the standard basis directions get rotated. So what are the special directions for this matrix? Well, they're going to be these vectors right here. So this vector along this 45-degree line gets transformed. It's not rotated. It gets stretched by a factor of 1. And this vector here gets stretched.

OK, so you can see that this matrix has eigenvectors that are along this 45-degree axis and that 45-degree axis. So in general, let's calculate what are the eigenvectors and eigenvalues for a general rotated transformation matrix. So let's do that. Let's take this matrix A and multiply it by a vector x. And we're going to ask what vectors x satisfy the properties that, when they're multiplied by A, are equal to a constant times x. So we're going to ask what are the eigenvectors of this matrix A that we've constructed in this form?

So what we're going to do is we're going to replace A with this product of matrices, of three matrices. We're going to multiply this equation on both sides by phi transpose on the left side, by phi transpose. OK, so phi transpose times this, is equal to A sabai, x subai, times 5 transpose on the left. What happens here? Remember phi is a rotation matrix. What is phi transpose phi? Anybody remember?

Good. Because for rotation matrix, the inverse, the transpose of a rotation matrix, is its inverse. And so phi transpose phi is just equal to the identity matrix. So that goes away. And we're left with lambda phi transpose x equals A phi transpose x. So remember that we just wrote down that if we have a diagonal matrix lambda, that the eigenvectors are the standard basis vectors. So what does that mean?

If we look at this equation here, and we look at this equation here, it seems like phi transpose x is an eigenvector of this equation as long as phi transpose x is equal to one of the standard basis vectors. Does that make sense? So we know this solution is satisfied by phi transpose x is equal to one of the standard basis vectors. Does that make sense? So if we replace phi transpose x with one of the standard basis vectors, then that solves this equation.

So what that means is that the solution to this eigenvalue equation is that the eigenvalues A are simply the diagonal elements of this lambda here. And the eigenvectors are just x, where x is equal to phi times the standard basis vectors. We just solve for x by multiplying both sides by phi transpose inverse. What's phi transpose inverse? phi. So we multiply both sides by phi. This becomes the identity matrix. And we have x equals phi times this set of standard basis vectors.

Any questions about that? That probably went by pretty fast. But does everyone believe this? We went through that. We went through both examples of how this equation is true for the case where lambda is a diagonal matrix and the e's are the standard basis vectors. And if we solve for the eigenvectors of this equation where A has this form of phi lambda phi transpose, you can see that the eigenvectors are given by this matrix times a standard basis vector. So any standard basis vector times phi will give you an eigenvector of this equation here.

Let's push on. And the eigenvalues are just these diagonal elements of this lambda. What are these? So now, we're going to figure out what these things are, and how to just see what they are. These eigenvectors here are given by phi times a standard basis vector. So phi is a rotation matrix, right? So phi times a standard basis vector is just what? It's just a standard basis vector rotated.

So let's just solve for these two x's. We're going to take phi, which was this 45-degree rotation matrix, and we're going to multiply it by the standard basis vector in the x direction. So what is that? Just multiply this out. You'll see that this is just a vector along a 45-degree line. So this eigenvector, this first eigenvector here, is just a vector on the 45-degree line, 1 over root 2. It's a unit vector. That's why it's got the 1 over root 2 in it.

The second eigenvector is just phi times e2. So it's a rotated version of the y standard basis vector, which is 1 over root 2 minus 1, 1. That's this vector. So our two eigenvectors we derived for this matrix that produces this stretch along a 45-degree line, the two eigenvectors are the vector, 45-degree vector in this quadrant, and the 45-degree vector in that quadrant. Notice it's just a rotated basis set. So notice that the eigenvectors are just the columns of our rotated matrix.

So let me recap. If you have a matrix that you've constructed like this, as a matrix

that produces a stretch in a rotated frame, the eigenvalues are just the diagonal elements of the lambda matrix that you put in there to build that thing, to build that matrix. And the eigenvectors are just the columns of the rotation matrix. OK, so let me summarize. A symmetric matrix can always be written like this, where phi is a rotation matrix. And lambda is a diagonal matrix that tells you how much the different axes are stretched.

The eigenvectors of this matrix A are the columns of phi. They are the basis vectors, the new basis vectors, in this rotated basis set. So remember, we can [AUDIO OUT] this rotation matrix as a set of basis vectors, as the columns. And that set of basis vectors are the eigenvectors of any matrix that you construct like this. And the eigenvalues are just the diagonal elements of the lambda that you put in there. All right, any questions about that? For the most part, we're going to be working with matrices that are symmetric, that can be built like this.

So eigenvectors are not unique. So if x eigenvector of A, then any scaled version of x is also an eigenvector. Remember, an eigenvector is a vector that when you multiply it by a matrix just gets stretched and not rotated. What that means is that any vector in that direction will also be stretched and not rotated. So eigenvectors are not unique. Any scaled version of an eigenvector is also an eigenvector. When we write down eigenvectors of a matrix, we usually write down unit vectors to avoid this ambiguity.

So we usually write eigenvectors as unit vectors. For matrices of n dimensions, there are typically n different unit eigenvectors-- n different vectors in different directions that have the special properties that they're just stretched and not rotated. So for our two-dimensional matrices that produce stretch in one direction, the special directions are-- sorry, so here is a two-dimensional, two-by-two matrix that produces a stretch in this direction. There are two eigenvectors, two unit eigenvectors, one in this direction and one in that direction.

And notice, that because the eigenvectors are the columns of this rotation matrix, the eigenvectors form a complete orthonormal basis set. And that is true. That statement is true only for symmetric matrices that are constructed like this. So now, let's calculate what the eigenvalues are for a general two-dimensional matrix A. So here's our matrix A. That's an eigenvector. Any vector x that satisfies that equation

is called an eigenvector. And that's the eigenvalue associated with that eigenvector.

We can rewrite this equation as A times x equals lambda i times x-- just like A equals b, then equals 1 times b. We can subtract that from both sides, and we get A minus lambda i times x equals zero. So that is a different way of writing an eigenvalue equation. Now, what we're to do is we're going to solve for lambdas that satisfy this equation. And we only want solutions where x is not equal to zero.

So this is just a matrix. A minus lambda i is just a matrix. So how do we know whether this matrix has solutions where x is not equal to zero? Any ideas? [INAUDIBLE]

AUDIENCE: [INAUDIBLE]

MICHALE FEE: Is, so what do we need the determinant to do?

AUDIENCE: [INAUDIBLE]

MICHALE FEE: Has to be zero. If the determinant of this matrix is not equal to zero, then the only solution to this equation is x equals zero. OK, so we solve this equation. We ask what values of lambda give us a zero determinant in this matrix. So let's write down an arbitrary A, an arbitrary two-dimensional matrix A, 2D, 2 by 2. We can write A minus lambda i like this. Remember, lambda i is just lambdas on the diagonals.

The determinant of A minus lambda i is just the product of the diagonal elements minus the product of the off-diagonal elements. And we set that equal to zero. And we solve for lambda. And that just looks like a polynomial. OK, so the solutions to that polynomial solve what's called the characteristic equation of this matrix A. And those are the eigenvalues of this arbitrary matrix A, this 2D, two-by-two matrix.

So there is characteristic equation. There is the characteristic polynomial. We can solve for lambda just by using the quadratic formula. And those are the eigenvalues of A. Notice, first of all, there are two of them given by the two roots of this quadratic equation. And notice that they can be real or complex. They can be complex. They are complex in general. And they can be real, or imaginary, or have real and imaginary components.

And that just depends on this quantity right here. If what's inside this square root is

negative, then eigenvalues will be complex. If what's inside the square root is positive, then the eigenvector will be real. So let's find the eigenvalues for a symmetric matrix. a, d on the diagonals and b on the off-diagonals. So let's see what happens. Let's plug these into this equation. The 4bc becomes 4b squared.

And you can see that this thing has to be greater than zero because a minus d squared has [INAUDIBLE] has to be positive. And b squared has to be positive. And so that quantity has to be greater than zero. And so what we find is that the eigenvalues of a symmetric matrix are always real. So let's just take this particular-- just an example-- and let's plug those into this equation. And what we find is that the eigenvalues are 1 plus or minus root 2 over 2. So two real eigenvalues.

So let's consider a special case of a symmetric matrix. Let's consider a matrix where the diagonal elements are equal, and the off-diagonal elements are equal. So we can update this equation for the case where the diagonal elements are equal. So a equals d. And what you find is that the eigenvalues are just a plus b and a minus b-- so a plus b and a minus b. And the eigenvectors can be found just by plugging these eigenvalues into the eigenvalue equation and solving for the eigenvectors.

So I'll just go through that real quick-- a times x. So we found two eigenvalues, so there are going to be two eigenvectors. We can just plug that first eigenvalue into here, call it lambda plus. And now, we can solve for the eigenvector associated with that eigenvalue. Just plug that in, solve for x. What you find is that the x associated with that eigenvalue is 1, 1-- if you just go through the algebra. So that's the eigenvector associated with that eigenvalue. And that is the eigenvector associated with that eigenvalue.

So I'll just give you a hint. In most of the problems that I'll give you to deal with on an exam or many of the ones in the problem sets, I think, in the problem set will have a form like this and [INAUDIBLE] eigenvectors along a 45-degree axis. So if you see a matrix like that, you don't have to plug it into MATLAB to extract the eigenvalues. You just know that the eigenvectors are on the 45-degree axis.

So the process of writing a matrix as phi lambda phi transpose is called eigen-decomposition of this matrix A. So if you have a matrix that you can write down like this, that you can write in that form, it's called eigen-decomposition. And the

lambdas, the diagonal elements of this lambda matrix, are real. And they're the eigenvalues. The columns of phi are the eigenvalues, and they form an orthogonal basis set.

And this, if you take this equation and you multiply it on both sides by phi, you can write down that equation in a slightly different form-- A times phi equals phi lambda. This is a matrix way, a matrix equivalent, to the set of equations that we wrote down earlier. So remember, we wrote down this eigenvalue equation that describes that when you multiply this matrix A times an eigenvector equals lambda times the eigenvector, this is equivalent to writing down this matrix equation.

So you'll often see this equation to describe the form of the eigenvalue equation rather than this form. Why? Because it's more compact. Any questions about that? We've just piled up all of these different f vectors into the columns of this rotation matrix phi. So if you see an equation like that, you'll know that you're just looking at an eigenvalue equation just like this.

Now in general, when you want to do eigen-decomposition, when you have a symmetric matrix that you want to write down in this form. It's really simple. You don't have to go through all of this stuff with the characteristic equation, and solve for the eigenvalues, and then plug them in here, and solve for the eigenvectors. You can do that if you really want to. But most people don't because in two dimensions, you can do it. But in higher dimensions, it's very hard or impossible.

So what you typically do is just use the eig function in MATLAB. If you just use this function eig on a matrix, it will return the eigenvectors and eigenvalues. So here, I'm just constructing a matrix A-- 1.5, 0.5, 0.5, and 1.5, like that. And if you just use the eig function, it returns the eigenvectors as the columns of the matrix and the eigenvalues as the diagonals of this matrix. So you have to pass it. Arguments F and V equals eig of A. And it returns eigenvectors and eigenvalues. Any questions about that?

So let's push on toward doing principal components analysis. So this is just the machinery that you use. Oh, and I think I had one more panel here just to show you that if you take F and V, you can reconstruct A. So A is just F, V, F transpose. F is just phi in the previous equation. And V is the lambda. Sorry, they didn't have phi and

lambda, and they're not options. For variable names, I used F and V. And you can see that F, V, F transpose is just equal to A. Any questions about that? No?

All right, so let's turn to how do you use eigenvectors and eigenvalues to describe data. So I'm going to briefly review the notion of variance, what that means in higher dimensions, and how you use a covariance matrix to describe data in high dimensions.

So let's say that we have a bunch of observations of a variable x-- so this is now just a scaler. So, we have m different observations, x superscript j is the j-th observation of that data. And you can see that if you make a bunch of measurements of most things in the world, you'll find a distribution of those measurements. Often, they will be distributed in a bump.

You can write down the mean of that distribution just as the average value overall distributions by summing together all those distributions and dividing by the number of observations. You can also write down the variance of that distribution by subtracting the mean from all of those observations, squaring that difference from the mean, summing up over all observations, and dividing by m.

So let's say that we now have m different observations of two variables, pressure and temperature. We have a distribution of those quantities. We can describe that observation of x1 and x2 as a vector. And we have m different observations of that vector. You can write down the mean and variance of x1 and x2. So for x1, we can write down the mean as mu1. We can write down the variance of x1. We can write down the mean and variance of x2, of the x2 observation.

And sometimes, that will give you a pretty good description of this two-dimensional observation. But sometimes, it won't. So in many cases, those variables, x1 and x2, are not correlated with each other. They're independent variables. In many cases, though, x1 and x2 are dependent on each other. The observations of x1 and x2 are correlated with each other, so that if x1 is big, x2 also tends to be big.

In these two cases, x1 can have the same variance. x2 can have the same variance. But there's clearly something different here. So we need something more than just describing the variance of x1 and x2 to describe these data. And that thing is the covariance. It just says how do x1 and x2 covary? If x1 is big, does x2 also tend to be

big? In this case, the covariance is zero. In this case, the covariance is positive.

So we're taking if a fluctuation of x1 above the mean is associated with a fluctuation of x2 above the mean, then these points will produce a positive contribution to the covariance. And these points here will also produce a positive contribution to the covariance. And the covariance here will be some number greater than zero. That's closely related to the correlation, just the Pearson correlation coefficient, which is the covariance divided by the geometric mean of the individual variances.

I'm assuming most of you have seen this many times, but just to get us up to speed. So if you have data, a bunch of observations, you can very easily fit those data to a Gaussian. And you do that simply by measuring the mean and variance of your data. And that turns out to be the best fit to a Gaussian. So if you have a bunch of observations in one dimension, you measure the mean and variance of that set of data. That turns out to be a best fit in the least squared sense to a Gaussian probability distribution defined by a mean and a variance.

So this is easy in one dimension. What we're interested in doing is understanding data in higher dimensions. So how do we describe data in higher dimensions? How do we describe a Gaussian in higher dimensions? So that's what we're going to turn to now. And the reason we're going to do this is not because every time we have data, we're really trying to fit a Gaussian into it. It's just that it's a powerful way of thinking about data, of describing data in terms of variances in different directions.

And so we often think about what we're doing when we are looking at high-dimensional data is understanding its distribution in different dimensions as kind of a Gaussian cloud that optimally best fits the data that we're looking at. And mostly because it just gives us an intuitive about how to best represent or think about data in high dimensions.

So we're going to get insights into how to think about high-dimensional data. We're going to develop that description using the vector and matrix notation that we've been developing all along because vectors and matrices provide a natural way of manipulating data sets, of doing transformations of basis, rotations, so on. It's very compact. And those manipulations are really trivial in MATLAB or Python.

So let's build up a Gaussian distribution in two dimensions. So we have, again, our

Gaussian random variables, x1 and x2. We have a Gaussian distribution, where the probability distribution is proportional to e to the minus 1/2 of x1 squared. We have probability distribution for x2-- again, probability of x2. We can write down the probability of x1 and x2, the joint probability distribution, assuming these are independent. We can write that as the product of p-- the product of the two probability distributions p of x1 and p of x2.

And we have some Gaussian cloud, some Gaussian distribution in two dimensions that we can write down like this. That's simply the product. So the product of these two distributions is e to the minus 1/2 x1 squared times e to the minus 1/2 x2 squared. And then, there's a constant in front that just normalizes, so that the total area under that curve is just 1.

We can write this as e to the minus 1/2 x1 squared plus x2 squared. And that's e to the minus 1/2 times some distance from the origin. So it falls off exponentially in a way that depends only on the distance from the origin or from the mean of the distribution. In this case, we set the mean to be zero.

Now, we can write that distance squared using vector notation. It's just the square magnitude of that vector x. So if we have a vector x sitting out here somewhere, we can measure the distance from the center of the Gaussian as the square magnitude of x, which is just x dot x, or x transpose x. So we're going to use this notation to find the distance of a vector from the center of the Gaussian distribution. So you're going to see a lot of x [INAUDIBLE] axis.

So this distribution that we just built is called an isotropic multivariate Gaussian distribution. And that distance d is called the Mahalanobis distance, which I'm going to say as little as possible. So that distribution now describes how these points-- the probability of finding these different points drawn from that distribution as a function of their position in this space. So you're going to draw a lot of points here in the middle and fewer points as you go away at larger distances.

So this particular distribution that I made here has one more word in it. It's an isotopic multivariate Gaussian distribution of unit variance. And what we're going to do now is we're going to build up all possible Gaussian distributions from this distribution by simply doing matrix transformations. So we're going to start by

taking that unit variance Gaussian distribution and build an isotopic Gaussian distribution that has an arbitrary variance-- that means an arbitrary width.

We're then going to build a Gaussian distribution that can be stretched arbitrarily along these two axes, y1 and y2. And we're going to do that by using a transformation with a diagonal matrix. And then, what we're going to do is build an arbitrary Gaussian distribution that can be stretched and worked in any direction by using a transformation matrix called a covariance matrix, which just tells you how that distribution is stretched in different directions. So we can stretch it in any direction we want. Yes.

AUDIENCE: Why is [INAUDIBLE]?

MICHALE FEE: OK, the distance squared is the square of magnitude. And the square of magnitude is x dot x, the dot product. But remember, we can write down the dot product in matrix notation as x transpose x. So if we have row vector times a column vector, you get the dot product. Yes, Lina.

AUDIENCE: What does isotropic mean?

MICHALE FEE: OK, isotropic just means the same in all directions. Sorry, I should have defined that.

AUDIENCE: [INAUDIBLE] when you stretched it, it's not isotropic?

MICHALE FEE: Yes, these are non-isotropic distributions because they're different. They have different variances in different directions. So you can see that this has a large variance in the y1 direction and a small variance in the y2 direction. So it's non-isotropic. Yes, [INAUDIBLE].

AUDIENCE: Why do you [INAUDIBLE]?

MICHALE FEE: Right here. OK, think about this. Variance, you put into this Gaussian distribution as the distance squared over the variance squared. It's distance squared over a variance, which is sigma squared. Here it's distance squared over a variance. Here it's distance squared over a variance. Does that makes sense? It's just that in order to describe these complex stretching and rotation of this Gaussian distribution in high-dimensional space, we need a matrix to do that.

And that covariance matrix describes the variances in the different direction and essentially the rotation. Remember, this distribution here is just a distribution that's stretched and rotated. Well, we learned how to build exactly such a transformation by taking the product of phi transpose lambda phi. So we're going to use this to build these arbitrary Gaussian distributions.

OK, so I'll just go through this quickly. If we have an isotopic unit variance Gaussian distribution as a function of this vector x, we can build a Gaussian distribution of arbitrary variance by writing down a y that's simply sigma times x. We're going to transform x into y, so that we can write down a distribution that has an arbitrary variance. Here this is variance 1. Here this is sigma squared.

So let's make just a change of variables y equals sigma x. So now, what's the probability distribution as a function of y? Well, there's probability distribution as a function of x. We're simply going to substitute y equals sigma x with x equals sigma inverse y. We're going to substitute this into here. The Mahalanobis distance is just x transpose x, which is just sigma inverse y transpose sigma inverse y.

And when you do that, you find that the distance squared is just y transpose sigma to the minus 2y. So there is our Gaussian distribution for this distribution. There's the expression for this Gaussian distribution with a variance sigma. We can rewrite this in different ways.

Now, let's build a Gaussian distribution that stretched arbitrarily in different directions, x and y. We're going to do the same trick. We're simply going to make a transformation y equals matrix, diagonal matrix, s times x and substitute this into our expression for a Gaussian. So x equals s inverse y. The Mahalanobis distance is given by x transpose x, which we can just get down here. Let's do that with this substitution. And we get an s squared here, s inverse squared, which we're just going to write as lambda inverse.

And you can see that you have these variances along the diagonal. So if that's lambda inverse, then lambda is just a matrix of variances along the diagonal. So sigma 1 squared is the variance in this direction. Sigma 2 squared is the variance in this direction.

I'm just showing you how you make a transformation to this vector x into another

vector y to build up a representation of this effective distance from the center of distribution for different kinds of Gaussian distributions. And now finally, let's build up an expression for a Gaussian distribution with arbitrary variance and covariance.

So we're going to make a transformation of x into a new vector y using this rotated stretch matrix. We're going to substitute this in, calculate the Mahalanobis distance-- is now x transpose x. Substitute this and solve for the Mahalanobis distance. And what you find is that distance squared is just y transpose phi lambda inverse phi transpose times y. And we just write that as y transpose sigma inverse y. So that is now an expression for an arbitrary Gaussian distribution in high-dimensional space. And that distribution is defined by this matrix of variances and covariances.

Again, I'm just writing down the definition of sigma inverse here. We can take the inverse of that, and we see that our covariance-- this is called a covariance matrix-- it describes the variance and correlations of those different dimensions as a matrix. That's just this rotated stretch matrix that we been working with. And that's just the same as this covariance matrix that we described for distribution.

I feel like all that didn't come out quite as clearly as I'd hoped. But let me just summarize for you. So we started with an isotopic Gaussian of unit variance. And we multiplied that vector, we transformed that vector x, by multiplying it by sigma so that we could write down a Gaussian distribution of arbitrary variance. We transformed that vector x with a diagonal covariance matrix to get arbitrary stretches along the axes.

And then, we made another kind of transformation with an arbitrary stretch and rotation matrix so that we can now write down a Gaussian distribution that has arbitrary stretch and rotation of its variances in different directions. So this is the punch line right here-- that you can write down the Gaussian distribution with arbitrary variances in this form. And that sigma right there is just the covariance matrix that describes how wide that distribution is in the different directions and how correlated those different directions are.

I think this just summarizes what I've already said. So now, let's compute the covariance matrix from data. So now, I've shown you how to represent Gaussians in high dimensions that have these arbitrary variances. Now, let's say that I actually

have some data. How do I fit one of these Gaussians to it? And it turns out that it's really simple. It's just a matter of calculating this covariance matrix. So let's do that.

So here is some high-dimensional data. Remember that to fit a Gaussian to a bunch of data, all we need to do is to find the mean and variants in one dimension. For higher dimensions, we just need to find the mean and the covariance matrix. So that's simple. So here's our set of observations. Now, instead of being scalars, they're vectors. First thing we do is subtract the mean. So we calculate the mean by summing all of those observations, dividing those numbers, divide by m. So there we find the mean.

We compute a new data set with the mean subtracted. So from every one of these observations, we subtract the mean. And we're going to call that z. So there is our mean subtracted here. I've subtracted the mean. So those are the x's. Subtract the mean. Those are now our z's, our mean-subtracted data. Does that makes sense? Now, we're going to calculate this covariance matrix.

Well, all we do is we find the variance in each direction and the covariances. So it's going to be a matrix in low-dimensional data. It's a two-by-two matrix. So we're going to find the variance in the z1 direction. It's just z1 times z1, summed over all the observations, divided by m. Th variance in the z2 direction is just the sum of z2, j, z2, j divided by m.

The covariance is just the cross terms, z1 one times z2 and z2 times z1. Of course, those are equal to each other. So in a covariance matrix, it's symmetric. So how do we calculate this? It turns out that in MATLAB, this is super-duper easy. So if this is our vector, that's our vector, one of our observations, we can compute the inner product z transpose z. So the inner product is just z transpose z, which is z1, z2, z1, z2. That's the square magnitude of z. There's another kind of product called the outer product. Remember that.

So the outer product looks like this. This is a 1 by 2. That's a rho vector times a column vector is equal to a scalar. 1 by 2 times 2 by 1 equals by 1 by 1-- two rows, one column-- times 1 by 2, gives you a 2 by 2 matrix that looks like this. z1 times z1, z1, z2, z1, z2, z2, z2. Why? It's z1 times z1 equals that. z1 times z2, z2 z1, one z2 z2. So that outer product already gives us the components to compute the correlation

matrix.

So what we do is we just take this vector, z the j-th observation of this vector z, and multiply it by the j-th observation of this vector z transpose. And that gives us this matrix. And we sum over all this. And you see that is exactly the covariance matrix.

So if we have m observations of vector z, we put them in matrix form. So we have a big, long data matrix. Like this. There are m observations of this two-dimensional vector z. The data dimension, the data vector has, mentioned 2. Their are m observations. So m is the number of samples. So this is an n-by-m matrix. So if you want to compute the covariance matrix, you just in MATLAB, literally do z. This big matrix z times that matrix transpose. And that automatically finds the covariance matrix for you in one line of MATLAB.

There's a little trick to subtract the mean easily. So remember, your original observations are x. You compute the mean across the rows. Thus, you're going you're going to sum across columns to give you a mean for each row. That gives you a mean of that first component of your vector, mean of the second component. That's really easy in the lab. mu is the mean of x summing cross the second component. That gives you a mean vector and then you use repmat to fill that mean out in all of the columns and [INAUDIBLE] subtract this mean from x to get this data matrix z.

So now, let's apply those tools to actually do some principal components analysis. So principal components analysis is really amazing. If you look at single nucleotide polymorphisms and populations of people, there are like hundreds of genes that you can look at. You can look at different variations of a gene across hundreds of genes. But it's this enormous data set.

And you can find out which directions in that space of genes give you information about the genome of people. And for example, if you look at a number of genes across people with different backgrounds, you can see that they're actually clusters corresponding to people with different backgrounds.

You can do single-cell profiling. So you can do the same thing in different cells with a tissue. So you look at RNA transcriptional profiling. You see what are the genes that are being expressed in individual cells. You can do principal components

analysis of those different genes and find clusters for different cell types within a tissue. This is now being applied very commonly in brain tissue now to extract different cell types. You can use images and find out which components of an image actually give you information about different faces.

So you can find a bunch of different faces, find the covariance matrix of those images, take that, do eigendecomposition on that covariance matrix. And extract what are called eigenfaces. These are dimensions on which the images carry information about face identity. You can use principal components analysis to decompose spike waveforms into different spikes. This is a very common way of doing spike sorting. So when you stick an electrode in the brain, you'd record from different cells at the end of the electrode.

Each one of those has a different way of form and you can use this method to extract the different waveforms people have even recently used this now to understand the low-dimensional trajectories of movements. So if you take a movie--

SPEAKER: After tracking, a reconstruction of the global trajectory can be made from the stepper motor movements, while the local shape changes of the worm can be seen in detail.

MICHALE FEE: OK, so here you see a c elegans, a worm moving along. This is an image, so it's a very high-dimensional. There are 1,000 pixels in this image. And you can decompose that image into a trajectory in a low-dimensional space. And it's been used to describe the movements in a low-dimensional space and relate to a representation of the neural activity in low dimensions as well. OK, so it's a very powerful technique.

So let me just first demonstrate PCA on just some simple 2D data. So here's a cloud of points given by a Gaussian distribution. So those are a bunch of vectors x. We can transform those vectors x using phi s phi transpose to produce a Gaussian, a cloud of points with a Gaussian distribution, rotated at 45 degrees, and stretched by 1.7-ish along one axis and compressed by that amount along another axis.

So we can build this rotation matrix, this stretch matrix, and build a transformation matrix-- r, s, r transpose. Multiply that by x. And that gives us this data set here. OK, so we're going to take that data set and do principal components analysis on it. And

what that's going to do is it's going to find the dimensions in this data set that have the highest variance. It's basically going to extract the variance in the different dimensions. So we take that set of points. We just compute the covariance matrix by taking z, z transpose, times 1 over m.

That computes that covariance matrix. And then, we're going to use the eig function in MATLAB to extract the eigenvectors and eigenvalues of the covariance matrix OK, so q-- we're going to call q is the variable name for the covariance matrix it's zz transpose over m. Call eig of q. That returns the rotation matrix. And that rotation matrix, the columns of which are the eigenvectors, it returns the matrix of eigenvalues, the diagonal elements are the eigenvalues.

Sometimes, you need to do a flip-left-right because I sometimes return the lowest eigenvalues first. But I generally want to plot put the largest eigenvalue first. So there's the largest one, there's the smallest one. And now, what we do, is we simply rotate. We [AUDIO OUT] basis. We can rotate this data set using the rotation matrix that the principal components analysis found.

OK, so we compute the covariance matrix. Find the eigenvectors and eigenvalues of the covariance matrix right there. And then, we just rotate the data set into that new basis of eigenvectors and eigenvalues. It's useful for clustering. So if we have two clusters, we can take the clusters, compute the covariance matrix. Find the eigenvectors and eigenvalues of that covariance matrix. And then, rotate the data set into a basis set in which the dimensions in the data on which variances largest are along the standard basis vectors.

Let's look at a problem in the time domain. So here we have a couple of time-dependent signals. So this is some amplitude as a function of time. These are signals that I constructed. They're some wiggly function that I added noise to. What we do is we take each one of those times series, and we stack them up in a bunch of columns. So our vector is now a set of 100 time samples. So there is a vector of 100 different time points. Does that make sense? And we have 200 observations of those 100-dimensional vectors.

So we have a data vector x that has columns. That are hundreds dimensional. And we have 200 of those observations. So it's 100-by-200 matrix. 100-by-200 matrix.

We do the means subtraction we subtract the mean using that trick that I showed you. Compute the covariance matrix. So there we compute the mean. We subtract the mean using repmat. Subtract the mean from the data to get z. Compute the covariance matrix Q. That's what the covariance matrix looks like for those data.

And now, we plug it into eig to extract the eigenvectors and eigenvalues. OK, so extract F and V. If we look at the eigenvalues, you can see that there are hundreds eigenvalues because those data have 100 dimensions. So there are hundreds eigenvalues. You could see that two of those eigenvalues are big, and the rest are small. This is on a log scale. What that says is that almost all of the variance in these data exist in just two dimensions. It's 100-dimensional space. But the data are living in two dimensions. And all the rest is noise. Does that makes sense?

So what you'll typically do is take some data, compute the covariance matrix, find the eigenvalues, and look at the spectrum of eigenvalues. And you'll very often see that there is a lot of variance in a small subset of eigenvalues. Then, it tells you that the data are really living in a lower-dimensional subspace than the full dimensionality of the data. So that's where your signal is. And all the rest of that is noise.

You can plot the cumulative of this. And you can say that the first two components explain over 60% of the total variance in the data. So since there are two large eigenvalues, let's look at the eigenvectors associated with those. And we can find those. Those are just the first two columns of this matrix F that the eig function returned to us. And that's what those two eigenvectors look like.

That's what the original data looked like. The eigenvectors, the columns of the F matrix, are an orthogonal basis set. A new basis set. So those are the first two eigenvectors. And you can see that the signal lives in this low-dimensional space of these two eigenvectors. All of the other eigenvectors are just noise. So we can do is we can project the data into this new basis set. So let's do that. We simply do a change of basis.

The f is a rotation matrix. We can project our data z into this new basis set and see what it looks like. Turns out, that's what it looks like. There are two clusters in those data corresponding to the two different wave forms that you could see in the data.

Right there, you can see that there are kind of two wave forms in the data. If you projected data into this low-dimensional space, you can see that there are two clusters there.

If you projected data into other projections, you don't see it. It's only in this particular projection that you have these two very distinct clusters corresponding to the two different wave forms in the data. Now, almost all of the variance is in the space of the first two principal components. So what you can actually do is, you can project the data into these first two principal components, set all of the other principal components to zero, and then rotate back to the original basis set.

That is that you're setting as much of the noise to zero as you can. You're getting rid of most of the noise. And then, when you rotate back to the original basis set, you've gotten rid of most of the noise. And that's called principal components filtering. So here's before filtering and here's after filtering. OK, so youve found the low-dimensional space, in which all the data sits, in which the signal sits, everything outside of that space is noise. So you rotate the data into a new basis set. You can filter out all the other dimensions that just have noise. You filter back.

And you just keep the signal. And that's it. So that's sort of a brief intro to principal component analysis. But there are a lot of things you can use it for. It's a lot of fun. And it's a great intro to all the other amazing dimensionality reduction techniques that there are. I apologize for going over.

Practical Assessment, Research & Evaluation

A peer-reviewed electronic journal.

Copyright is retained by the first or sole author, who grants right of first publication to the *Practical Assessment, Research & Evaluation.* Permission is granted to distribute this article for nonprofit, educational purposes if it is copied in its entirety and the journal is credited.

Volume 15, Number 12, October, 2010 ISSN 1531-7714

Improving your data transformations: Applying the Box-Cox transformation

Jason W. Osborne, *North Carolina State University*

Many of us in the social sciences deal with data that do not conform to assumptions of normality and/or homoscedasticity/homogeneity of variance. Some research has shown that parametric tests (e.g., multiple regression, ANOVA) can be robust to modest violations of these assumptions. Yet the reality is that almost all analyses (even nonparametric tests) benefit from improved the normality of variables, particularly where substantial non-normality is present. While many are familiar with select traditional transformations (e.g., square root, log, inverse) for improving normality, the Box-Cox transformation (Box & Cox, 1964) represents a family of power transformations that incorporates and extends the traditional options to help researchers easily find the optimal normalizing transformation for each variable. As such, Box-Cox represents a potential best practice where normalizing data or equalizing variance is desired. This paper briefly presents an overview of traditional normalizing transformations and how Box-Cox incorporates, extends, and improves on these traditional approaches to normalizing data. Examples of applications are presented, and details of how to automate and use this technique in SPSS and SAS are included.

Data transformations are commonly-used tools that can serve many functions in quantitative analysis of data, including improving normality of a distribution and equalizing variance to meet assumptions and improve effect sizes, thus constituting important aspects of data cleaning and preparing for your statistical analyses. There are as many potential types of data transformations as there are mathematical functions. Some of the more commonly-discussed traditional transformations include: adding constants, square root, converting to logarithmic (e.g., base 10, natural log) scales, inverting and reflecting, and applying trigonometric transformations such as sine wave transformations.

While there are many reasons to utilize transformations, the focus of this paper is on transformations that improve normality of data, as both parametric and nonparametric tests tend to benefit from normally distributed data (e.g., Zimmerman, 1994, 1995, 1998). However, a cautionary note is in order. While transformations are important tools, they should be utilized thoughtfully as they fundamentally alter the nature of the variable, making the interpretation of the results somewhat more complex (e.g., instead of predicting student achievement test scores, you might be predicting the natural log of student achievement test scores). Thus, some authors suggest reversing the transformation once the analyses are done for reporting of means, standard deviations, graphing, etc. This decision ultimately depends on the nature of the hypotheses and analyses, and is best left to the discretion of the researcher.

Unfortunately for those with data that do not conform to the standard normal distribution, most statistical texts provide only cursory overview of best practices in transformation. Osborne (2002, 2008a) provides some detailed recommendations for utilizing traditional transformations (e.g., square root, log, inverse), such as anchoring the minimum value in a distribution at exactly 1.0, as the efficacy of some transformations are severely degraded as the minimum deviates above 1.0 (and having values in a distribution

less than 1.0 can cause mathematical problems as well). Examples provided in this paper will revisit previous recommendations.

The focus of this paper is streamlining and improving data normalization that should be part of a routine data cleaning process. For those researchers who routinely clean their data, Box-Cox (Box & Cox, 1964; Sakia, 1992) provides a family of transformations that will optimally normalize a particular variable, eliminating the need to randomly try different transformations to determine the best option. Box and Cox (1964) originally envisioned this transformation as a panacea for simultaneously correcting normality, linearity, and homoscedasticity. While these transformations often improve all of these aspects of a distribution or analysis, Sakia (1992) and others have noted it does not *always* accomplish these challenging goals.

Why do we need data transformations?

Many statistical procedures make two assumptions that are relevant to this topic: (a) an assumption that the variables (or their error terms, more technically) are normally distributed, and (b) an assumption of homoscedasticity or homogeneity of variance, meaning that the variance of the variable remains constant over the observed range of some other variable. In regression analyses this second assumption is that the variance around the regression line is constant across the entire observed range of data. In ANOVA analyses, this assumption is that the variance in one cell is not significantly different from that of other cells. Most statistical software packages provide ways to test both assumptions.

Significant violation of either assumption can increase your chances of committing either a Type I or II error (depending on the nature of the analysis and violation of the assumption). Yet few researchers test these assumptions, and fewer still report correcting for violation of these assumptions (Osborne, 2008b). This is unfortunate, given that in most cases it is relatively simple to correct this problem through the application of data transformations. Even when one is using analyses considered "robust" to violations of these assumptions or non-parametric tests (that do not explicitly assume normally distributed error terms), attending to these issues can improve the results of the analyses (e.g., Zimmerman, 1995).

How does one tell when a variable is violating the assumption of normality?

There are several ways to tell whether a variable deviates significantly from normal. While researchers tend to report favoring "eyeballing the data," or visual inspection of either the variable or the error terms (Orr, Sackett, & DuBois, 1991), more sophisticated tools are available, including tools that statistically test whether a distribution deviates significantly from a specified distribution (e.g., the standard normal distribution). These tools range from simple examination of skew (ideally between -0.80 and 0.80; closer to 0.00 is better) and kurtosis (closer to 3.0 in most software packages, closer to 0.00 in SPSS) to examination of P-P plots (plotted percentages should remain close to the diagonal line to indicate normality) and inferential tests of normality, such as the Kolmorogov-Smirnov or Shapiro-Wilk's W test (a $p > .05$ indicates the distribution does *not* differ significantly from the standard normal distribution; researchers wanting more information on the K-S test and other similar tests should consult the manual for their software (as well as Goodman, 1954; Lilliefors, 1968; Rosenthal, 1968; Wilcox, 1997)).

Traditional data transformations for improving normality

Square root transformation. Most readers will be familiar with this procedure-- when one applies a square root transformation, the square root of every value is taken (technically a special case of a power transformation where all values are raised to the one-half power). However, as one cannot take the square root of a negative number, a constant must be added to move the minimum value of the distribution above 0, preferably to 1.00. This recommendation from Osborne (2002) reflects the fact that numbers above 0.00 and below 1.0 behave differently than numbers 0.00, 1.00 and those larger than 1.00. The square root of 1.00 and 0.00 remain 1.00 and 0.00, respectively, while numbers above 1.00 always become smaller, and numbers between 0.00 and 1.00 become *larger* (the square root of 4 is 2, but the square root of 0.40 is 0.63). Thus, if you apply a square root transformation to a continuous variable that contains values between 0 and 1 as well as above 1, you are treating some numbers differently than others, which may not be desirable. Square root transformations are traditionally thought of as good for normalizing Poisson distributions (most common with

data that are counts of occurrences, such as number of times a student was suspended in a given year or the famous example of the number of soldiers in the Prussian Cavalry killed by horse kicks each year (Bortkiewicz, 1898) presented below) and equalizing variance.

Log transformation(s). Logarithmic transformations are actually a class of transformations, rather than a single transformation, and in many fields of science log-normal variables (i.e., normally distributed after log transformation) are relatively common. Log-normal variables seem to be more common when outcomes are influenced by many independent factors (e.g., biological outcomes), also common in the social sciences.

In brief, a logarithm is the power (exponent) a base number must be raised to in order to get the original number. Any given number can be expressed as y^x in an infinite number of ways. For example, if we were talking about base 10, 1 is 10^0, 100 is 10^2, 16 is $10^{1.2}$, and so on. Thus, $\log_{10}(100)=2$ and $\log_{10}(16)=1.2$. Another common option is the Natural Logarithm, where the constant e (2.7182818…) is the base. In this case the natural log of 100 is 4.605. As this example illustrates, a base in a logarithm can be almost any number, thus presenting infinite options for transformation. Traditionally, authors such as Cleveland (1984) have argued that a range of bases should be examined when attempting log transformations (see Osborne (2002) for a brief overview on how different bases can produce different transformation results). The argument that a variety of transformations should be considered is compatible with the assertion that Box-Cox can constitute a best practice in data transformation.

Mathematically, the logarithm of number less than 0 is undefined, and similar to square root transformations, numbers between 0 and 1 are treated differently than those above 1.0. Thus a distribution to be transformed via this method should be anchored at 1.00 (the recommendation in Osborne, 2002) or higher.

Inverse transformation. To take the inverse of a number (x) is to compute 1/x. What this does is essentially make very small numbers (e.g., 0.00001) very large, and very large numbers very small, thus reversing the order of your scores (this is also technically a class of transformations, as inverse square root and inverse of other powers are all discussed in the literature). Therefore one must be careful to reflect, or reverse the distribution prior to (or after) applying an inverse transformation. To reflect, one multiplies a variable by -1, and then adds a constant to the distribution to bring the minimum value back above 1.00 (again, as numbers between 0.00 and 1.00 have different effects from this transformation than those at 1.00 and above, the recommendation is to anchor at 1.00).

Arcsine transformation. This transformation has traditionally been used for proportions, (which range from 0.00 to 1.00), and involves of taking the arcsine of the square root of a number, with the resulting transformed data reported in radians. Because of the mathematical properties of this transformation, the variable must be transformed to the range −1.00 to 1.00. While a perfectly valid transformation, other modern techniques may limit the need for this transformation. For example, rather than aggregating original binary outcome data to a proportion, analysts can use logistic regression on the original data.

Box- Cox transformation. If you are mathematically inclined, you may notice that many potential transformations, including several discussed above, are all members of a class of transformations called power transformations. Power transformations are merely transformations that raise numbers to an exponent (power). For example, a square root transformation can be characterized as $x^{1/2}$, inverse transformations can be characterized as x^{-1} and so forth. Various authors talk about third and fourth roots being useful in various circumstances (e.g., $x^{1/3}$, $x^{1/4}$). And as mentioned above, log transformations embody a class of power transformations. Thus we are talking about a potential continuum of transformations that provide a range of opportunities for closely calibrating a transformation to the needs of the data. Tukey (1957) is often credited with presenting the initial idea that transformations can be thought of as a class or family of similar mathematical functions. This idea was modified by Box and Cox (1964) to take the form of the Box-Cox transformation:

$$y_i^\lambda = (y_i^\lambda - 1) / \lambda \text{ where } \lambda \neq 0;$$

$$y_i^\lambda = \log_e(y_i) \text{ where } \lambda = 0.$$[1]

[1] Since Box and Cox (1964) other authors have introduced modifications of this transformations for special applications and circumstances (e.g., John & Draper, 1980), but for most researchers, the original Box-Cox suffices and is preferable due to computational simplicity.

While not implemented in all statistical packages[2], there are ways to estimate lambda, the Box-Cox transformation coefficient using any statistical package or by hand to estimate the effects of a selected range of λ automatically. This is discussed in detail in the appendix. Given that λ can take on an almost infinite number of values, we can theoretically calibrate a transformation to be maximally effective in moving a variable toward normality, regardless of whether it is negatively or positively skewed.[3] Additionally, as mentioned above, this family of transformations incorporates many traditional transformations:

λ = 1.00: no transformation needed; produces results identical to original data

λ = 0.50: square root transformation

λ = 0.33: cube root transformation

λ = 0.25: fourth root transformation

λ = 0.00: natural log transformation

λ = -0.50: reciprocal square root transformation

λ = -1.00: reciprocal (inverse) transformation

and so forth.

Examples of application and efficacy of the Box-Cox transformation

Bortkiewicz's data on Prussian cavalrymen killed by horse-kicks. This classic data set has long been used as an example of non-normal (poisson, or count) data. In this data set, Bortkiewicz (1898) gathered the number of cavalrymen in each Prussian army unit that had been killed each year from horse-kicks between 1875 and 1894. Each unit had relatively few (ranging from 0-4 per year), resulting in a skewed distribution (presented in Figure 1; skew = 1.24), as is often the case in count data. Using square root, $\log_e$, and $\log_{10}$, will improve normality in this variable (resulting in skew of 0.84, 0.55, and 0.55, respectively). By utilizing Box-Cox with a variety of λ ranging from -2.00 to 1.00, we can determine that the optimal transformation after being anchored at 1.0 would be a Box-Cox transformation with λ = - 2.00 (see Figure 2) yielding a variable that is almost symmetrical (skew = 0.11; note that although transformations between λ = - 2.00 and λ = - 3.00 yield slightly better skew, it is not substantially better).

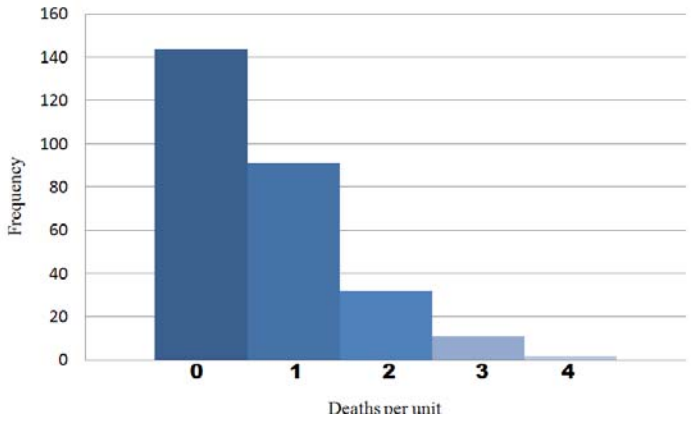


Figure 1. *Deaths from horse kicks, Prussian Army 1875-1984*

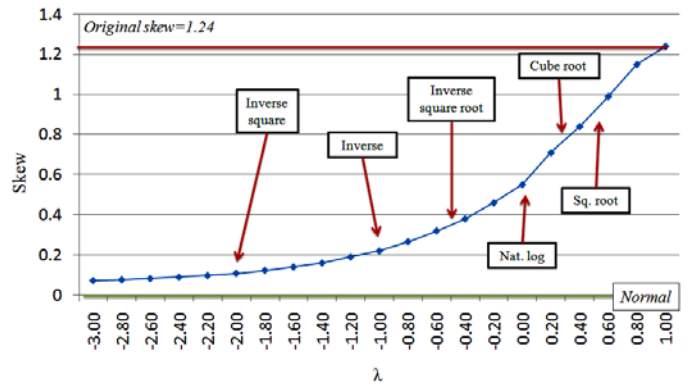


Figure 2.*Box-Cox transforms of horse-kicks with various λ*

University size and faculty salary in the USA. Data from 1161 institutions in the USA were collected on the size of the institution (number of faculty) and average faculty salary by the AAUP (American Association of University Professors) in 2005. As Figure 3 shows, the variable *number of faculty* is highly skewed (skew = 2.58), and Figure 4 shows the results of Box-Cox transformation after being anchored at 1.0 over the range of λ from -2.00 to 1.00. Because of the nature of these data (values ranging from 7 to over 2000 with a strong skew), this transformation attempt produced a wide range of outcomes across the thirty-two examples of Box-Cox transformation, from extremely bad outcomes (skew < -30.0 where λ < -1.20) to very positive

[2] For example, SAS has a convenient and very well done implementation of Box-Cox within *proc transreg* that iteratively tests a variety of λ and identifies the best options for you. Many resources on the web, such as http://support.sas.com/rnd/app/da/new/802ce/stat/chap15/sect8.htm provide guidance on how to use Box-Cox within SAS.

[3] Most common transformations reduce positive skew but may exacerbate negative skew unless the variable is reflected prior to transformation. Box –Cox eliminates the need for this.

outcomes of λ = 0.00 (equivalent to a natural log transformation) achieved the best result. (skew = 0.14 at λ = 0.00) . Figure 5 shows results of the same analysis when the distribution is anchored at the original mean (132.0) rather than 1.0. In this case, there are no extremely poor outcomes for any of the transformations, and one (λ = - 1.20) achieves a skew of 0.00. However, it is not advisable to stray too far from 1.0 as an anchor point. As Osborne (2002) noted, as minimum values of distributions deviate from 1.00, power transformations tend to become less effective. To illustrate this, Figure 5 shows the same data anchored at a minimum of 500. Even this relatively small change from anchoring at 132 to 500 eliminates the possibility of reducing the skew to near zero.

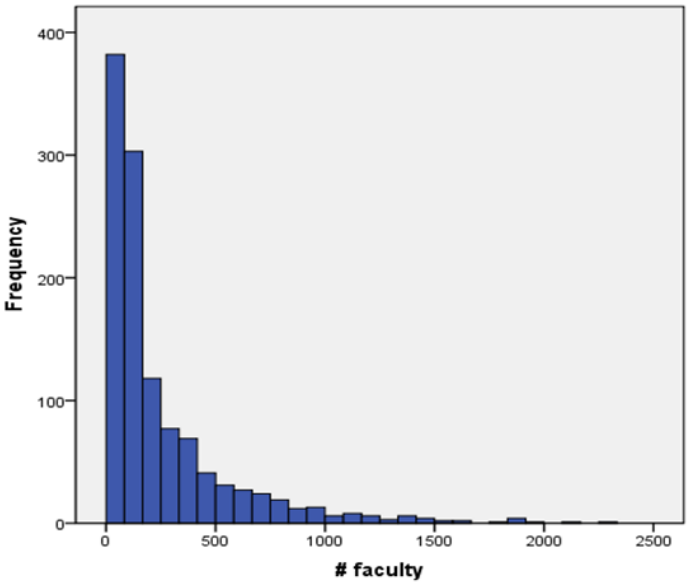


Figure 3. *Number of faculty at institutions in the USA*

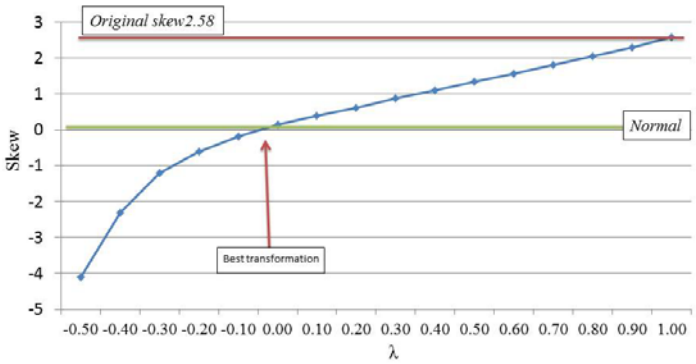


Figure 4. *Box-Cox transform of university size with various λ, anchored at 1.00*

Faculty salary (associate professors) was more normally distributed to begin with, with a skew of 0.36. A Box-Cox transformation with λ = 0.70 produced a skew of -0.03.

To demonstrate the benefits of normalizing data via Box-Cox a simple correlation between number of faculty and associate professor salary (computed prior to any transformation) produced a correlation of $r_{(1161)} = 0.49$, $p < .0001$. This represents a coefficient of determination (% variance accounted for) of 0.24, which is substantial yet probably under-estimates the true population effect due to the substantial non-normality present. Once both variables were optimally transformed, the simple correlation was calculated to be $r_{(1161)} = 0.66$, $p < .0001$. This represents a coefficient of determination (% variance accounted for) of 0.44, or an 81.5% increase in the coefficient of determination over the original.

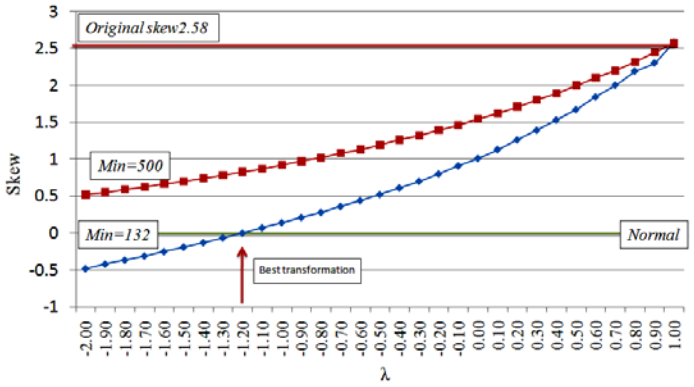


Figure 5. *Box-Cox transform of university size with various λ anchored at 132, 500*

Student test grades. Positively skewed variables are easily dealt with via the above procedures. Traditionally, a negatively skewed variable had to be *reflected* (reversed), anchored at 1.0, transformed via one of the traditional (square root, log, inverse) transformations, and reflected again. While this reflect-and-transform procedure also works fine with Box-Cox, researchers can merely use a different range of λ to create a transformation that deals with negatively skewed data. In this case I use data from a test in an undergraduate Educational Psychology class several years ago. These 174 scores range from 48% to 100%, with a mean of 87.3% and a skew of -1.75. Anchoring the distribution at 1.0 by subtracting 47 from all scores, and applying Box-Cox transformation from 1.0 to 4.0, we get the results presented in Figure 6,

indicating a Box-Cox transformation with a λ = 2.70 produces a skew of 0.02.

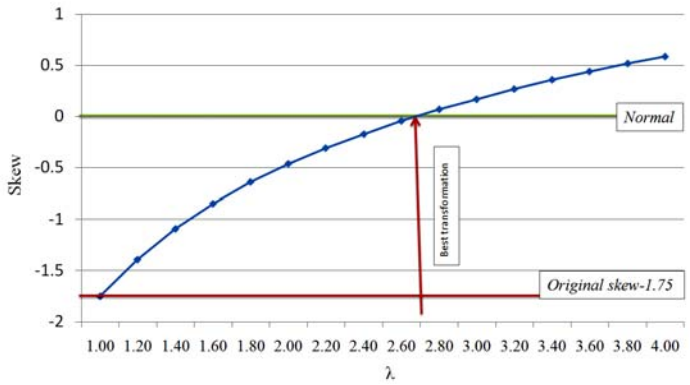


Figure 6. *Box-Cox transform of student grades, negatively skewed*

SUMMARY AND CONCLUSION

The goal of this paper was to introduce Box-Cox transformation procedures to researchers as a potential best practice in data cleaning. Although many of us have been briefly exposed to data transformations, few researchers appear to use them or report data cleaning of any kind (Osborne, 2008b). Box-Cox takes the idea of having a range of power transformations (rather than the classic square root, log, and inverse) available to improve the efficacy of normalizing and variance equalizing for both positively- and negatively-skewed variables.

As the three examples presented above show, not only does Box-Cox easily normalize skewed data, but normalizing the data also can have a dramatic impact on effect sizes in analyses (in this case, improving the effect size of a simple correlation over 80%).

Further, many modern statistical programs (e.g., SAS) incorporate powerful Box-Cox routines, and in others (e.g., SPSS) it is relatively simple to use a script (see appendix) to automatically examine a wide range of λ to quickly determine the optimal transformation.

Data transformations can introduce complexity into substantive interpretation of the results (as they change the nature of the variable, and any λ less than 0.00 has the effect of reversing the order of the data, and thus care should be taken when interpreting results.). Sakia (1992) briefly reviews the arguments revolving around this issue, as well as techniques for utilizing variables that have been power transformed in prediction or converting results back to the original metric of the variable. For example, Taylor (1986) describes a method of approximating the results of an analysis following transformation, and others (see Sakia, 1992) have shown that this seems to be a relatively good solution in most cases. Given the potential benefits of utilizing transformations (e.g., meeting assumptions of analyses, improving generalizability of the results, improving effect sizes) the drawbacks do not seem compelling in the age of modern computing.

REFERENCES

Bortkiewicz, L., von. (1898). *Das Gesetz der kleinen Zahlen.* Leipzig: G. Teubner.

Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society, B,, 26*(211-234).

Cleveland, W. S. (1984). Graphical methods for data presentation: Full scale breaks, dot charts, and multibased logging. *The American Statistician, 38*(4), 270-280.

Goodman, L. A. (1954). Kolmogorov-Smirnov tests for psychological research. . *Psychological Bulletin, 51*, 160-168.

John, J. A., & Draper, N. R. (1980). An alternative family of transformations. *applied statistics, 29*, 190-197.

Lilliefors, H. W. (1968). On the kolmogorov-smirnov test for normality with mean and variance unknown. *Journal of the American Statistical Association, 62*, 399-402.

Orr, J. M., Sackett, P. R., & DuBois, C. L. Z. (1991). Outlier detection and treatment in I/O Psychology: A survey of researcher beliefs and an empirical illustration. *Personnel Psychology, 44*, 473-486.

Osborne, J. W. (2002). Notes on the use of data transformations. *Practical Assessment, Research, and Evaluation., 8*, Available online at http://pareonline.net/getvn.asp?v=8&n=6 .

Osborne, J. W. (2008a). Best Practices in Data Transformation: The overlooked effect of minimum values. In J. W. Osborne (Ed.), *Best Practices in Quantitative Methods.* Thousand Oaks, CA: Sage Publishing.

Osborne, J. W. (2008b). Sweating the small stuff in educational psychology: how effect size and power reporting failed to change from 1969 to 1999, and what that means for the future of changing practices. *Educational Psychology, 28*(2), 1 - 10.

Rosenthal, R. (1968). An application of the kolmogorov-smirnov test for normality with estimated mean and variance. *Psychological-Reports, 22*(570).

Sakia, R. M. (1992). The Box-Cox transformation technique: A review. *The statistician, 41*, 169-178.

Taylor, M. J. G. (1986). the retransformed mean after a fitted power transformation. *Journal of the American Statistical Association, 81*, 114-118.

Tukey, J. W. (1957). The comparative anatomy of transformations. *Annals of Mathematical Statistics, 28*, 602-632.

Wilcox, R. R. (1997). Some practical reasons for reconsidering the Kolmogorov-Smirnov test. *British Journal of Mathematical and Statistical Psychology, 50*(1), 71-78.

Zimmerman, D. W. (1994). A note on the influence of outliers on parametric and nonparametric tests. *Journal of General Psychology, 121*(4), 391-401.

Zimmerman, D. W. (1995). Increasing the power of nonparametric tests by detecting and downweighting outliers. *Journal of Experimental Education, 64*(1), 71-78.

Zimmerman, D. W. (1998). Invalidation of parametric and nonparamteric statistical tests by concurrent violation of two assumptions. *Journal of Experimental Education, 67*(1), 55-68.

Citation:

Osborne, Jason (2010). Improving your data transformations: Applying the Box-Cox transformation. *Practical Assessment, Research & Evaluation*, 15(12). Available online: http://pareonline.net/getvn.asp?v=15&n=12.

Note:

The author wishes to thank to Raynald Levesque for his web page: http://www.spsstools.net/Syntax/Compute/Box-CoxTransformation.txt, from which the SPSS syntax for estimating lambda was derived.

Corresponding Author:

Jason W. Osborne
Curriculum, Instruction, and Counselor Education
North Carolina State University
Poe 602, Campus Box 7801
Raleigh, NC 27695-7801
919-244-3538
Jason_osborne@ncsu.edu

APPENDIX

Calculating Box-Cox λ by hand

If you desire to estimate λ by hand, the general procedure is to:

- divide the variable into at least 10 regions or parts,
- calculate the mean and s.d. for each region or part,
- Plot log(s.d.) vs. log(mean) for the set of regions,
- Estimate the slope of the plot, and use the slope (1-b) as the initial estimate of λ

As an example of this procedure, we revisit the second example, number of faculty at a university. After determining the ten cut points that divides this variable into even parts, selecting each part and calculating the mean and standard deviation, and then taking the $\log_{10}$ of each mean and standard deviation, Figure 7 shows the plot of these data. I estimated the slope for each segment of the line since there was a slight curve (segment slopes ranged from -1.61 for the first segment to 2.08 for the last) and averaged all, producing an average slope of 1.02. Interestingly, the estimated λ from this exercise would be -0.02, very close to the empirically derived 0.00 used in the example above.

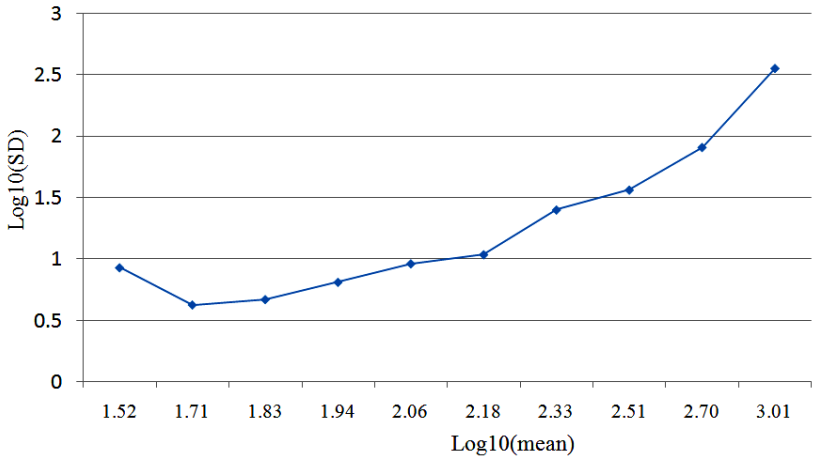


Figure 7. *Figuring λ by hand*

Estimating λ empirically in SPSS

Using the syntax below, you can estimate the effects of Box-Cox using 32 different lambdas simultaneously, choosing the one that seems to work the best. Note that the first COMPUTE anchors the variable (NUM_TOT) at 1.0, as the minimum value in this example was 7. You need to edit this to move your variable to 1.0.

```
*****************************.
*** faculty #, anchored 1.0
*****************************.

COMPUTE var1=num_tot-6.
execute.

VECTOR lam(31) /xl(31).
LOOP idx=1 TO 31.
- COMPUTE lam(idx)=-2.1 + idx * .1.
- DO IF lam(idx)=0.
-   COMPUTE xl(idx)=LN(var1).
- ELSE.
-   COMPUTE xl(idx)=(var1**lam(idx) - 1)/lam(idx).
- END IF.
END LOOP.
EXECUTE.
```

```
FREQUENCIES VARIABLES=var1 xl1 xl2 xl3 xl4 xl5 xl6 xl7 xl8 xl9 xl10 xl11 xl12 xl13 xl14 xl15
xl16 xl17 xl18 xl19 xl20 xl21 xl22 xl23 xl24 xl25 xl26 xl27 xl28 xl29 xl30 xl31
  /format=notable
  /STATISTICS=MINIMUM MAXIMUM SKEWNESS
  /HISTOGRAM
  /ORDER=ANALYSIS.
```

Note that this syntax tests λ from -2.0 to 1.0, a good initial range for positively skewed variables. There is no reason to limit analyses to this range, however, so that depending on the needs of your analysis, you may need to change the range of lamda tested, or the interval of lambda. To do this, you can either change the starting value on the above line:

```
- COMPUTE lam(idx)=-2.1 + idx * .1.
```

For example, changing -2.1 to 0.9 starts lambda at 1.0 for exploring variables with negative skew. Changing the number at the end (0.1) changes the interval SPSS examines—in this case it examines lambda in 0.1 intervals, but changing to 0.2 or 0.05 can help fine-tune an analysis.

FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 3

PRINCIPAL COMPONENT ANALYSIS AND VARIABLE TRANSFORMATION

Reading Time	60 minutes
Prior Knowledge	Linear Regression, Correlation, Linear Algebra
Keywords	Principal Component Analysis, Dimension Reduction, Standardized Variable, Covariance Matrix, Eigenvector & Eigenvalue, Raw Variables & Transformed Variables, Box and Cox Method

In the last lesson, we discussed the multicollinearity issue in a linear regression model. We proposed some methods to handle multicollinearity. The last one is principal component analysis. In this lesson, we will introduce principal component analysis and discuss how it can help to mitigate multicollinearity issue.

We then move to the next topic we brought up in Lesson 1: variable transformation. We will explain why we need to apply variable transformation and how to conduct transformation.

In [1]:
```
# Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.formula.api as smf
from scipy.stats import boxcox
from sklearn import preprocessing
from sklearn.decomposition import PCA
```

1. Principal Component Analysis

1.1 Overview

Often in a project, we need to handle many variables. For example, there are 500 stocks in the *S&P 500* index. If you are tasked to study this index, it would be time consuming to investigate each stock. You need a more efficient method to study the components in this stock index. Among many variables, usually, some of them can be correlated and some of them cannot. Correlated variables can cause a multicollinearity issue, as we

discussed in the last section. Is there a way to reduce the impact of multicollinearity and also reduce the amount of variables to a manageable level?

Principal Component Analysis (PCA) is a statistical technique that can help us tackle the above problems. PCA has the following properties:

1. PCA creates new variables called **principal components** from current variables in the dataset. All principal components are independent from each other.
2. Principal components will preserve as much information as possible in the current dataset. What do we mean by preserving information from a dataset? It means that the new principal components will be able to explain the variation in the dataset.
3. PCA also ranks each principal component's ability to explain the dataset variation. The first principal component has the highest ability to explain the variation of the dataset.

With the above properties from PCA, we can select the first few principal components instead of all variables for analysis. The benefit of using a smaller number of principal components to run analysis is important when there are too many variables to consider in a model. You can choose fewer variables without losing too much information and run the analysis. That's why PCA is also considered a "dimension reduction" method. The other great benefit of PCA is that all principal components are independent. Geometrically, this means that in the new coordinate system, all components are orthogonal or perpendicular to each other. Statistically, this means they are uncorrelated. When principal components are selected to run a regression model, there won't be a multicollinearity issue.

1.2 The Basics of Principal Components

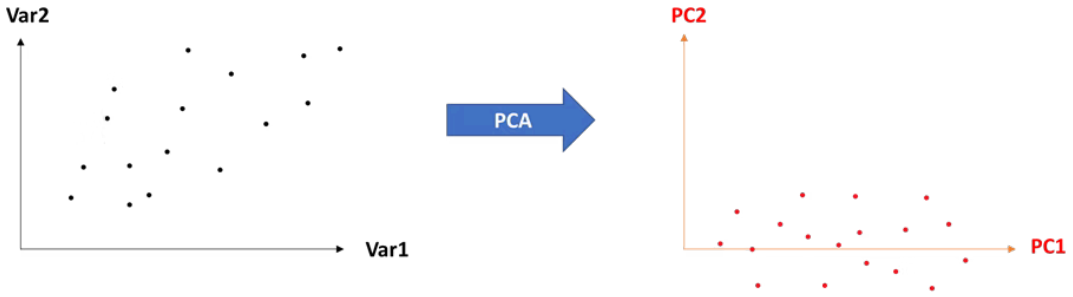
In this section, we are going to take a deeper look at principal components. As discussed in the above section, PCA creates principal components (new variables) from original variables in a dataset. A principal component is a linear function of the original variables. For example, if we have two variables in a dataset, Var_1 and Var_2, then PCA will transform Var_1 and Var_2 and create the following two principal components:

$$PC_1 = a_1\mathrm{Var}_1 + a_2\mathrm{Var}_2$$

$$PC_2 = b_1\mathrm{Var}_1 + b_2\mathrm{Var}_2$$

Since we only have two variables in our example, we can draw two-dimensional graphs to demonstrate the concept.

Figure 1: Principal Component Analysis Data Transformation Graph



The scatter plot on the left of Figure 1 is the dataset with original variables. Through the transformation of PCA, we can see the new data distribution under principal components on the right of Figure 5. In the left graph, the data points scatter between both Var_1 and Var_2 axes. After the PCA transformation, new data points now mostly gather round the PC_1 axis. What does this mean for data analysis? In the original dataset, you need Var_1 and Var_2 to explain the variation of the data points. In the transformed dataset from PCA, PC_1 alone can explain most of the variation in the dataset. PC_2 doesn't provide much help in explaining the variation. We can also see that PC_1 and PC_2 form a 90-degree angle. We can say that PC_1 and PC_2 are orthogonal, meaning that PC_1 and PC_2 are independent. Because PC_1 explains most of the data variation, you can use one variable instead of two for analysis. In other words, instead of analyzing the data in two dimensions, you can analyze the data in one dimension now. That's why many scientists use PCA as a dimension reduction method for their data analysis work.

1.3 Derivation of Principal Components

From the example in the last section, we know we need to get the transformation matrix

$$\begin{pmatrix} a_1 & a_2 \\ b_1 & b_2 \end{pmatrix}$$

in order to get principal components. In this section, we will outline the steps for how to get the transformation matrix.

1.3.1 Calculate Standardized Dataset

In the above example, we have Var_1 and Var_2 in our dataset. We need to standardize Var_1 and Var_2. Here is the formula to standardize a variable:

$$Standardized\ Variable = \frac{Original\ Variable - Mean}{Standard\ Deviation}$$

All the standardized variables will have the same scale, which means mean = 0 and variance = 1.

1.3.2 Calculate Covariance Matrix of the Standardized Variables

Covariance is a metric to determine the relationship of the two variables.

$$\mathrm{Covariance} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{N}$$

where N is the number of data points

A positive covariance means that the two variables are moving in the same direction and vice versa.

A covariance matrix is a matrix that contains all pairwise covariances from all variables. From our two-variable example, a covariance matrix for two variables will be a 2 X 2 covariance matrix as follows.

$$\begin{pmatrix} \mathrm{Cov}(Var_1, Var_1) & \mathrm{Cov}(Var_2, Var_1) \\ \mathrm{Cov}(Var_1, Var_2) & \mathrm{Cov}(Var_2, Var_2) \end{pmatrix}$$

The diagonal elements of the covariance matrix are also the variance of the variables according to covariance's formula.

1.3.3 Calculate Eigenvectors and Eigenvalues

Once we obtain the covariance matrix, we can use linear algebra to calculate eigenvectors and eigenvalues from the covariance matrix. What are eigenvectors and eigenvalues? It is best to explain them in our example.

In our two-variable example, (a_1, a_2) and (b_1, b_2) are the two column vectors in our transformation matrix

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}$$

We know (a_1, a_2) are the coefficients to transform Var_1 and Var_2 to PC_1. (a_1, a_2) is the eigenvector for PC_1 and (b_1, b_2) is the eigenvector for PC_2. The eigenvalue of PC_1 is the value that shows the amount of data variance PC_1 explains. The eigenvalue of PC_2 is the value that shows the amount of data variance PC_2 explains. Since PC_1 is the first principal component, its eigenvalue is the largest. The way PCA orders principal components is that the principal component with the largest eigenvalue is the first one and the one with the second largest eigenvalue will be the second. In general, most software doesn't readily provide eigenvalues. They instead provide a proportion of the total variance explained by the principal component. This proportion is calculated from the eigenvalue of the principal component.

1.3.4 Transform the Data Using PCA

After getting the transformation matrix

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}$$

we can generate new variables, PC_1 and PC_2

$$PC_1 = a_1\mathrm{Var}_1 + a_2\mathrm{Var}_2$$

$$PC_2 = b_1\mathrm{Var}_1 + b_2\mathrm{Var}_2$$

2. Apply Principal Component Analysis to KO's Excess Return Project

In our *KO* excess return project, we find there is multicollinearity among independent variables. In this section, let's use PCA to see if we could reduce independent variables in the model and mitigate multicollinearity.

Figure 2: Principal Component Analysis Result Summary for 8 Independent Variables

In [2]:
```
# Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

In [3]:
```
# Select 8 variables and standardise
pc = data_set[
    [
        "Dow_Q_EX_R",
        "Pepsi_Q_EX_R",
        "REV_Q_R",
        "GOOG_Q_EX_R",
        "BAC_Q_EX_R",
        "WMT_Q_EX_R",
        "GE_Q_EX_R",
        "SBUX_Q_EX_R",
    ]
]
pca_data = preprocessing.scale(pc)
pca = PCA(n_components=8)
pca.fit(pca_data)
```

Out[3]:

PCA

PCA(n_components=8)

In [4]:
```
# Get proportions of variance and cumulative proportion of variance
pr_var = pca.explained_variance_ratio_
cum_pr = np.cumsum(pca.explained_variance_ratio_)
ind = ["Proportion of variance", "Cumulative proportion of variance"]
cols = ["PC1", "PC2", "PC3", "PC4", "PC5", "PC6", "PC7", "PC8"]
pd.DataFrame(np.vstack((pr_var, cum_pr)), ind, columns=cols)
```

Out[4]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	P
Proportion of variance	0.417539	0.193287	0.171490	0.108867	0.042000	0.028306	0.026893	0.011
Cumulative proportion of variance	0.417539	0.610826	0.782316	0.891183	0.933183	0.961489	0.988381	1.000

Let's look at the proportion of variance and cumulative proportion in the above chart. According to the chart, we can see that the first principal component accounts for 42% of

the total variance in the data. The first six principal components explain about 96% of the total variance.

Next, we would like to see how PCA can help us reduce the number of independent variables in the model. We know each principal component is a linear function of eight original independent variables. We can choose the original variable with the highest absolute value of the coefficient in the linear combination as a proxy of that principal component.

Figure 3: Coefficients (Loadings) of 8 Principal Components

In [5]:
```
# Coefficients (Loadings) of 8 Principal Components
pc_res = pd.DataFrame(pca.components_.T, index=list(pc.columns), columns=cols)
pc_res
```

Out[5]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Dow_Q_EX_R	0.501013	0.081975	-0.201491	0.151476	0.172552	-0.161302	-0.116793
Pepsi_Q_EX_R	0.370323	0.416369	0.215351	-0.267349	-0.486188	-0.340799	-0.42313
REV_Q_R	0.089143	-0.205496	0.723855	-0.314653	0.525310	0.024685	-0.213269
GOOG_Q_EX_R	0.481635	0.115076	0.033447	-0.219037	-0.147889	0.789841	0.241923
BAC_Q_EX_R	0.399072	-0.169106	-0.470595	0.014795	0.462351	-0.034007	-0.323626
WMT_Q_EX_R	0.116011	0.659655	0.213805	0.389464	0.370728	-0.129019	0.392582
GE_Q_EX_R	0.399918	-0.444523	0.093024	-0.117590	-0.150220	-0.440478	0.622124
SBUX_Q_EX_R	0.198186	-0.321096	0.334835	0.769979	-0.249503	0.146820	-0.239618

Figure 3 shows the coefficients of eight independent variables for each principal component. The coefficients here can also be called loadings. For PC_1, the Dow 30 Index excess return has the highest absolute value of the coefficient. This means that the Dow 30 Index excess return has the largest impact on PC_1. Therefore, we can use the Dow 30 Index excess return as a proxy for PC_1. Following the same logic, we can use Walmart's excess return as a proxy for PC_2. We also know that the first six principal components cover around 96% of the data variation. In this case, we can just choose *Dow 30* Index excess return, Walmart excess return, *KO*'s revenue growth, Starbucks excess return, and Google excess return to run a regression model.

Let's rerun to our revised regression model with fewer independent variables to see the result.

Figure 4: KO Excess Return Revised Model Result Applying PCA Information

In [9]:
```
# OLS for revised model
model_4 = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + REV_Q_R + GOOG_Q_EX_R + WMT_Q_EX_R + SBUX_Q_EX_R
    data=data_set,
```

```
).fit()
model_4.summary()
```

Out[9]:

OLS Regression Results

Dep. Variable:	Coke_Q_EX_R	**R-squared:**	0.584
Model:	OLS	**Adj. R-squared:**	0.454
Method:	Least Squares	**F-statistic:**	4.493
Date:	Sun, 23 Nov 2025	**Prob (F-statistic):**	0.00950
Time:	00:05:39	**Log-Likelihood:**	38.030
No. Observations:	22	**AIC:**	-64.06
Df Residuals:	16	**BIC:**	-57.51
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.0109	0.014	-0.757	0.460	-0.042	0.020
Dow_Q_EX_R	0.6091	0.286	2.133	0.049	0.004	1.214
REV_Q_R	0.1418	0.103	1.380	0.186	-0.076	0.359
GOOG_Q_EX_R	0.1060	0.177	0.600	0.557	-0.268	0.480
WMT_Q_EX_R	-0.0004	0.144	-0.003	0.998	-0.305	0.304
SBUX_Q_EX_R	0.0551	0.117	0.472	0.644	-0.193	0.303

Omnibus:	1.148	**Durbin-Watson:**	1.998
Prob(Omnibus):	0.563	**Jarque-Bera (JB):**	1.031
Skew:	-0.466	**Prob(JB):**	0.597
Kurtosis:	2.493	**Cond. No.**	30.4

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

From the result in Figure 4, we can see that the *Dow 30* Index has become significant, and the adjusted R^2 has improved from 0.406 at the beginning of this lesson to 0.454. By using the information provided by PCA, we can reduce the impact of multicollinearity among independent variables.

2.1 Use Principal Components in a Regression Model

As you can see from the previous example, it is hard to integrate principal components directly into a regression model. If we use a principal component directly as an

independent variable in a regression model, it will be hard to interpret the meaning of the coefficient estimate. Since principal components are a linear combination of several financial variables, they don't translate directly into a specific financial concept. However, we can investigate the loadings for each principal component and pick the original variable with the highest absolute value of the loadings as a proxy for that principal component for regression. This is what we did in the last example. Another way is to also look for a financial theme from the loadings of a principal component and select original variables related to the theme.

We have also noticed that by selecting the top variables from PCA for data analysis, we lose some accuracy in representing complete data variation. We mitigate the multicollinearity from the dataset at the expense of losing some ability to explain the whole data variation. How to choose the degree of the trade-off between reducing multicollinearity and losing data explaining accuracy will depend on a researcher's experience in data analysis.

3. Variable Transformation

In the previous sections, we looked at the relationships among variables and their impact on linear regression model results. In this section, we turn our focus to the individual variable itself. In the field of data analysis, oftentimes, the raw data we collect is not ready for analysis. Even if the data is in a form ready for analysis, sometimes, the variables in the dataset don't meet the model assumptions and therefore require some transformation work. In this section, we will discuss when and how to transform variables as well as choices for transforming variables and how to analyze regression models with transformed variables.

3.1 Raw Variables and Transformed Variables

A **raw variable** is a variable in its original form when the data is collected. The concept is different from raw data. Raw data is the dataset that comes straight from the data source. There might be missing values, outliers, wrong data input, etc. A researcher often needs to go through certain steps to clean the dataset for use. A raw variable is a variable from the cleaned dataset that has not otherwise been manipulated. For example, the X variable is from a cleaned dataset. X is a raw variable. If $Y = 2X$, then Y is not a raw variable. Y is a **transformed variable**.

3.2 When Do We Need to Transform Variables?

Situation 1: The raw variables are not in the proper form for analysis.

Sometimes, the data you collect is not in a suitable or practical form for your analysis. For example, in our *KO* stock excess return model, the raw variables we obtained from the internet are *KO*'s stock price, Google's stock price, etc. However, their stock price levels are different. Once we transform stock prices to price returns, the two variables are now on the same scale and easier to compare and analyze.

Sometimes, there are not many raw variables in a dataset. As a data scientist, you can try to create new variables by manipulating raw variables. For example, suppose you are analyzing consumer activities in a supermarket. You have each consumer's sales numbers and the number of trips made in a year. From the two raw variables, you can create a new variable called *Sales Per Trip*, which is sales number divided by number of trips. This new variable might provide new insight for the analysis. Alternatively, perhaps you might form a variable that looks at related purchases, such as always buying two items together. This new variable might provide different insight for the analysis.

Situation 2: The raw variables don't meet analysis assumptions.

In financial models, usually there are assumptions about the variables in the model. For example, in a regression model, the key assumption is that the dependent variable and independent variables have a linear relationship. For a simple regression model, it is easy to draw a scatter plot for the dependent variable and independent variable to see if they form a linear relationship in the plot.

Figure 5: Non-Linear Relationship between X and Y

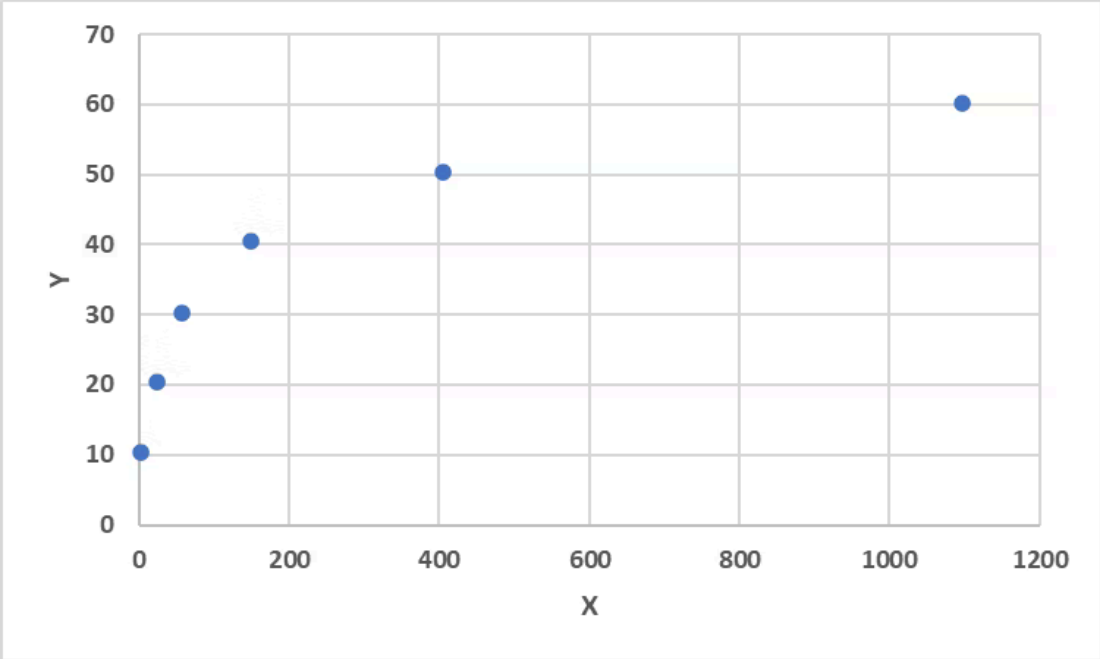
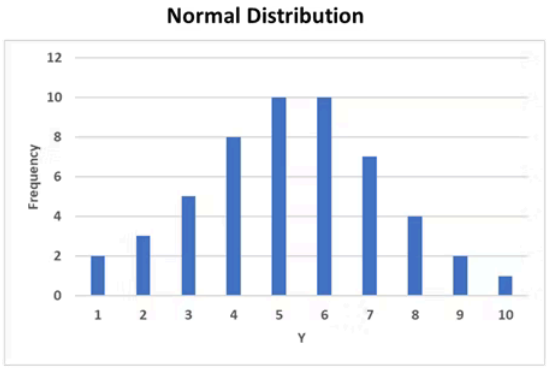
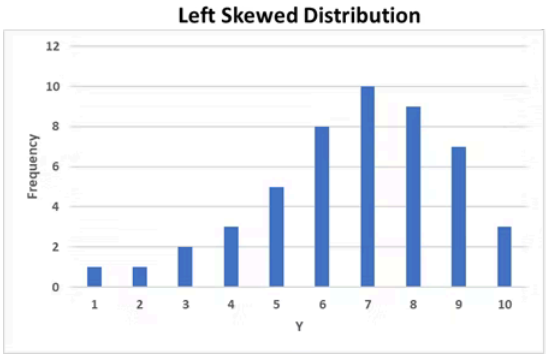


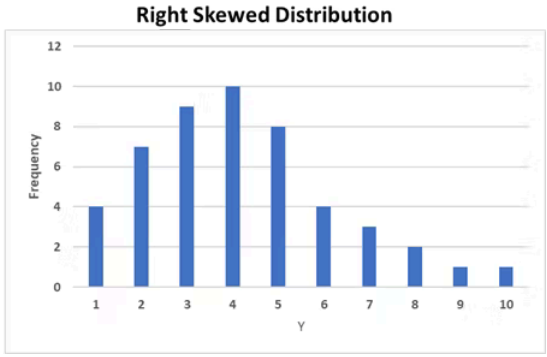
Figure 5 is a scatter plot for two variables, X and Y. We can see X and Y don't have a linear relationship in the plot.

Another assumption in many financial models to consider is whether a variable is normally distributed. Suppose one of the assumptions for a financial model is to required variable Y to be normally distributed. We can draw a histogram for the variable and see if the histogram shows a bell-shaped pattern. Figure 6 shows three histograms for three different Y distributions.

Figure 6: Histograms with Different Distributions







3.3 Methods to Transform Variables

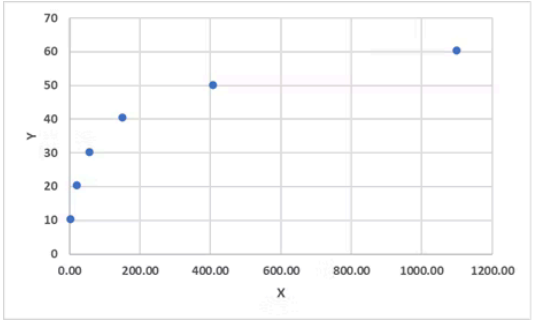
Method 1: Transformation for Linear Relationship

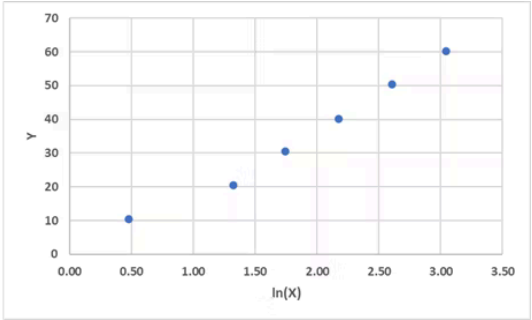
If one assumption of a model requires variables to be linearly dependent but this relationship doesn't exist, we need to transform the variables so that the linear relationship is present. The methods to try are log transformation, square root transformation, or cube root transformation. These methods have two things in common:

- They preserve the order of the data points
- They "tame" outliers by making them less extreme. Note the change in the scales between the two graphs.

Following up from the example in Figure 5, we can do a log transformation on X. In Figure 7, we can see the original scatter plot from Figure 5 on the left and the transformed X in the scatter plot on the right. You can see that there is a linear relationship between Y and $\ln(X)$.

Figure 7: Pre-Transformation Scatter Plot vs. Post-Transformation Scatter Plot





Sometimes, you may need to transform both X and Y to obtain the linear relationship for the two variables. Variable transformation is a trial and error process. Based on your model assumptions, you will need to try to either transform one variable or two or even more to achieve your modeling goal.

Method 2: Transformation for Normal Distribution

When a model requires a variable to be normally distributed but the variable does not meet the assumption, we need to transform this variable to be normally distributed. We can use log transformation, square root transformation, or cube root transformation to get the result. One more thing to consider is the outliers for a non-normal variable with skewed distribution. If you drop the outliers on the tail, it may bring the variable back to normal distribution.

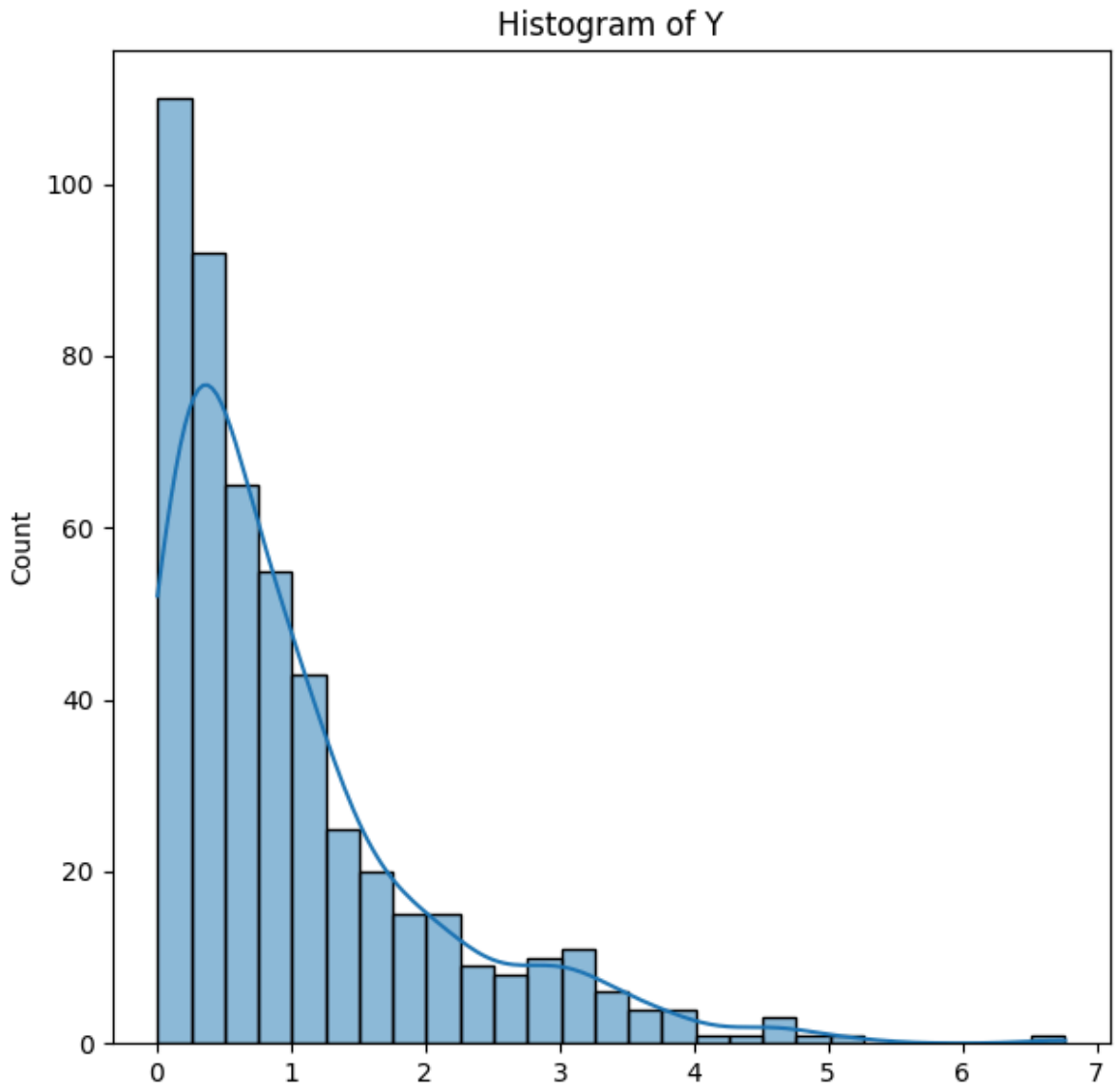
Another method to consider is the **Box and Cox method**. Box and Cox method is a variable transformation method that can transform a non-normally distributed variable to a normally distributed variable. Assume we have a variable Y which is not normally distributed. The Box and Cox method will try to find a λ satisfying the following rules and converting Y to a normally distributed variable.

$$\text{new}Y = \begin{cases} \frac{(Y^{\lambda}-1)}{\lambda}, & \text{if } \lambda \neq 0 \\ \log(Y), & \text{if } \lambda = 0 \end{cases}$$

Box and Cox Method Transformation

In [7]:
```
# Generate random dataset
np.random.seed(0)  # to make reproducible example
Y = np.random.exponential(size=500)

# Plot distribution of data
plt.figure(figsize=(7, 7))
plt.title("Histogram of Y")
sns.histplot(Y, kde=True)
plt.show()
```

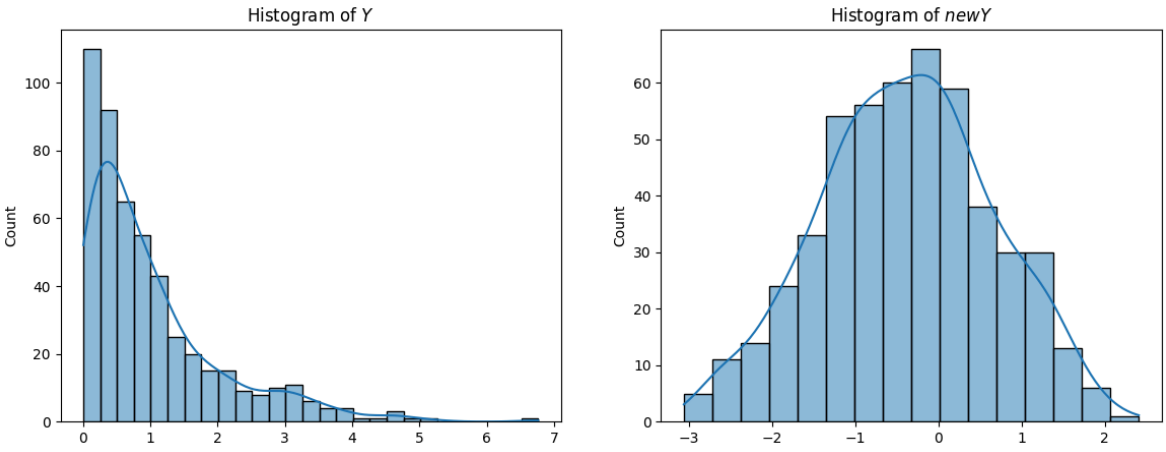


In [8]:
```
# Apply Box-Cox transformation on original data
new_Y, best_lambda = boxcox(Y)
print("Optimal Box-Cox transformation lambda:", best_lambda)

# Plot distributions of original data
plt.figure(figsize=(14, 5))
ax1 = plt.subplot(121)
sns.histplot(Y, kde=True, ax=ax1)
plt.title("Histogram of $Y$")

# Plot distribution of transformed data
ax2 = plt.subplot(122)
sns.histplot(new_Y, kde=True, ax=ax2)
plt.title("Histogram of $newY$")
plt.show()
```

```
Optimal Box-Cox transformation lambda: 0.2325441269447074
```



The figure above shows us the histogram of our original variable Y on the left and the histogram of the transformed variable Y using the Box and Cox method on the right. We can see from the figure that the original variable Y is right skewed. With Box and Cox transformation, the new variable is normally distributed.

3.4 Concluding Words for Variable Transformation

Variable transformation is both an art and a science. Oftentimes, we need to draw a scatter plot and/or histogram to check the variables. Then, we apply different transformations to the variable. We use the transformed variable in the regression model and check the coefficient significance and model fit. If one transformation doesn't work, we try another transformation. It's an iterative process until the regression model result is good.

4. Conclusion

In this lesson, we first introduced principal component analysis. We explained what principal components are and demonstrated the derivation of principal components. We then learned how to use the information obtained from principal component analysis to rerun a regression model to mitigate the multicollinearity issue. The second topic we discussed in this lesson is variable transformation. We showed that when the dependent variable and independent variables are not linearly related, we need to transform independent variables to make sure a linear relationship exists to run a regression model. We also showed what to do when we need to convert a non-normally distributed variable to a normally distributed one.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 4

MONTE CARLO SIMULATION

Reading Time	60 minutes
Prior Knowledge	Time series analysis, Basic statistics
Keywords	Deterministic Model, Stochastic Model, Monte Carlo Simulation, Law of Large Numbers, Central Limit Theorem

In the previous lessons, we learned that a linear regression model always includes an error term. This error term provides a random characteristic to the prediction from the linear regression model. In this lesson, we will explore this random characteristic more. Specifically, we will discuss deterministic models and stochastic models. Then, we will introduce Monte Carlo simulation.

In [1]:
```
# Load libraries
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams["figure.figsize"] = (12, 9)  # Figure size and width
```

1. Deterministic Models

Deterministic models are models that do not have a random term. Let's look at one example. Assume you put $10 dollars in an investment account at the beginning of each month. The account would give you a **fixed annual interest** of 40% compounding monthly. We can use the following equation to calculate the amount of money you will get by the end of a year.

$$\sum_{i=0}^{11} P_t(1+\frac{r}{12})^{12-i}$$

where P_t is the amount of money put in at the beginning of each month and r is the fixed annual interest rate.

Figure 1: Deterministic Model: Cumulative Account Balance with Monthly $10 Dollars Investment and Fixed Annual Interest

In [2]:
```
# Deterministic Model
month = np.linspace(start=1, stop=12, num=12)
```

```
x = []

for i in range(12):
    x.append(10 * (1 + 0.4 / 12) ** (12 - i))

cum_x = np.cumsum(x)

# Plot
plt.plot(cum_x, marker="o")
plt.title("Deterministic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(
    round(cum_x[11], 2), (10.5, cum_x[11] - 10), fontsize=13
)  # Annotate last point
plt.show()
```

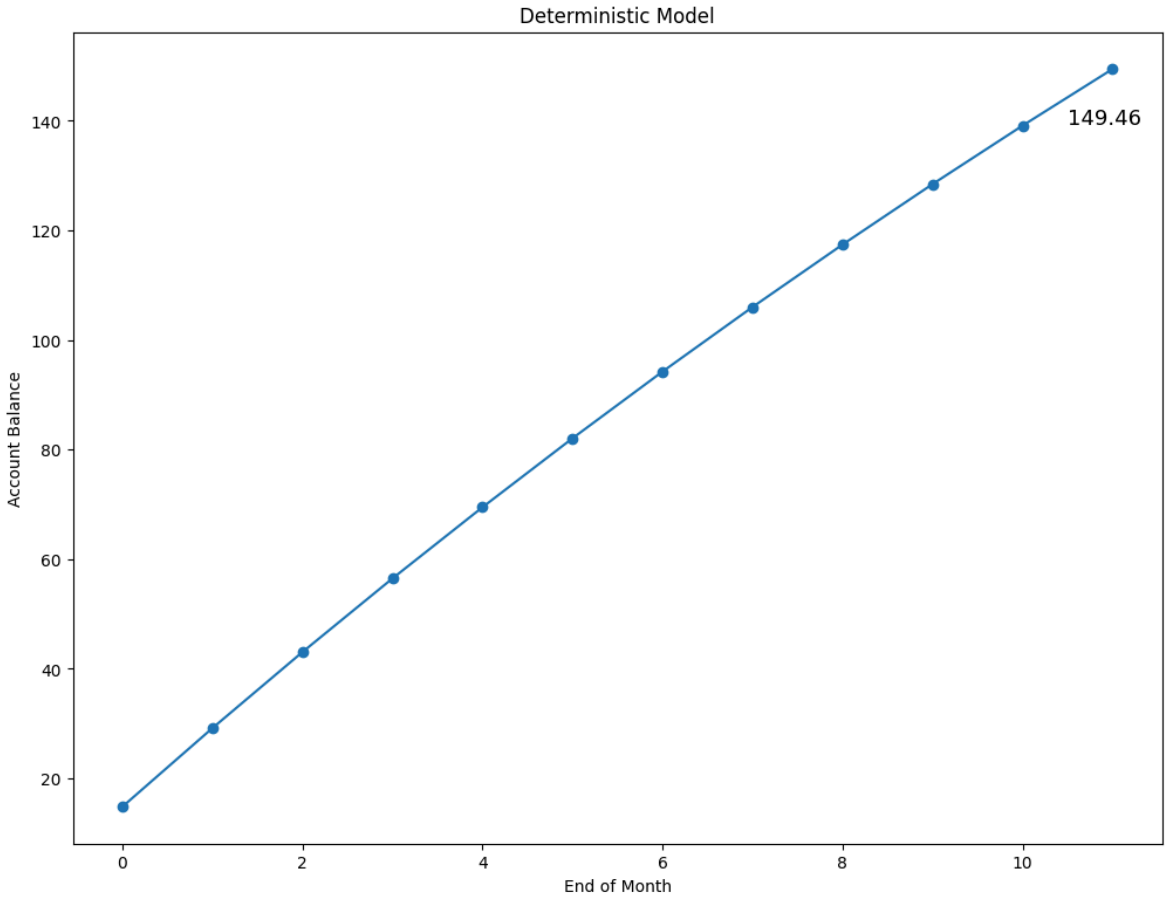


Figure 1 shows the cumulative account balance at the end of each month. With this model, we can accurately predict how much money is in the account at each month's end. There is no uncertainty within this model. If one invests $10 dollars at the beginning of every month, one will get $149.46 dollars by the end of the year. One can be 100% sure how much money will be in the account after one year if the amount of monthly investment is known in advance.

A deterministic model will produce the same result given the input information is fixed. Repeated prediction processes with the same input for a deterministic model will not change the prediction outcome. Other common deterministic models are principal component analysis and factor analysis. The classification or cluster analysis we will

discuss later in Lesson 4 is also a deterministic model. In contrast to the deterministic model, a stochastic model will have a random component in the model.

2. Stochastic Models

A stochastic model is a model that has a random and unknown component in it. Let's go back to the example in the last section. Assume you put $10 dollars in an investment account at the beginning of each month. However, now the account would give you a **variable annual interest** adjusting and compounding monthly. The annual interest rate is a random variable and, for instance, follows a normal distribution with mean 40%. We cannot know the definitive numbers of all future interest revenues from the investment at time 0. There is uncertainty in predicting the future cumulative account balance. We can use the following equation to calculate the amount of money you will get by the end of a year.

$$\sum_{i=0}^{11} P_t(1+\frac{r_t}{12})^{12-i}$$

where P_t is the amount of money put in at the beginning of each month and r_t is the annual interest rate adjusted each month.

In this case, we have introduced an uncertain component, the variable annual interest rate. In the following figure, we sample 12 interest rates from the normal distribution with mean 40% and apply them to the above model to come up with one possible account balance outcome for each month in a year.

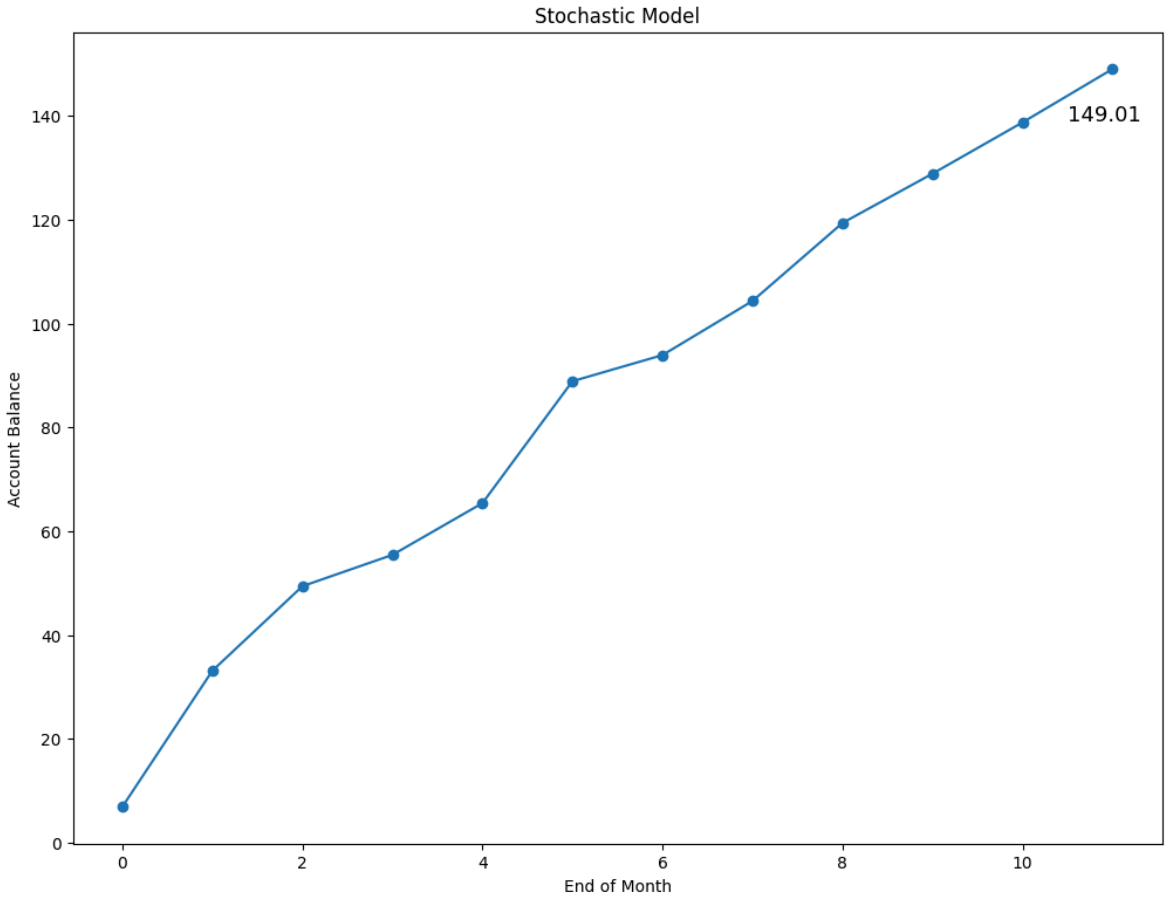
Figure 2: Stochastic Model: Cumulative Account Balance with a Monthly $10 Dollar Investment and Variable Annual Interest

In [3]:
```
# Stochastic model 1
np.random.seed(123)
int_1 = np.random.normal(0.4, 0.7, 12)  # np.random.normal(mean, sd, size)
bal_1 = []

for i in range(12):
    bal_1.append(10 * (1 + int_1[i] / 12) ** (12 - i))

cum_bal_1 = np.cumsum(bal_1)

# Plot
plt.plot(cum_bal_1, marker="o")
plt.title("Stochastic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(round(cum_bal_1[11], 2), (10.5, cum_bal_1[11] - 10), fontsize=13)
plt.show()
```



From figure 2, we can see the trend in the stochastic model is going up and is similar to the case for the deterministic model. However, a stochastic model has a squiggly line instead of a smooth line as in a deterministic model. This squiggly pattern comes from the fact that the variable annual interest rates applied to each month are different. Unlike a deterministic model, at time 0, the stochastic model cannot predict a precise number for what the account balance will be by the end of the year. We know the variable annual interest comes from a normal distribution with a mean of 40%, but we do not know the definite numbers. What we can do is sample 12 numbers from the distribution and apply them to the model to predict the account balance in a year. This is what we did in this example. However, if we repeat the same prediction process—sample another 12 numbers from the distribution and apply them to the model—we will very likely get another predicted account balance in a year. Every time we repeat the prediction process, we get a different account balance. The key difference between the deterministic model and stochastic model is that the prediction from the deterministic model is always the same no matter how many times one repeats the prediction process. However, the prediction outcome will be different from a stochastic model each time a researcher repeats the prediction process as shown in the following plot.

Figure 3: Three Different Prediction Paths for a Stochastic Investment Account Balance Model

In [4]:
```
# Three different prediction paths

# Stochastic model 2
np.random.seed(375)
int_2 = np.random.normal(0.4, 0.7, 12)
```

```
bal_2 = []

for i in range(12):
    bal_2.append(10 * (1 + int_2[i] / 12) ** (12 - i))

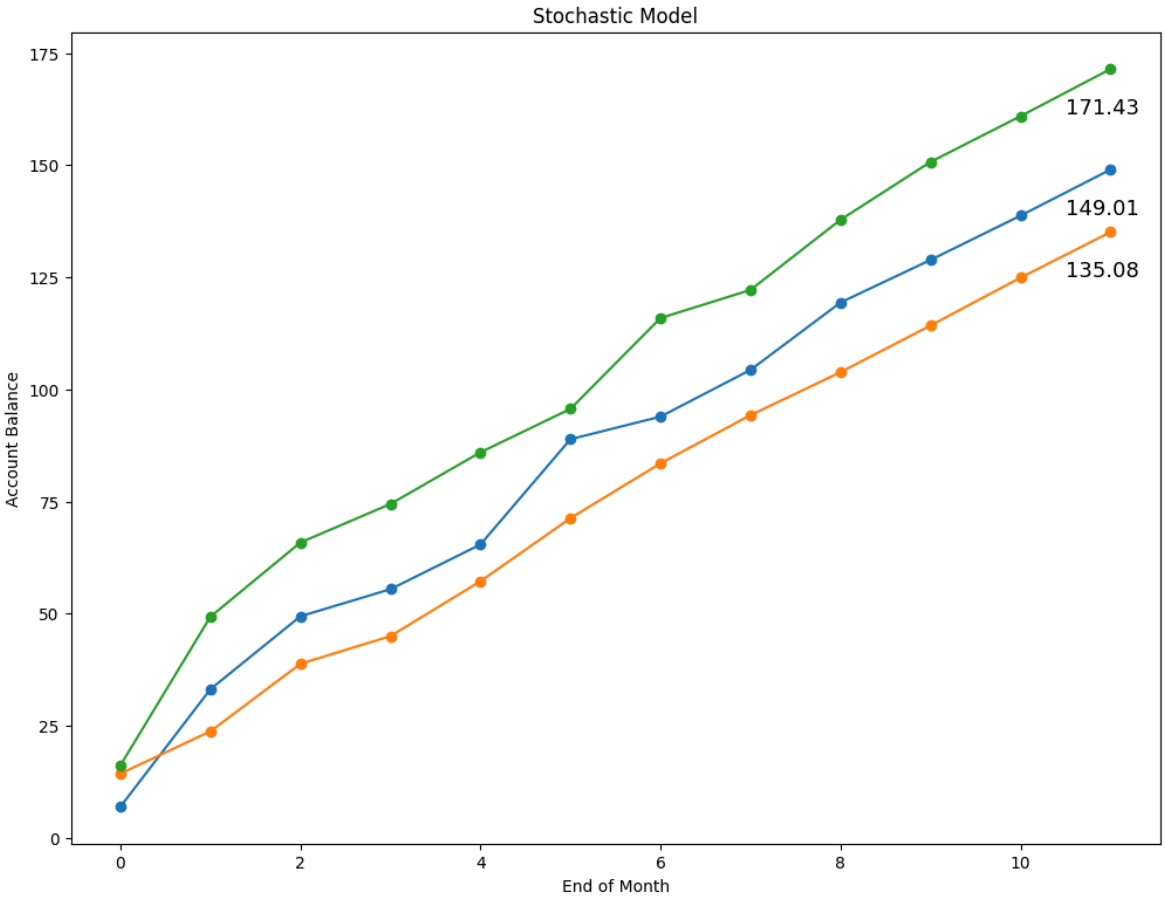
cum_bal_2 = np.cumsum(bal_2)


# Stochastic model 3
np.random.seed(999)
int_3 = np.random.normal(0.4, 0.7, 12)
bal_3 = []

for i in range(12):
    bal_3.append(10 * (1 + int_3[i] / 12) ** (12 - i))

cum_bal_3 = np.cumsum(bal_3)


# Plot three stochastic model lines
plt.plot(cum_bal_1, marker="o")
plt.plot(cum_bal_2, marker="o")
plt.plot(cum_bal_3, marker="o")
plt.title("Stochastic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(round(cum_bal_1[11], 2), (10.5, cum_bal_1[11] - 10), fontsize=13)
plt.annotate(round(cum_bal_2[11], 2), (10.5, cum_bal_2[11] - 10), fontsize=13)
plt.annotate(round(cum_bal_3[11], 2), (10.5, cum_bal_3[11] - 10), fontsize=13)
plt.show()
```



In figure 3, we show three predicted cumulative account balance paths of the stochastic model. As you can see, all three of them follow the same trend in general with different variations. Their predicted outcomes for cumulative account balances by the end of a year are also different, as the plot indicates. This is the key difference between a deterministic model and a stochastic model. In a stochastic model, for each prediction activity, because of the random component in the model, you get a new line when you have a new set of sampled input data. In a deterministic model, no matter how many times you run prediction activities, you should always get the same line.

The linear regression model is an example of stochastic models. As long as there is a random element with a probability distribution in a model, this model is a stochastic model.

3. Monte Carlo Simulation

3.1 An Overview of Monte Carlo Simulation

In the last section, we learned that a stochastic model would generate a different prediction outcome for each prediction run due to the random component within the model. The prediction outcome of a stochastic model is not definite. We can generate samples from independent random variables in a stochastic model and apply them to the model to derive one predicted outcome. In the last section, we drew three sets of sampled data to generate three predicted lines in figure3. We can repeat the process multiple times, and we will get multiple predicted cumulative account balances. These predicted outcomes will form a distribution. We can calculate the average and use the average as our final prediction for the cumulative account balance. However, how accurate is this averaged cumulative account balance? We can use the variance derived from the distribution from sampled data to conduct statistical tests to check the accuracy of the averaged outcome. This technique is called Monte Carlo (MC) simulation.

In general, we only have one set of realized datasets in a real-world project. For example, we only have one set of Google's historical stock price to analyze. However, if we have some idea what the probability distribution of Google's stock price should be or use Google's historical price to form an empirical distribution, with MC simulation, we can sample repeatedly from this distribution to generate several sets of samples and apply them to the model to generate predicted outcomes. Then, we use these predicted outcomes to conduct inferential statistics. Hence, we can see MC simulation as a random sampling technique from a probability distribution to estimate an unknown population parameter.

MC simulation is based on two important statistics theories: law of large numbers (LLN) and central limit theorem (CLT).

3.2 Law of Large Numbers and Central Limit Theorem

Let's quickly review LLN and CLT first. Assume we have a sequence of identically and independently distributed n random variables $x_1, x_2, \cdots, x_n$. All the random variables have mean μ and variance σ^2. Let's define the average of the sequence as follows:

$$y_n = \frac{x_1 + x_2 + \ldots + x_n}{n}, \text{ where } n = 1, 2, \cdots$$

The law of large numbers states that y_n will converge to μ as $n \to \infty$.

Let's also define z_n as follows:

$$z_n = \frac{(x_1 - \mu) + (x_2 - \mu) + \cdots + (x_n - \mu)}{\sqrt{n}}$$

The central limit theorem states that z_n will converge to $N(0, \sigma^2)$ as $n \to \infty$.

3.3 Estimators for Unknown Stochastic Model Outcome Expected Value and Variance

According to the law of large numbers, the average of a very large sample is a good approximation of the unknown model outcome, meaning μ. Hence, if we run random samplings for m times to get m model outcomes and m is large enough, the sample average

$$\bar{\mu} = \frac{1}{m} \sum_{i=1}^{m} x_i$$

will be a good approximation of μ.

Also, we can obtain an estimate of the variance from the samples using the formula:

$$\sigma_m^2 = \frac{\sum_{i=1}^{m} (x_i - \bar{\mu})^2}{m - 1}$$

According to central limit theorem, we can write the following equation:

$$\bar{\mu} - \mu \sim \frac{\sigma}{\sqrt{m}} z, \text{ where } z \sim N(0, 1)$$

Hence, $\bar{\mu}$ converges to μ with speed of $\frac{\sigma}{\sqrt{m}}$ as m is large enough.

With the above information, we can construct a 95% confidence interval as follows:

$$\left[\bar{\mu} - \frac{1.96\sigma_m}{\sqrt{m}}, \ \bar{\mu} + \frac{1.96\sigma_m}{\sqrt{m}}\right]$$

where σ_m is the standard deviation of sampled outcomes.

From introductory statistics, we know that the true value has a 95% chance of being within the confidence interval. The width of the confidence interval is used as a measure of the precision of the estimate.

We can see that the width is controlled by two elements: the number of sampling frequencies and the variance of the sampled outcomes.

> **a.** The size of number of sampling frequencies m: as m goes up, the width of the CI shrinks.
> **b.** The variance of the sampled outcomes σ_m^2: as σ_m^2 goes up, the width of the CI expands.

Therefore, in order to increase the precision of the Monte Carlo simulation, we can either increase the number of simulations or reduce the variance of the outcome variable. In this lesson, we will focus on increasing the number of simulations to improve Monte Carlo simulation results. For those who are interested in reducing the variance of the outcome variable, you can research variance reduction techniques online.

3.4 Monte Carlo Simulation Application: Investment Account Balance Stochastic Model

Let's use our investment account balance stochastic model as an example to run Monte Carlo simulation. The stochastic model is as follows:

$$\sum_{i=0}^{11} 10\big(1+\frac{r_t}{12}\big)^{12-i}$$

Where r_t is the annual interest rate adjusted each month.

We will put in $10 dollars at the beginning of each month for a year or 12 months. The variable annual interest rate follows a normal distribution with mean of 40% and standard deviation of 0.7. We would like to forecast the account balance by the end of a year.

Here are the steps to run a Monte Carlo simulation to get samples of year-end account balance:

> 1. Sample 12 rates from variable annual interest rate distribution N(40%, 0.7).
> 2. Apply 12 rates into the model and generate one data point for year-end account balance.
> 3. Repeat step 1 and step 2 m times to get m data points for year-end account balance.
> 4. Calculate the mean and standard deviation of the m data points and construct confidence intervals.

For our investment account balance model, we will run MC simulations 100 times, 1,000 times, 100,000 times, and 1 million times to compare the results. The following plots and tables show the simulation results from different runs.

Figure 4: MC Simulation with 100 Trials, 1,000 Trials, 100,000 Trials and 1 Million Trials

In [5]:
```
# Monte Carlo Simulation


# Helper function that plots histogram and prints statistics
def Hist(trials):
    res = []

    # Simulate Monte Carlo sample
    for j in range(trials):
        int = np.random.normal(0.4, 0.7, 12)
        bal = []

        for i in range(12):
            bal.append(10 * (1 + int[i] / 12) ** (12 - i))

        res.append(sum(bal))

    # Plot Histogram
    plt.hist(res, bins=30, density=True, histtype="stepfilled", alpha=0.7)
    plt.title("Histogram of {} Simulations".format(trials))
    plt.xlabel("Cumulatice Account Balance")
    plt.ylabel("Frequency")

    # Add statistics
    avg = np.mean(res)  # Mean
    sigma = np.std(res)  # Std
    lowb = avg - 1.96 * sigma / np.sqrt(trials)  # Low Bound
    upb = avg + 1.96 * sigma / np.sqrt(trials)  # Upper Bound
    width = upb - lowb

    textstr = "{0} trials\n Average: {1:.2f}\nStandard Dev: {2:.2f}\
\n Upper Bound: {3:.2f}\n Low Bound: {4:.2f}\n Width: {5:.2f}".format(
        (trials), (avg), (sigma), (upb), (lowb), (width)
    )
    return textstr
```

In [6]:
```
# Plot four histograms
plt.figure(figsize=(16, 12))

# No. trials = 100
ax1 = plt.subplot(221)
fig1 = Hist(100)
plt.text(
    0.95,
    0.85,
    fig1,
    ha="right",
    va="center",
    fontsize=10,
    bbox={"facecolor": "orange", "alpha": 0.5},
    transform=ax1.transAxes,
)

# No. trials = 1,000
ax2 = plt.subplot(222)
fig2 = Hist(1000)
plt.text(
    0.95,
    0.85,
```

```
    fig2,
    ha="right",
    va="center",
    fontsize=10,
    bbox={"facecolor": "orange", "alpha": 0.5},
    transform=ax2.transAxes,
)

# No. trials = 100,000
ax3 = plt.subplot(223)
fig3 = Hist(100000)
plt.text(
    0.95,
    0.85,
    fig3,
    ha="right",
    va="center",
    fontsize=10,
    bbox={"facecolor": "orange", "alpha": 0.5},
    transform=ax3.transAxes,
)

# No. trials = 1,000,000
ax4 = plt.subplot(224)
fig4 = Hist(1000000)
plt.text(
    0.95,
    0.85,
    fig4,
    ha="right",
    va="center",
    fontsize=10,
    bbox={"facecolor": "orange", "alpha": 0.5},
    transform=ax4.transAxes,
)
plt.show()
```

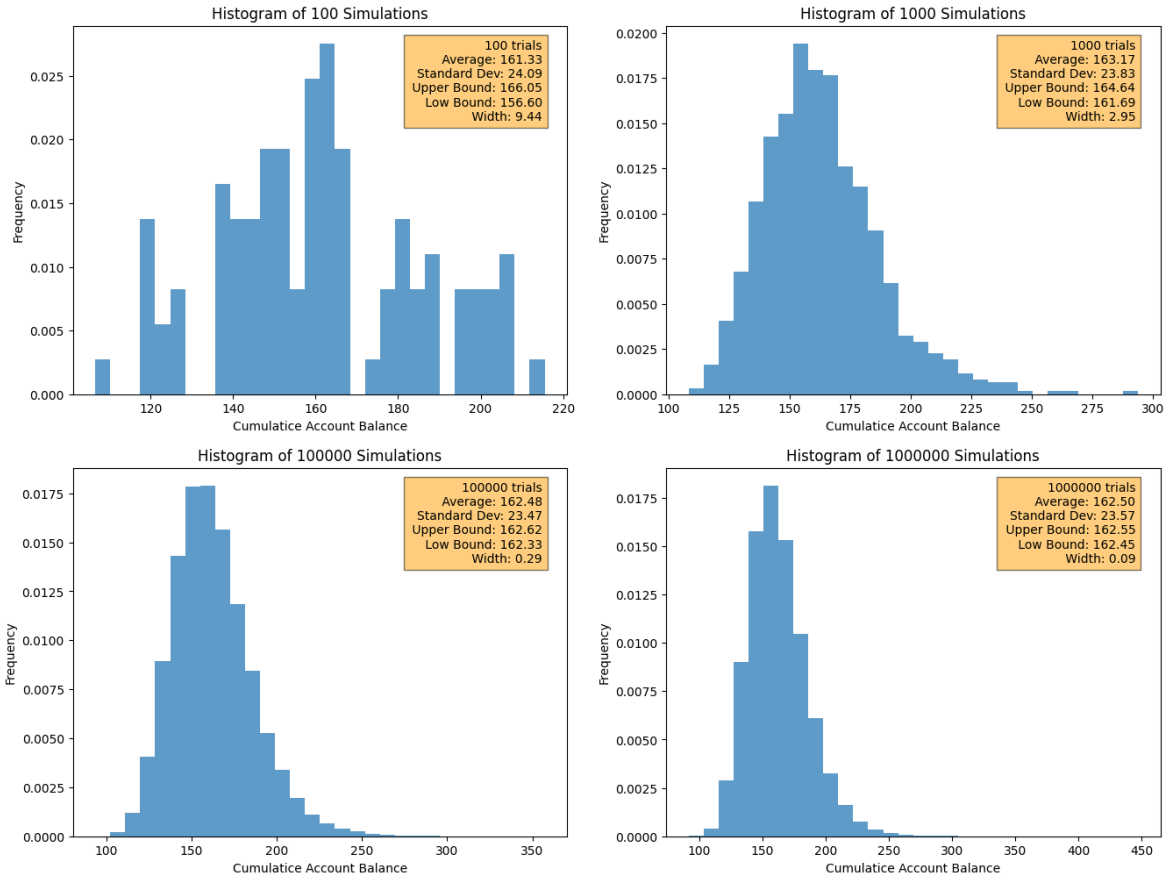


Figure 4 shows the histograms of predicted account balance distribution with a different number of trials. We can see as the number of trials increases, the shape of the histogram will become more concentrated.

In figure 4, we can see as the number of trails increases, the average predicted year-end account balance converges to 162.5. We can see when the number of trials increases to 1,000 runs from 100 runs, the average drops a lot and then stays very stable around 162.5. Also, the width of the confidence intervals drops a lot from 1,000 runs to 100 runs. From these two numbers, we can say 1,000 trials are enough to get convergent results for a year-end account balance estimator.

4. Conclusion

In this lesson, we introduced some new concepts. We first discussed deterministic models, that is, models without any random component or uncertainty within the model. When one uses a deterministic model for prediction, the prediction outcome is certain. Then, we considered stochastic models, i.e., models with a random component. Because of the randomness, if one runs simulations many times and uses simulation results, the predictions obtained will in general be different for different simulations. In order to analyze multiple prediction outputs, we introduced the concept of Monte Carlo simulation. A Monte Carlo simulation repeatedly samples data from the model to generate predicted outcomes and perform inference about the model.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

The following content is provided under a Creative Commons license. Your support will help MIT OpenCourseWare continue to offer high quality educational resources for free. To make a donation or to view additional materials from hundreds of MIT courses, visit MIT OpenCourseWare at ocw.mit.edu.

JOHN GUTTAG: Welcome to Lecture 6. As usual, I want to start by posting some relevant reading. For those who don't know, this lovely picture is of the Casino at Monte Carlo, and shortly you'll see why we're talking about casinos and gambling today. Not because I want to encourage you to gamble your life savings away.

A little history about Monte Carlo simulation, which is the topic of today's lecture. The concept was invented by the Polish American mathematician, Stanislaw Ulam. Probably more well known for his work on thermonuclear weapons than on mathematics, but he did do a lot of very important mathematics earlier in his life.

The story here starts that he was ill, recovering from some serious illness, and was home and was bored and was playing a lot of games of solitaire, a game I suspect you've all played. Being a mathematician, he naturally wondered, what's the probability of my winning this stupid game which I keep losing?

And so he actually spent quite a lot of time trying to work out the combinatorics, so that he could actually compute the probability. And despite being a really amazing mathematician, he failed. The combinatorics were just too complicated.

So he thought, well suppose I just play lots of hands and count the number I win, divide by the number of hands I played. Well then he thought about it and said, well, I've already played a lot of hands and I haven't won yet. So it probably will take me years to play enough hands to actually get a good estimate, and I don't want to do that.

So he said, well, suppose instead of playing the game, I just simulate the game on a computer. He had no idea how to use a computer, but he had friends in high places. And actually talked to John von Neumann, who is often viewed as the inventor of the stored program computer. And said, John, could you do this on your fancy new ENIAC machine?

And on the lower right here, you'll see a picture of the ENIAC. It was a very large machine. It

filled a room.

And von Neumann said, sure, we could probably do it in only a few hours of computation. Today we would think of a few microseconds, but those machines were slow. Hence was born Monte Carlo simulation, and then they actually used it in the design of the hydrogen bomb. So it turned out to be not just useful for cards.

So what is Monte Carlo simulation? It's a method of estimating the values of an unknown quantity using what is called inferential statistics. And we've been using inferential statistics for the last several lectures. The key concepts-- and I want to be careful about these things will be coming back to them-- are the population.

So think of the population as the universe of possible examples. So in the case of solitaire, it's a universe of all possible games of solitaire that you could possibly play. I have no idea how big that is, but it's really big,

Then we take that universe, that population, and we sample it by drawing a proper subset. Proper means not the whole thing. Usually more than one sample to be useful. Certainly more than 0.

And then we make an inference about the population based upon some set of statistics we do on the sample. So the population is typically a very large set of examples, and the sample is a smaller set of examples.

And the key fact that makes them work is that if we choose the sample at random, the sample will tend to exhibit the same properties as the population from which it is drawn. And that's exactly what we did with the random walk, right?

There were a very large number of different random walks you could take of say, 10,000 steps. We didn't look at all possible random walks of 10,000 steps. We drew a small sample of, say 100 such walks, computed the mean of those 100, and said, we think that's probably a good expectation of what the mean would be of all the possible walks of 10,000 steps. So we were depending upon this principle.

And of course the key fact here is that the sample has to be random. If you start drawing the sample and it's not random, then there's no reason to expect it to have the same properties as that of the population. And we'll go on throughout the term, and talk about the various ways you can get fooled and think of a random sample when exactly you don't.

All right, let's look at a very simple example. People like to use flipping coins because coins are easy. So let's assume we have some coin. All right, so I bought two coins slightly larger than the usual coin. And I can flip it. Flip it once, and let's consider one flip, and let's assume it came out heads. I have to say the coin I flipped is not actually a $20 gold piece, in case any of you were thinking of stealing it.

All right, so we've got one flip, and it came up heads. And now I can ask you the question-- if I were to flip the same coin an infinite number of times, how confident would you be about answering that all infinite flips would be heads? Or even if I were to flip it once more, how confident would you be that the next flip would be heads? And the answer is not very.

Well, suppose I flip the coin twice, and both times it came up heads. And I'll ask you the same question-- do you think that the next flip is likely to be heads? Well, maybe you would be more inclined to say yes and having only seen one flip, but you wouldn't really jump to say, sure.

On the other hand, if I flipped it 100 times and all 100 flips came up heads, well, you might be suspicious that my coin only has a head on both sides, for example. Or is weighted in some funny way that it mostly comes up heads. And so a lot of people, maybe even me, if you said, I flipped it 100 times and it came up heads. What do you think the next one will be? My best guess would be probably heads.

How about this one? So here I've simulated 100 flips, and we have 50 heads here, two heads here, And 48 tails. And now if I said, do you think that the probability of the next flip coming up heads-- is it 52 out of 100?

Well, if you had to guess, that should be the guess you make. Based upon the available evidence, that's the best guess you should probably make. You have no reason to believe it's a fair coin. It could well be weighted. We don't see it with coins, but we see weighted dice all the time. We shouldn't, but they exist. You can buy them on the internet.

So typically our best guess is what we've seen, but we really shouldn't have very much confidence in that guess. Because well, could've just been an accident. Highly unlikely even if the coin is fair that you'd get 50-50, right?

So why when we see 100 samples and they all come up heads do we feel better about guessing heads for the 101st than we did when we saw two samples? And why don't we feel

so good about guessing 52 out of 100 when we've seen a hundred flips that came out 52 and 48? And the answer is something called variance.

When I had all heads, there was no variability in my answer. I got the same answer all the time. And so there was no variability, and that intuitively-- and in fact, mathematically-- should make us feel confident that, OK, maybe that's really the way the world is.

On the other hand, when almost half are heads and almost half are tails, there's a lot of variance. Right, it's hard to predict what the next one will be. And so we should have very little confidence that it isn't an accident that it happened to be 52-48 in one direction. So as the variance grows, we need larger samples to have the same amount of confidence.

All right, let's look at that with a detailed example. We'll look at roulette in keeping with the theme of Monte Carlo simulation. This is a roulette wheel that could well be at Monte Carlo.

There's no need to simulate roulette, by the way. It's a very simple game, but as we've seen with our earlier examples, it's nice when we're learning about simulations to simulate things where we actually can know what the actual answer is so that we can then understand our simulation better.

For those of you who don't know how roulette is played-- is there anyone here who doesn't know how roulette is played? Good for you. You grew up virtuous. All right, so-- well all right. Maybe I won't go there.

So you have a wheel that spins around, and in the middle are a bunch of pockets. Each pocket has a number and a color. You bet in advance on what number you think is going to come up, or what color you think is going to come up. Then somebody drops a ball in that wheel, gives it a spin. And through centrifugal force, the ball stays on the outside for a while. But as the wheel slows down and heads towards the middle, and eventually settles in one of those pockets. And you win or you lose.

Now you can bet on it, and so let's look at an example of that. So here is a roulette game. I've called it fair roulette, because it's set up in such a way that in principle, if you bet, your expected value should be 0. You'll win some, you'll lose some, but it's fair in the sense that it's not either a negative or positive sum game.

So as always, we have an underbar underbar in it. Well we're setting up the wheel with 36 pockets on it, so you can bet on the numbers 1 through 36. That's way range work, you'll

recall. Initially, we don't know where the ball is, so we'll say it's none. And here's the key thing is, if you make a bet, this tells you what your odds are. That if you bet on a pocket and you win, you get [? len ?] of pockets minus 1.

So This is why it's a fair game, right? You bet $1. If you win, you get $36, your dollar plus $35 back. If you lose, you lose.

All right, self dot spin will be random dot choice among the pockets. And then there is simply bet, where you just can choose an amount to bet and the pocket you want to bet on. I've simplified it. I'm not allowing you to bet here on colors.

All right, so then we can play it. So here is play roulette. I've made game the class a parameter, because later we'll look at other kinds of roulette games.

You tell it how many spins. What pocket you want to bet on. For simplicity, I'm going to bet on this same pocket all the time. Pick your favorite lucky number and how much you want to bet, and then we'll have a simulation just like the ones we've already looked at.

So the number you get right starts at 0. For I and range number of spins, we'll do a spin. And then tote pocket plus equal game dot that pocket. And it will come back either 0 if you've lost, or 35 if you've won. And then we'll just print the results. So we can do it. In fact, let's run it.

So here it is. I guess I'm doing a million games here, so quite a few. Actually I'm going to do two. What happens when you spin it 100 times? What happens when you spin it a million times? And we'll see what we get.

So what we see here is that we do 100 spins. The first time I did it my expected return was minus 100%. I lost everything I bet. Not so unlikely, given that the odds are pretty long that you could do 100 times without winning. Next time I did a 100, my return was a positive 44%, and then a positive 28%.

So you can see, for 100 spins it's highly variable what the expected return is. That's one of the things that makes gambling attractive to people. If you go to a casino, 100 spins would be a pretty long night at the table. And maybe you'd won 44%, and you'd feel pretty good about it.

What about a million spins? Well people aren't interested in that, but the casino is, right? They don't really care what happens with 100 spins. They care what happens with a million spins. What happens when everybody comes every night to play.

And there what we see is-- you'll notice much less variance. Happens to be minus 0.04 plus 0.6 plus 0.79. So it's still not 0, but it's certainly, these are all closer to 0 than any of these are. We know it should be 0, but it doesn't happen to be in these examples.

But not only are they closer to 0, they're closer together. There is much less variance in the results, right? So here I show you these three numbers, and ask what do you expect to happen? You have no clue, right? So I don't know, maybe I'll win a lot. Maybe I'll lose everything. I show you these three numbers, you're going to look at it and say, well you know, I'm going to be somewhere between around 0 and maybe 1%.

But you're never going to guess it's going to be radically different from that. And if I were to change this number to be even higher, it would go even closer to 0. But we won't bother.

OK, so these are the numbers we just looked at, because I said the seed to be the same. So what's going on here is something called the law of large numbers, or sometimes Bernoulli's law. This is a picture of Bernoulli on the stamp. It's one of the two most important theorems in all of statistics, and we'll come to the second most important theorem in the next lecture.

Here it says, "in repeated independent tests with the same actual probability, the chance that the fraction of times the outcome differs from p converges to 0 as the number of trials goes to infinity." So this says if I were to spin this fair roulette wheel an infinite number of times, the expected-- the return would be 0. The real true probability from the mathematics.

Well, infinite is a lot, but a million is getting closer to infinite. And what this says is the closer I get to infinite, the closer it will be to the true probability. So that's why we did better with a million than with a hundred. And if I did a 100 million, we'd do way better than I did with a million.

I want to take a minute to talk about a way this law is often misunderstood. This is something called the gambler's fallacy. And all you have to do is say, let's go watch a sporting event. And you'll watch a batter strike out for the sixth consecutive time. The next time they come to the plate, the idiot announcer says, well he struck out six times in a row. He's due for a hit this time, because he's usually a pretty good hitter.

Well that's nonsense. It says, people somehow believe that if deviations from expected occur, they'll be evened out in the future. And we'll see something similar to this that is true, but this is

not true.

And there is a great story about it. This is told in a book by [INAUDIBLE] and [INAUDIBLE]. And this truly happened in Monte Carlo, with Roulette. And you could either bet on black or red. Black came up 26 times in a row. Highly unlikely, right? 2 to the 26th is a giant number.

And what happened is, word got out on the casino floor that black had kept coming up way too often. And people more or less panicked to rush to the table to bet on red, saying, well it can't keep coming up black. Surely the next one will be red.

And as it happened when the casino totaled up its winnings, it was a record night for the casino. Millions of francs got bet, because people were sure it would have to even out. Well if we think about it, probability of 26 consecutive reds is that. A pretty small number. But the probability of 26 consecutive reds when the previous 25 rolls were red is what? No, that.

AUDIENCE: Oh, I thought you meant [INAUDIBLE].

JOHN GUTTAG: No, if you had 25 reds and then you spun the wheel once more, the probability of it having 26 reds is now 0.5, because these are independent events. Unless of course the wheel is rigged, and we're assuming it's not.

People have a hard time accepting this, and I know it seems funny. But I guarantee there will be some point in the next month or so when you will find yourself thinking this way, that something has to even out. I did so badly on the midterm, I will have to do better on the final. That was mean, I'm sorry.

All right, speaking of means-- see? Professor [? Grimm's ?] not the only one who can make bad jokes. There is something-- it's not the gambler's fallacy-- that's often confused with it, and that's called regression to the mean. This term was coined in 1885 by Francis Galton in a paper, of which I've shown you a page from it here.

And the basic conclusion here was-- what this table says is if somebody's parents are both taller than average, it's likely that the child will be smaller than the parents. Conversely, if the parents are shorter than average, it's likely that the child will be taller than average.

Now you can think about this in terms of genetics and stuff. That's not what he did. He just looked at a bunch of data, and the data actually supported this. And this led him to this notion of regression to the mean. And here's what it is, and here's the way in which it is subtly

different from the gambler's fallacy.

What he said here is, following an extreme event-- parents being unusually tall-- the next random event is likely to be less extreme. He didn't know much about genetics, and he kind of assumed the height of people were random. But we'll ignore that. OK, but the idea is here that it will be less extreme.

So let's look at it in roulette. If I spin a fair roulette wheel 10 times and get 10 reds, that's an extreme event. Right, here's a probability of basically 1.1024. Now the gambler's fallacy says, if I were to spin it another 10 times, it would need to even out. As in I should get more blacks than you would usually get to make up for these excess reds.

What regression to the mean says is different. It says, it's likely that in the next 10 spins, you will get fewer than 10 reds. You will get a less extreme event. Now it doesn't have to be 10. If I'd gotten 7 reds instead of 5, you'd consider that extreme, and you would bet that the next 10 would have fewer than 7. But you wouldn't bet that it would have fewer than 5.

Because of this, if you now look at the average of the 20 spins, it will be closer to the mean of 50% reds than you got from the extreme first spins. So that's why it's called regression to the mean. The more samples you take, the more likely you'll get to the mean. Yes?

AUDIENCE: So, roulette wheel spins are supposed to be independent.

JOHN GUTTAG: Yes.

AUDIENCE: So it seems like the second 10--

JOHN GUTTAG: Pardon?

AUDIENCE: It seems like the second 10 times that you spin it. that shouldn't have to [INAUDIBLE].

JOHN GUTTAG: Has nothing to do with the first one.

AUDIENCE: But you said it's likely [INAUDIBLE].

JOHN GUTTAG: Right, because you have an extreme event, which was unlikely. And now if you have another event, it's likely to be closer to the average than the extreme was to the average. Precisely because it is independent. That makes sense to everybody? Yeah?

AUDIENCE: Isn't that the same as the gambler's fallacy, then? By saying that, because this was super

unlikely, the next one [INAUDIBLE].

JOHN GUTTAG: No, the gambler's fallacy here-- and it's a good question, and indeed people often do get these things confused. The gambler's fallacy would say that the second 10 spins would-- we would expect to have fewer than 5 reds, because you're trying to even out the unusual number of reds in the first Spin

Whereas here we're not saying we would have fewer than 5. We're saying we'd probably have fewer than 10. That it'll be closer to the mean, not that it would be below the mean. Whereas the gambler's fallacy would say it should be below that mean to quote, even out, the first 10. Does that makes sense? OK, great questions. Thank you.

All right, now you may not know this, but casinos are not in the business of being fair. And the way they don't do that is in Europe, they're not all red and black. They sneak in one green. And so now if you bet red, well sometimes it isn't always red or black. And furthermore, there is this 0. They index from 0 rather than from one, and so you don't get a full payoff.

In American roulette, they manage to sneak in two greens. They have a 0 in a double 0. Tilting the odds even more in favor of the casino. So we can do that in our simulation. We'll look at European roulette as a subclass of fair roulette. I've just added this extra pocket, 0.

And notice I have not changed the odds. So what you get if you get your number is no higher, but you're a little bit less likely to get it because we snuck in that 0. Than American roulette is a subclass of European roulette in which I add yet another pocket.

All right, we can simulate those. Again, nice thing about simulations, we can play these games. So I've simulated 20 trials of 1,000 spins, 10,000 spins, 100,000, and a million. And what do we see as we look at this? Well, right away we can see that fair roulette is usually a much better bet than either of the other two. That even with only 1,000 spins the return is negative.

And as we get more and more as I got to a million, it starts to look much more like closer to 0. And these, we have reason to believe at least, are much closer to true expectation saying that, while you break even in fair roulette, you'll lose 2.7% in Europe and over 5% in Las Vegas, or soon in Massachusetts.

All right, we're sampling, right? That's why the results will change, and if I ran a different simulation with a different seed I'd get different numbers. Whenever you're sampling, you can't

be guaranteed to get perfect accuracy. It's always possible you get a weird sample.

That's not to say that you won't get exactly the right answer. I might have spun the wheel twice and happened to get the exact right answer of the return. Actually not twice, because the math doesn't work out, but 35 times and gotten exactly the right answer.

But that's not the point. We need to be able to differentiate between what happens to be true and what we actually know, in a rigorous sense, is true. Or maybe don't know it, but have real good reason to believe it's true. So it's not just a question of faith.

And that gets us to what's in some sense the fundamental question of all computational statistics, is how many samples do we need to look at before we can have real, justifiable confidence in our answer? As we've just seen-- not just, a few minutes ago-- with the coins, our intuition tells us that it depends upon the variability in the underlying possibilities.

So let's look at that more carefully. We have to look at the variation in the data. So let's look at first something called variance.

So this is variance of x. Think of x as just a list of data examples, data items. And the variance is we first compute the average of value, that's mu. So mu is for the mean. For each little x and big X, we compare the difference of that and the mean. How far is it from the mean? And square of the difference, and then we just sum them.

So this takes, how far is everything from the mean? We just add them all up. And then we end up dividing by the size of the set, the number of examples.

Why do we have to do this division? Well, because we don't want to say something has high variance just because it has many members, right? So this sort of normalizes is by the number of members, and this just sums how different the members are from the mean.

So if everything is the same value, what's the variance going to be? If I have a set of 1,000 6's, what's the variance? Yes?

AUDIENCE: 0.

JOHN GUTTAG: 0. You think this is going to be hard, but I came prepared. I was hoping this would happen. Look out, I don't know where this is going to go. [FIRES SLINGSHOT]

AUDIENCE: [LAUGHTER]

JOHN GUTTAG: All right, maybe it isn't the best technology. I'll go home and practice. And then the thing you're more familiar with is the standard deviation. And if you look at the standard deviation is, it's simply the square root of the variance.

Now, let's understand this a little bit and first ask, why am I squaring this here, especially because later on I'm just going to take a square root anyway? Well squaring it has one virtue, which is that it means I don't care whether the difference is positive or negative. And I shouldn't, right? I don't care which side of the mean it's on, I just care it's not near the mean.

But if that's all I wanted to do I could take the absolute value. The other thing we see with squaring is it gives the outliers extra emphasis, because I'm squaring that distance. Now you can think that's good or bad, but it's worth knowing it's a fact.

The more important thing to think about is standard deviation all by itself is a meaningless number. You always have to think about it in the context of the mean. If I tell you the standard deviation is 100, you then say, well-- and I ask you whether it's big or small, you have no idea.

If the mean is 100 and the standard deviation is 100, it's pretty big. If the mean is a billion and the standard deviation is 100, it's pretty small. So you should never want to look at just the standard deviation.

All right, here is just some code to compute those, easy enough. Why am I doing this? Because we're now getting to the punch line. We often try and estimate values just by giving the mean. So we might report on an exam that the mean grade was 80.

It's better instead of trying to describe an unknown value by it-- an unknown parameter by a single value, say the expected return on betting a roulette wheel, to provide a confidence interval. So what a confidence interval is is a range that's likely to contain the unknown value, and a confidence that the unknown value is within that range.

So I might say on a fair roulette wheel I expect that your return will be between minus 1% and plus 1%, and I expect that to be true 95% of the time you play the game if you play 100 rolls, spins. If you take 100 spins of the roulette wheel, I expect that 95% of the time your return will be between this and that.

So here, we're saying the return on betting a pocket 10 times, 10,000 times in European roulette is minus 3.3%. I think that was the number we just saw. And now I'm going to add to

that this margin of error, which is plus or minus 3.5% with a 95% level of confidence.

What does this mean? If I were to conduct an infinite number of trials of 10,000 bets each, my expected average return would indeed be minus 3.3%, and it would be between these values 95% of the time. I've just subtracted and added this 3.5, saying nothing about what would happen in the other 5% of the time. How far away I might be from this, this is totally silent on that subject. Yes?

AUDIENCE: I think you want 0.2 not 9.2.

JOHN GUTTAG: Oh, let's see. Yep, I do. Thank you. We'll fix it on the spot. This is why you have to come to lecture rather than just reading the slides, because I make mistakes. Thank you, Eric.

All right, so it's telling me that, and that's all it means. And it's amazing how often people don't quite know what this means. For example, when they look at a political pole and they see how many votes somebody is expected to get. And they see this confidence interval and say, what does that really mean? Most people don't know. But it does have a very precise meaning, and this is it.

How do we compute confidence intervals? Most of the time we compute them using something called the empirical rule. Under some assumptions, which I'll get to a little bit later, the empirical rule says that if I take the data, find the mean, compute the standard deviation as we've just seen, 68% of the data will be within one standard deviation in front of or behind the mean. Within one standard deviation of the mean.

95% will be within 1.96 standard deviations. And that's what people usually use. Usually when people talk about confidence intervals, they're talking about the 95% confidence interval. And they use this 1.6 number.

And 99.7% of the data will be within three standard deviations. So you can see if you are outside the third standard deviation, you are a pretty rare bird, for better or worse depending upon which side.

All right, so let's apply the empirical rule to our roulette game. So I've got my three roulette games as before. I'm going to run a simple simulation. And the key thing to notice is really this print statement here.

Right, that I'll print the mean, which I'm rounding. And then I'm going to give the confidence

intervals, plus or minus, and I'll just take the standard deviation times 1.6 times 100, y times 100, because I'm showing you percentages.

All right so again, very straightforward code. Just simulation, just like the ones we've been looking at. And well, I'm just going-- I don't think I'll bother running it for you in the interest of time. You can run it yourself. But here's what I got when I ran it.

So when I simulated betting a pocket for 20 trials, we see that the-- of 1,000 spins each, for 1,000 spins the expected return for fair roulette happened to be 3.68%. A bit high. But you'll notice the confidence interval plus or minus 27 includes the actual answer, which is 0. And we have very large confidence intervals for the other two games.

If you go way down to the bottom where I've spun, spun the wheel many more times, what we'll see is that my expected return for fair roulette is much closer to 0 than it was here. But more importantly, my confidence interval is much smaller, 0.8. So now I really have constrained it pretty well.

Similarly, for the other two games you will see-- maybe it's more accurate, maybe it's less accurate, but importantly the confidence interval is smaller. So I have good reason to believe that the mean I'm computing is close to the true mean, because my confidence interval has shrunk. So that's the really important concept here, is that we don't just guess-- compute the value in the simulation. We use, in this case, the empirical rule to tell us how much faith we should have in that value.

All right, the empirical rule doesn't always work. There are a couple of assumptions. One is that the mean estimation error is 0. What is that saying? That I'm just as likely to guess high as gas low.

In most experiments of this sort, most simulations, that's a very fair assumption. There's no reason to guess I'd be systematically off in one direction or another. It's different when you use this in a laboratory experiment, where in fact, depending upon your laboratory technique, there may be a bias in your results in one direction.

So we have to assume that there's no bias in our errors. And we have to assume that the distribution of errors is normal. And we'll come back to this in just a second. But this is a normal distribution, called the Gaussian. Under those two assumptions the empirical rule will always hold.

All right, let's talk about distributions, since I just introduced one. We've been using a probability distribution. And this captures the notion of the relative frequency with which some random variable takes on different values.

There are two kinds. , Discrete and these when the values are drawn from a finite set of values. So when I flip these coins, there are only two possible values, head or tails. And so if we look at the distribution of heads and tails, it's pretty simple. We just list the probability of heads. We list the probability of tails. We know that those two probabilities must add up to 1, and that fully describes our distribution.

Continuous random variables are a bit trickier. They're drawn from a set of reals between two numbers. For the sake of argument, let's say those two numbers are 0 and 1. Well, we can't just enumerate the probability for each number.

How many real numbers are there between 0 and 1? An infinite number, right? And so I can't say, for each of these infinite numbers, what's the probability of it occurring? Actually the probability is close to 0 for each of them. Is 0, if they're truly infinite.

So I need to do something else, and what I do that is what's called the probability density function. This is a different kind of PDF than the one Adobe sells. So there, we don't give the probability of the random variable taking on a specific value. We give the probability of it lying somewhere between two values. And then we define a curve, which shows how it works.

So let's look at an example. So we'll go back to normal distributions. This is-- for the continuous normal distribution, it's described by this function. And for those of you who don't know about the magic number e, this is one of many ways to define it.

But I really don't care whether you remember this. I don't care whether you know what e is. I don't care if you know what this is. What we really want to say is, it looks like this. In this case, the mean is 0. It doesn't have to be 0. I've [INAUDIBLE] a mean of 0 and a standard deviation of 1. This is called the so-called standard normal distribution.

But it's symmetric around the mean. And that gets back to, it's equally likely that our errors are in either direction, right? So it peaks at the mean. The peak is always at the mean. That's the most probable value, and it's symmetric about the mean.

So if we look at it, for example, and I say, what's the probability of the number being between

0 and 1? I can look at it here and say, all right, let's draw a line here, and a line here. And then I can integrate the curve under here. And that tells me the probability of this random variable being between 0 and 1.

If I want to know between minus 1 and 1. I just do this and then I integrate over that area. All right, so the area under the curve in this case defines the likelihood. Now I have to divide and normalize to actually get the answer between 0 and 1.

So the question is, what fraction of the area under the curve is between minus 1 and 1? And that will tell me the probability. So what does the empirical rule tell us? What fraction is between minus 1 and 1, roughly? Yeah? 68%, right?

So that tells me 68% of the area under this curve is between minus 1 and 1, because my standard deviation is 1, roughly 68%. And maybe your eyes will convince you that's a reasonable guess.

OK, we'll come back and look at this in a bit more detail on Monday of next week. And also look at the question of, why does this work in so many cases where we don't actually have a normal distribution to start with?

Section 15

Multiple linear regression.

Let us consider a model

$$Y_i = \beta_1 X_{i1} + \ldots + \beta_p X_{ip} + \varepsilon_i$$

where random noise variables $\varepsilon_1, \ldots, \varepsilon_n$ are i.i.d. $N(0, \sigma^2)$. We can write this in a matrix form

$$Y = X\beta + \varepsilon,$$

where Y and ε are $n \times 1$ vectors, β is $p \times 1$ vector and X is $n \times p$ matrix. We will denote the columns of matrix X by $X_1, \ldots X_p$, i.e.

$$X = (X_1, \ldots, X_p)$$

and we will assume that these columns are linearly independent. If they are not linearly independent, we can not reconstruct parameters β from X and Y even if there is no noise ε. In simple linear regression this would correspond to all Xs being equal and we can not estimate a line from observations only at one point. So from now on we will assume that $n > p$ and the rank of matrix X is equal to p. To estimate unknown parameters β and σ we will use maximum likelihood estimators.

Lemma 1. *The MLE of β and σ^2 are given by:*

$$\hat{\beta} = (X^T X)^{-1} X^T Y \quad \textit{and} \quad \hat{\sigma}^2 = \frac{1}{n}|Y - X\hat{\beta}|^2 = \frac{1}{n}|Y - X(X^T X)^{-1} X^T Y|^2.$$

Proof. The p.d.f. of Y_i is

$$f_i(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(-\frac{1}{2\sigma^2}(x - \beta_1 X_{i1} - \ldots - \beta_p X_{ip})^2\Big)$$

and, therefore, the likelihood function is

$$\begin{aligned} \prod_{i=1}^n f_i(Y_i) &= \Big(\frac{1}{\sqrt{2\pi}\sigma}\Big)^n \exp\Big(-\frac{1}{2\sigma^2}\sum_{i=1}^n (Y_i - \beta_1 X_{i1} - \ldots - \beta_p X_{ip})^2\Big) \\ &= \Big(\frac{1}{\sqrt{2\pi}\sigma}\Big)^n \exp\Big(-\frac{1}{2\sigma^2}|Y - X\beta|^2\Big). \end{aligned}$$

To maximize the likelihood function, first, we need to minimize $|Y - X\beta|^2$. If we rewrite the norm squared using scalar product:

$$\begin{aligned} |Y - X\beta|^2 &= (Y - \sum_{i=1}^{p} \beta_i X_i, Y - \sum_{i=1}^{p} \beta_i X_i) \\ &= (Y,Y) - 2\sum_{i=1}^{p} \beta_i (Y, X_i) + \sum_{i,j=1}^{p} \beta_i \beta_j (X_i, X_j). \end{aligned}$$

Then setting the derivatives in each β_i equal to zero

$$-2(Y, X_i) + 2\sum_{j=1}^{p} \beta_j (X_i, X_j) = 0$$

we get

$$(Y, X_i) = \sum_{i=1}^{p} \beta_j (X_i, X_j) \quad \text{for all} \quad i \le p.$$

In matrix notations this can be written as $X^T Y = X^T X\beta$. Matrix $X^T X$ is a $p \times p$ matrix. Is is invertible since by assumption X has rank p. So we can solve for β to get the MLE

$$\hat{\beta} = (X^T X)^{-1} X^T Y.$$

It is now easy to minimize over σ to get

$$\hat{\sigma}^2 = \frac{1}{n}|Y - X\hat{\beta}|^2 = \frac{1}{n}|Y - X(X^T X)^{-1} X^T Y|^2.$$

□

To do statistical inference we need to compute the joint distribution of these estimates. We will prove the following.

Theorem. *We have*

$$\hat{\beta} \sim N\Big(\beta, \sigma^2 (X^T X)^{-1}\Big), \;\; \frac{n\hat{\sigma}^2}{\sigma^2} \sim \chi^2_{n-p}$$

and estimates $\hat{\beta}$ and $\hat{\sigma}^2$ are independent.

Proof. First of all, let us rewrite the estimates in terms of random noise ε using $Y = X\beta + \varepsilon$. We have

$$\begin{aligned} \hat{\beta} &= (X^T X)^{-1} X^T Y = (X^T X)^{-1} X^T (X\beta + \varepsilon) \\ &= (X^T X)^{-1}(X^T X)\beta + (X^T X)^{-1} X^T \varepsilon = \beta + (X^T X)^{-1} X^T \varepsilon \end{aligned}$$

and since

$$\begin{aligned} Y - X(X^T X)^{-1} X^T Y &= X\beta + \varepsilon - X(X^T X)^{-1} X^T (X\beta + \varepsilon) \\ &= X\beta + \varepsilon - X\beta - X(X^T X)^{-1} X^T \varepsilon = (I - X(X^T X)^{-1} X^T)\varepsilon \end{aligned}$$

we have

$$\hat{\sigma}^2 = \frac{1}{n}|(I - X(X^TX)^{-1}X^T)\varepsilon|^2.$$

Since $\hat{\beta}$ is a linear transformation of a normal vector ε it will also be normal with mean

$$\mathbb{E}\hat{\beta} = \mathbb{E}(\beta + (X^TX)^{-1}X^T\varepsilon) = \beta$$

and covariance matrix

$$\begin{aligned}
\mathbb{E}(\hat{\beta}-\beta)(\hat{\beta}-\beta)^T &= \mathbb{E}(X^TX)^{-1}X^T\varepsilon\varepsilon^TX(X^TX)^{-1} \\
&= (X^TX)^{-1}X^T\mathbb{E}\varepsilon\varepsilon^TX(X^TX)^{-1} \\
&= (X^TX)^{-1}X^T(\sigma^2 I)X(X^TX)^{-1} \\
&= \sigma^2(X^TX)^{-1}(X^TX)(X^TX)^{-1} = \sigma^2(X^TX)^{-1}.
\end{aligned}$$

This proves that $\hat{\beta} \sim N(\beta, \sigma^2(X^TX)^{-1})$. To prove that $\hat{\beta}$ and $\hat{\sigma}^2$ are independent and to find the distribution of $n\hat{\sigma}^2/\sigma^2$ we will use the following trick. This trick can also be very useful computationally since it will relate all quantities of interest expressed in terms of $n \times p$ matrix X to quantities expressed in terms of a certain $p \times p$ matrix R which can be helpful when n is very large compared to p. We would like to manipulate the columns of matrix X to make them orthogonal to each other, which can be done by Gram-Schmidt orthogonalization. In other words, we want to represent matrix X as

$$X = X_0R$$

where X_0 is $n \times p$ matrix with columns $X_0^1, \ldots, X_0^p$ that are orthogonal to each other and, moreover, form an orthonormal basis, and matrix R is $p \times p$ invertible (and upper triangular) matrix. In Matlab this can be done using economy size QR factorization

```
[X0, R]=qr(X,0).
```

The fact that columns of X_0 are orthonormal implies that

$$X_0^TX_0 = I$$

- a $p \times p$ identity matrix. Let us replace X by X_0R everywhere in the estimates. We have

$$(X^TX)^{-1}X^T = (R^TX_0^TX_0R)^{-1}R^TX_0^T = (R^TR)^{-1}R^TX_0^T = R^{-1}(R^T)^{-1}R^TX_0^T = R^{-1}X_0^T,$$

$$X(X^TX)^{-1}X^T = X_0R(R^TX_0^2X_0R)^{-1}R^TX_0^T = X_0RR^{-1}(R^T)^{-1}R^TX_0^T = X_0X_0^T.$$

As a result

$$\hat{\beta} - \beta = R^{-1}X_0^T\varepsilon \quad \text{and} \quad n\hat{\sigma}^2 = |(I - X_0X_0^T)\varepsilon|^2. \tag{15.0.1}$$

By construction p columns of X_0, which are also the rows of X_0^T, are orthonormal. Therefore, we can choose the last $n - p$ rows of a $n \times n$ matrix

$$A = \begin{pmatrix} X_0^T \\ \cdots \end{pmatrix}$$

to make A an orthogonal matrix, we just need to choose them to complete, together with rows of X_0^T, the orthonormal basis in $\mathbb{R}^n$. Let us define a vector

$$g = A\varepsilon, \text{ i.e. } \begin{pmatrix} g_1 \\ g_2 \\ \vdots \\ g_n \end{pmatrix} = \begin{pmatrix} X_0^T \\ \cdots \end{pmatrix} \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}.$$

Since ε is a vector of i.i.d. standard normal, we proved before that its orthogonal transformation g will also be a vector of independent $N(0,\sigma^2)$ random variables $g_1,\dots,g_n$. First of all, since

$$\hat{g} := \begin{pmatrix} g_1 \\ \vdots \\ g_p \end{pmatrix} = X_0^T \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix},$$

we have

$$\hat{\beta} - \beta = R^{-1} X_0^T \varepsilon = R^{-1} \begin{pmatrix} g_1 \\ \vdots \\ g_p \end{pmatrix} = R^{-1}\hat{g}. \tag{15.0.2}$$

Next, we will prove that

$$|(I - X_0 X_0^T)\varepsilon|^2 = g_{p+1}^2 + \ldots + g_n^2. \tag{15.0.3}$$

First of all, orthogonal transformation preserves lengths, so $|g|^2 = |A\varepsilon|^2 = |\varepsilon|^2$. On the other hand, let us write $|\varepsilon|^2 = \varepsilon^T\varepsilon$ and break ε into a sum of two terms

$$\varepsilon = X_0 X_0^T \varepsilon + (I - X_0 X_0^T)\varepsilon.$$

Then we get

$$|g|^2 = |\varepsilon|^2 = \varepsilon^T\varepsilon = \Big(\varepsilon^T X_0 X_0^T + \varepsilon^T(I - X_0 X_0^T)\Big)\Big(X_0 X_0^T\varepsilon + (I - X_0 X_0^T)\varepsilon\Big).$$

When we multiply all the terms out we will use that $X_0^T X_0 = I$ since the matrix $X_0^T X_0$ consists of scalar products of columns of X_0 which are orthonormal. This also implies that

$$X_0 X_0^T (I - X_0 X_0^T) = X_0 X_0^T - X_0 I X_0^T = 0.$$

Using this we get

$$\begin{aligned} |g|^2 = |\varepsilon|^2 &= \varepsilon^T X_0 X_0^T \varepsilon + \varepsilon^T (I - X_0 X_0^T)(I - X_0 X_0^T)\varepsilon \\ &= |X_0^T\varepsilon|^2 + |(I - X_0 X_0^T)\varepsilon|^2 = |\hat{g}|^2 + |(I - X_0 X_0^T)\varepsilon|^2 \end{aligned}$$

because $\hat{g} = X_0^T\varepsilon$ so we finally proved that

$$|(I - X_0 X_0^T)\varepsilon|^2 = |g|^2 - |\hat{g}|^2 = g_1^2 + \ldots + g_n^2 - g_1^2 - \ldots - g_p^2 = g_{p+1}^2 + \ldots + g_n^2$$

which is (15.0.3). This proves that $n\hat{\sigma}^2/\sigma^2 \sim \chi^2_{n-p}$ and it is also independent of $\hat{\beta}$ which depends only on $g_1,\dots,g_p$ by (15.0.2).

□

Let us for convenience write down equation (15.0.2) as a separate result.

Lemma 2. *Given a decomposition $X = X_0 R$ with $n \times p$ matrix X_0 with orthonormal columns and invertible (upper triangular) $p \times p$ matrix R we can represent*

$$\hat{\beta} - \beta = R^{-1}\hat{g} = R^{-1} \begin{pmatrix} g_1 \\ \vdots \\ g_p \end{pmatrix}$$

for independent $N(0, \sigma^2)$ random variables $g_1, \ldots, g_p$.

Confidence intervals and t-tests for linear combination of parameters β. Let us consider a linear combination

$$c_1\beta_1 + \ldots + c_p\beta_p = c^T\beta$$

where $c = (c_1, \ldots, c_p)^T$. To construct confidence intervals and t-tests for this linear combination we need to write down a distribution of $c^T\hat{\beta}$. Clearly, it has a normal distribution with mean $\mathbb{E}c^T\hat{\beta} = c^T\beta$ and variance

$$\mathbb{E}(c^T(\hat{\beta} - \beta))^2 = \mathbb{E}c^T(\hat{\beta} - \beta)(\hat{\beta} - \beta)^T c = c^T \mathrm{Cov}(\hat{\beta})c = \sigma^2 c^T (X^T X)^{-1} c.$$

Therefore,

$$\frac{c^T(\hat{\beta} - \beta)}{\sqrt{\sigma^2 c^T (X^T X)^{-1} c}} \sim N(0, 1)$$

and using that $n\hat{\sigma}^2/\sigma^2 \sim \chi^2_{n-p}$ we get

$$\frac{c^T(\hat{\beta} - \beta)}{\sqrt{\sigma^2 c^T (X^T X)^{-1} c}} \Big/ \sqrt{\frac{1}{n-p}\frac{n\hat{\sigma}^2}{\sigma^2}} = c^T(\hat{\beta} - \beta)\sqrt{\frac{n-p}{n\hat{\sigma}^2 c^T (X^T X)^{-1} c}} \sim t_{n-p}.$$

To obtain the distribution of one parameter $\hat{\beta}_i$ we need to choose a vector c that has all zeros and 1 in the ith coordinate. Then we get

$$(\hat{\beta}_i - \beta_i)\sqrt{\frac{n-p}{n\hat{\sigma}^2((X^T X)^{-1})_{ii}}} \sim t_{n-p}.$$

Here $((X^T X)^{-1})_{ii}$ is the ith diagonal element of the matrix $(X^T X)^{-1}$. This is a good time to mention how the quality of estimation of β depends on the choice of X. For example, we mentioned before that the columns of X should be linearly independent. What happens if some of them are nearly collinear? Then some eigenvalues of $(X^T X)$ will be 'small' (in some sense) and some eigenvalues of $(X^T X)^{-1}$ will be 'large'. (Small and large here are relative terms because the size of the matrix also grows with n.) As a result, the confidence intervals for some parameters will get very large too which means that their estimates are not very accurate. To improve the quality of estimation we need to avoid using collinear predictors. We will see this in the example below.

□

Joint confidence set for β and F-test. By Lemma 2, $R(\hat{\beta}-\beta)=\hat{g}$ and, therefore,

$$g_1^2+\ldots+g_p^2=|\hat{g}|^2=\hat{g}^T\hat{g}=(\hat{\beta}-\beta)^TR^TR(\hat{\beta}-\beta)=(\hat{\beta}-\beta)^TX^TX(\hat{\beta}-\beta).$$

Since $g_i\sim N(0,\sigma^2)$ this proves that

$$\frac{(\hat{\beta}-\beta)^TX^TX(\hat{\beta}-\beta)}{\sigma^2}\sim\chi^2_p.$$

Using that $n\hat{\sigma}^2/\sigma^2\sim\chi^2_{n-p}$ gives

$$\frac{(\hat{\beta}-\beta)^TX^TX(\hat{\beta}-\beta)}{p\sigma^2}\Big/\frac{n\hat{\sigma}^2}{(n-p)\sigma^2}=\frac{(n-p)}{np\hat{\sigma}^2}(\hat{\beta}-\beta)^TX^TX(\hat{\beta}-\beta)\sim F_{p,n-p}.$$

If we take c such that $F_{p,n-p}(0,c_\alpha)=\alpha$ then

$$\frac{(n-p)}{np\hat{\sigma}^2}(\hat{\beta}-\beta)^TX^TX(\hat{\beta}-\beta)\le c_\alpha \tag{15.0.4}$$

defines a joint confidence set for all parameters β simultaneously with confidence level α.

Suppose that we want to test a hypothesis about all parameters simultaneously, for example,

$$H_0:\beta=\beta_0.$$

Then we consider a statistic

$$F=\frac{(n-p)}{np\hat{\sigma}^2}(\hat{\beta}-\beta_0)^TX^TX(\hat{\beta}-\beta_0), \tag{15.0.5}$$

which under null hypothesis has $F_{p,n-p}$ distribution, and define a decision rule by

$$\delta=\begin{cases} H_0: & F\le c \\ H_1: & F>c, \end{cases}$$

where a threshold c is determined by $F_{p,n-p}(c,\infty)=\alpha$ - a level of significance. Of course, this test is equivalent to checking if vector β_0 belongs to a confidence set (15.0.4)! (We just need to remember that confidence level = 1 - level of significance.)

□

Simultaneous confidence set and F-test for subsets of β. Let

$$s=\{i_1,\ldots,i_k\}\subseteq\{1,\ldots,p\}$$

be s subset of size $k\le p$ of indices $\{1,\ldots,p\}$ and let $\beta_s=(\beta_{i_1},\ldots,\beta_{i_k})^T$ be a vector that consists of the corresponding subset of parameters β. Suppose that we would like to test the hypothesis

$$H_0:\beta_s=\beta_s^0$$

for some given vector β_s^0, for example, $\beta_s^0=0$. Let $\hat{\beta}_s$ be a corresponding vector of estimates. Let

$$\Sigma_s=\Big((X^TX)^{-1}_{i,j}\Big)_{i,j\in s}$$

be a $k \times k$ submatrix of $(X^T X)^{-1}$ with row and column indices in the set s. By the above Theorem, the joint distribution of $\hat{\beta}_s$ is

$$\hat{\beta}_s \sim N(\beta_s, \sigma^2 \Sigma_s).$$

Let $A = \Sigma_s^{1/2}$, i.e. A is a symmetric $k \times k$ matrix such that $\Sigma_s = AA^T$. As a result, a centered vector of estimates can be represented as

$$\hat{\beta}_s - \beta_s = Ag,$$

where $g = (g_1, \ldots, g_k)^T$ are independent $N(0, \sigma^2)$. Therefore, $g = A^{-1}(\hat{\beta}_s - \beta_s)$ and the rest is similar to the above argument. Namely,

$$\begin{aligned} g_1^2 + \ldots + g_k^2 &= |g|^2 = g^T g = (\hat{\beta}_s - \beta_s)^T (A^{-1})^T A^{-1} (\hat{\beta}_s - \beta_s) \\ &= (\hat{\beta}_s - \beta_s)^T (AA^T)^{-1} (\hat{\beta}_s - \beta_s) = (\hat{\beta}_s - \beta_s)^T \Sigma_s^{-1} (\hat{\beta}_s - \beta_s) \sim \sigma^2 \chi_k^2. \end{aligned}$$

As before we get

$$F = \frac{(n-p)}{nk\hat{\sigma}^2} (\hat{\beta}_s - \beta_s)^T \Sigma_s^{-1} (\hat{\beta}_s - \beta_s) \sim F_{k,n-p}$$

and we can now construct a simultaneous confidence set and F-tests.

□

Remark. Matlab regression function 'regress' assumes that a matrix X of explanatory variables will contain a first column of ones that corresponds to an "intercept" parameter β_1. The F-statistic output by 'regress' corresponds to F-test about all other "slope" parameters:

$$H_0 : \beta_2 = \ldots = \beta_p = 0.$$

In this case $s = \{2, 3, \ldots, p\}$, $k = p - 1$ and

$$F = \frac{(n-p)}{n(p-1)\hat{\sigma}^2} \hat{\beta}_s^T \Sigma_s^{-1} \hat{\beta}_s \sim F_{p-1,n-p}.$$

□

Example. Let us take a look at the 'cigarette' dataset from previous lecture. We saw that tar, nicotine and carbon monoxide content are positively correlated and any pair is well described by a simple linear regression. Suppose that we would like to predict carbon monoxide as a linear function of both tar and nicotine content. We create a 25×3 matrix X:

```
X=[ones(25,1),tar,nic];
```

We introduce a first column of ones to allow an intercept parameter β_1 in our multiple linear regression model:

$$\mathrm{CO}_i = \beta_1 + \beta_2 \mathrm{Tar}_i + \beta_3 \mathrm{Nicotin}_i + \varepsilon_i.$$

If we perform a multiple linear regression:

```
[b,bint,r,rint,stats] = regress(carb,X);
```

We get the estimates of parameters and 95% confidence intervals for each parameter

```
b = 3.0896      bint = 1.3397   4.8395
    0.9625               0.4717   1.4533
   -2.6463             -10.5004   5.2079
```

and, in order, R^2-statistic, F-statistic from (15.0.5), p-value for this statistic

$$F_{p,n-p}(F,+\infty) = F_{3,25-3}(F,+\infty)$$

and the estimate of variance $\hat{\sigma}^2$:

```
stats = 0.9186  124.1102  0.000  1.9952.
```

First of all, we see that high R^2 means that linear model explain most of the variability in the data and small p-value means that we reject the hypothesis that all parameters are equal to zero. On the other hand, simple linear regression showed that carbon monoxide had a positive correlation with nicotine and now we got $\hat{\beta}_3 = -2.6463$. Also, notice that the confidence interval for β_3 is very poor. The reason for this is that tar and nicotine are nearly collinear. Because of this the matrix

$$(X^T X)^{-1} = \begin{pmatrix} 0.3568 & 0.0416 & -0.9408 \\ 0.0416 & 0.0281 & -0.4387 \\ -0.9408 & -0.4387 & 7.1886 \end{pmatrix}$$

has relatively large last diagonal diagonal value. We recall that Theorem gives that the variance of estimate $\hat{\beta}_3$ is $7.1886\sigma^2$ and we also see that the estimate of σ^2 is $\hat{\sigma}^2 = 1.9952$. As a result the confidence interval for β_3 is rather poor.

Of course, looking at linear combinations of tar and nicotine as new predictors does not make sense because they lose their meaning, but for the sake of illustrations let us see what would happen if our predictors were not nearly collinear but, in fact, orthonormal. Let us use economic QR decomposition

```
[X0,R]=qr(X,0)
```

a new matrix of predictor X_0 with orthonormal columns that are some linear combinations of tar and nicotine. Then regressing carbon monoxide on these new predictors

```
 [b,bint,r,rint,stats] = regress(carb,X0);
```

we would get

```
b = -62.6400     bint = -65.5694  -59.7106
    -22.2324             -25.1618  -19.3030
     0.9870              -1.9424    3.9164
```

all confidence intervals of the same relatively better size.

□

Example. The following data presents per capita income of 20 countries for 1960s. Also presented are the percentages of labor force employed in agriculture, industry and service for each country. (Data source: lib.stat.cmu.edu/DASL/Datafiles/oecdat.html)

```
COUNTRY     PCINC AGR IND SER

CANADA       1536  13  43  45
SWEEDEN      1644  14  53  33
SWITZERLAND  1361  11  56  33
LUXEMBOURG   1242  15  51  34
U. KINGDOM   1105  4   56  40
DENMARK      1049  18  45  37
W. GERMANY   1035  15  60  25
FRANCE       1013  20  44  36
BELGUIM      1005  6   52  42
NORWAY       977   20  49  32
ICELAND      839   25  47  29
NETHERLANDS  810   11  49  40
AUSTRIA      681   23  47  30
IRELAND      529   36  30  34
ITALY        504   27  46  28
JAPAN        344   33  35  32
GREECE       324   56  24  20
SPAIN        290   42  37  21
PORTUGAL     238   44  33  23
TURKEY       177   79  12  9
```

We can perform simple linear regression of income on each of the other explanatory variables or multiple linear regression on any pair of the explanatory variables. Fitting simple linear regression of income vs. percent of labor force in agriculture, industry and service:

```
polytool(agr,income,1),
```

etc., produces figure 15.1. Next, we perform statistical inference using 'regress' function. Statistical analysis of linear regression fit of income vs. percent of labor force in agriculture:

```
[b,bint,r,rint,stats]=regress(income,[ones(20,1),agr])

b = 1317.9    bint = 1094.7   1541.1
     -18.9             -26.0   -11.7

stats =   0.6315  30.8472  2.8e-005  74596.
```

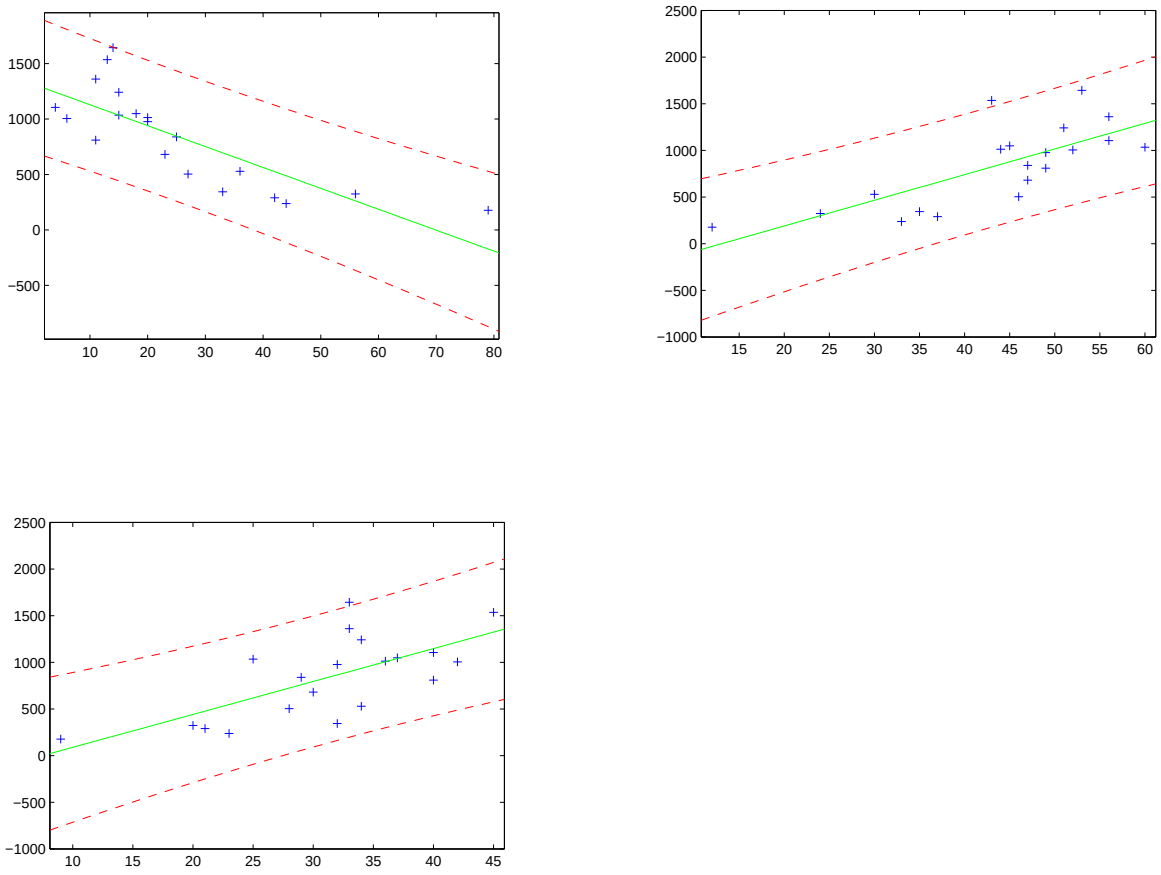


Figure 15.1: Linear regression of income on percent of labor force in agriculture, industry and service.

For income vs. percent of labor force in industry

```
[b,bint,r,rint,stats]=regress(income,[ones(20,1),ind]);

b = -359.3115    bint = -907.1807   188.5577
    27.4905                15.3058   39.6751

stats = 0.5552   22.4677   0.0002   90042
```

and for income vs. labor force in service

```
[b,bint,r,rint,stats]=regress(income,[ones(20,1),serv]);

b = -264.5199     bint =  -858.0257  328.9858
      35.3024                16.8955   53.7093
```

```
stats =   0.4742  16.2355   0.0008   106430.
```

We see that in all three cases, the hypotheses that parameters of least-squares line are both zero can be rejected at conventional level of significance $\alpha = 0.05$. Looking at the confidence intervals for the estimates of slopes we observe that the correlation of income with percent of labor force in agriculture is negative, and other two correlations are positive.

We can also perform a multiple regression on any two explanatory variables. We can not perform multiple linear regression with all three explanatory variables because they add up to 100%, i.e. they are linearly dependent. If we create a predictor matrix

```
X=[ones(20,1),agr,ind];
```

and perform multiple linear regression

```
[b,bint,r,rint,stats]=regress(income,X);
```

we get

```
b = 1272.1    bint = -632.6   3176.9
     -18.4            -39.1      2.3
       0.8            -31.4     32.9

stats =  0.6316  14.5703  0.0002  78972
```

Of course, one can find many shortcomings of this model. For example, having the entire population in agriculture results in prediction of $1272.1 - 1840 < 0$ negative income per capita.

□